Vision par ordinateur et TALN — Support de cours INF1522

Oct 20, 2026 · @df

Introduction

Ce support couvre l'UE INF1522, *Développement d'applications basées sur l'apprentissage automatique*. Il comporte deux parties : la vision par ordinateur (EC1) et le traitement automatique du langage naturel (EC2). Chaque chapitre suit le même plan : notions de cours, illustration, puis exemples de code courts à exécuter et à modifier.

Objectifs de l'UE. À la fin du cours, l'étudiant sait :

- traiter des images et des vidéos avec OpenCV ;
- détecter des objets avec un réseau de neurones (YOLO) ;
- prétraiter et représenter un texte ;
- utiliser des réseaux de neurones pour comprendre et traduire un texte, et évaluer ces modèles avec des métriques adaptées ;
- intégrer un modèle dans une application et la tester.

Installation. Toutes les bibliothèques du cours s'installent en une fois :

```
pip install numpy opencv-python matplotlib ultralytics torch torchvision
datasets
pip install nltk spacy gensim scikit-learn transformers sentence-transformers
sacrebleu
python -m spacy download fr_core_news_sm
```

Conseil : utiliser Jupyter ou Google Colab pour exécuter les exemples cellule par cellule. Dans un notebook, remplacer `cv2.imshow` par `plt.imshow` (voir chapitre 2).

Partie I — Vision par ordinateur

La vision par ordinateur permet à une machine d'extraire de l'information d'images et de vidéos : reconnaître un objet, lire une plaque d'immatriculation, compter des personnes, guider un robot. Cette partie part de l'histoire du domaine, descend jusqu'au pixel, puis remonte progressivement vers les réseaux de neurones et un mini-projet de détection d'objets.

Progression de la partie

Étape	Chapitre	Question à laquelle il répond
Comprendre le contexte	1. Vision par ordinateur, histoire et OpenCV	Qu'est-ce que la vision, et d'où vient-elle ?
Démarrer	2. Installation et premiers programmes	Comment lire, afficher et enregistrer une image ?
Regarder sous le capot	3. Représentation et parcours des pixels	Qu'est-ce qu'une image pour un ordinateur ?
Modifier une image	4. Manipulation et transformations	Comment déplacer, redimensionner, éclaircir ou binariser ?
Extraire de l'information	5. Filtres, convolutions et contours	Comment lisser, rendre plus net et trouver les bords ?
Isoler des objets	6. Segmentation et morphologie	Comment séparer un objet du fond et le compter ?
Reconnaître un objet	7. Identifier un objet dans une image	Couleur, forme ou points caractéristiques : quelle méthode ?
Changer d'approche	8. Introduction à l'apprentissage automatique	Et si la machine trouvait les règles seule ?
Comprendre l'apprentissage	9. Réseaux de neurones et descente de gradient	Comment un réseau corrige-t-il ses erreurs ?
Combiner les deux mondes	10. Deep learning appliqué aux images	Comment OpenCV et les réseaux travaillent-ils ensemble ?
Mettre en pratique	11. Mini-projet YOLO	Comment entraîner son propre détecteur ?

Organisation de chaque chapitre : objectifs, prérequis, notions illustrées, exemples de code, encadrés, exercices corrigés et compétences acquises.

> **À retenir** — encadré qui résume l'idée essentielle d'une section.

> **Erreur fréquente** — encadré qui signale un piège classique et la façon de l'éviter.

> **Question de réflexion** — encadré qui invite à prendre du recul avant de passer à la suite.

Toutes les images d'exemple du cours sont des images de test libres de droits fournies avec la bibliothèque scikit-image (`from skimage import data`). Elles peuvent être remplacées par n'importe quelle photo.

Chapitre 1 — La vision par ordinateur : définition, histoire et apparition d'OpenCV

Objectifs : situer la vision par ordinateur dans son histoire ; comprendre les difficultés du développement avant OpenCV ; savoir ce qu'apporte une bibliothèque comme OpenCV.

Prérequis : aucun.

1.1 Qu'est-ce que la vision par ordinateur ?

Section rédigée d'après R. Szeliski, Computer Vision: Algorithms and Applications, 2e édition, Springer, 2022, chapitre 1.

Une tâche facile pour l'humain. Quand nous regardons un vase de fleurs posé sur une table, nous percevons sans effort sa forme en trois dimensions, la transparence de chaque pétale et la séparation entre chaque fleur et le fond. Devant une photo de groupe, nous comptons les personnes, les reconnaissons et devinons même leurs émotions. Pourtant, après des décennies de recherche, les psychologues de la perception n'ont toujours pas une explication complète du fonctionnement de notre système visuel.

La vision par ordinateur cherche à obtenir le même résultat avec des mathématiques et des programmes : à partir d'une ou plusieurs images, **décrire le monde** qu'elles montrent et **reconstruire ses propriétés** — formes, objets, éclairage, couleurs, mouvements.

Pourquoi est-ce si difficile ? C'est un **problème inverse**. Une caméra fait le chemin direct : la scène en 3D, la lumière et l'objectif produisent une image en 2D. La vision par ordinateur doit faire le chemin inverse, alors qu'une partie de l'information a été perdue : une même image peut provenir de plusieurs scènes différentes (un petit objet proche ou un grand objet lointain, une surface sombre ou une surface claire mal éclairée). Pour choisir la bonne interprétation, on s'appuie sur des modèles physiques, sur les probabilités, ou sur l'**apprentissage à partir de très nombreux exemples** (chapitres 8 à 11).

> **À retenir** — Les débutants sous-estiment souvent la difficulté de la vision. Aux débuts de l'intelligence artificielle, on pensait que raisonner ou jouer aux échecs était plus difficile que « voir ». C'est l'inverse : percevoir est extrêmement complexe, justement parce que nous le faisons sans y penser.

Applications industrielles

Domaine	Exemple
Reconnaissance de caractères (OCR)	Lire les codes postaux manuscrits, lire les plaques d'immatriculation
Inspection industrielle	Contrôler la qualité de pièces, détecter des défauts par rayons X
Commerce	Caisses automatiques qui reconnaissent les articles
Logistique	Robots qui trient et saisissent des colis
Imagerie médicale	Superposer des images avant et pendant une opération ; suivre l'évolution du cerveau
Véhicules autonomes	Repérer la route, les piétons et les panneaux
Photogrammétrie	Construire un modèle 3D à partir de photos aériennes ou de drone
Capture de mouvement	Enregistrer les gestes d'un acteur pour l'animation
Surveillance	Détecter des intrus, analyser le trafic routier
Biométrie	Reconnaissance d'empreintes digitales et de visages

Applications grand public : assemblage de photos en panorama ; fusion de plusieurs expositions (HDR) ; morphing d'un visage à un autre ; modélisation 3D à partir de photos ; stabilisation de vidéos ; détection de visages pour la mise au point de l'appareil photo ; déverrouillage d'un téléphone par le visage.

Vision par ordinateur et traitement d'images : quelle différence ?



	Traitement d'images	Vision par ordinateur
Entrée	Une image	Une ou plusieurs images, une vidéo
Sortie	Une autre image (améliorée, filtrée, compressée)	Une information : objets, positions, mesures, forme 3D, décision
Question posée	Comment modifier l'image ?	Que montre l'image ?
Exemples	Flou, contraste, détection de contours, compression	Reconnaissance de visages, détection d'objets, reconstruction 3D
Dans ce cours	Chapitres 3 à 6	Chapitres 6 à 11

Les deux domaines sont liés : la vision utilise presque toujours le traitement d'images comme **première étape** (nettoyer, convertir, trouver les contours) avant d'interpréter la scène.

1.2 Les débuts : calculer directement sur les pixels

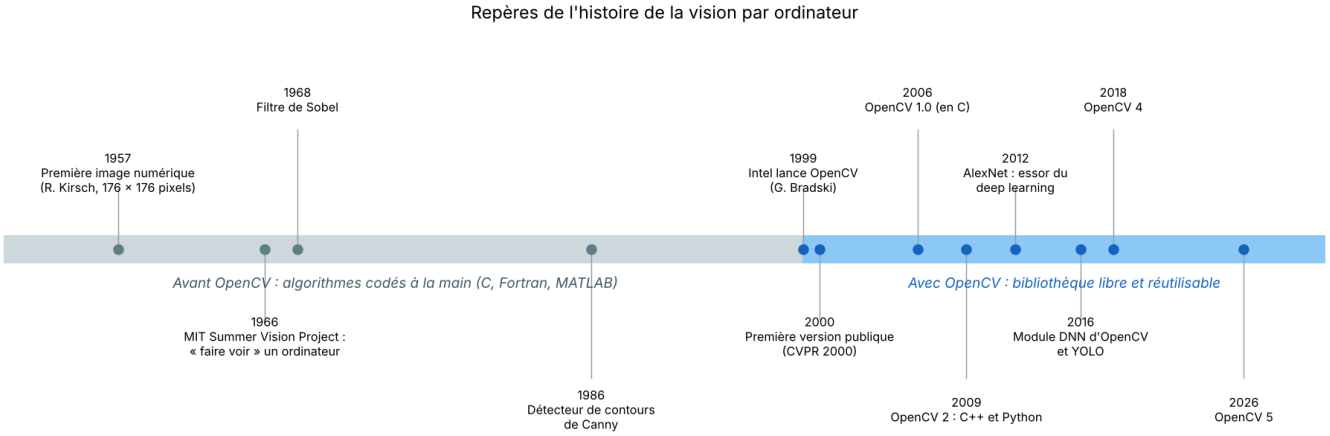
En 1957, Russell Kirsch et son équipe du National Bureau of Standards (États-Unis) numérisent la photo de son fils de trois mois avec un scanner relié à l'ordinateur SEAC. L'image fait 176 × 176 pixels : c'est l'une des toutes premières images numériques. Le principe de l'image numérique est posé dès cette époque : une grille de nombres, un nombre par point.

Dans les années 1960, les chercheurs tentent de faire « voir » les ordinateurs. En 1966, le *Summer Vision Project* du MIT propose à des étudiants de construire en un été un système capable de reconnaître des objets. Le projet montre surtout que le problème est beaucoup plus difficile que prévu.

Les décennies suivantes produisent les algorithmes qu'on utilise encore aujourd'hui, et que ce cours présente :

Année	Algorithme	Utilisé dans le cours
1968	Filtre de Sobel (dérivées de l'image)	Chapitre 5
1979	Méthode d'Otsu (seuil automatique)	Chapitre 4
années 1960–1980	Morphologie mathématique (érosion, dilatation)	Chapitre 6

Année	Algorithme	Utilisé dans le cours
1986	Détecteur de contours de Canny	Chapitre 5
2001	Détection de visages de Viola et Jones (cascades de Haar)	Chapitre 8



1.3 Avant OpenCV : tout coder soi-même

Avant les années 2000, un chercheur ou un ingénieur qui voulait traiter une image devait presque tout programmer lui-même, en général en C, en C++ ou en Fortran, parfois avec des logiciels scientifiques comme MATLAB. Il fallait :

- lire le fichier image octet par octet en respectant son format (BMP, PGM…) ;
- écrire chaque algorithme à la main avec des boucles sur les pixels ;
- gérer les bords de l'image, la mémoire et les conversions de type ;
- optimiser le code pour qu'il ne soit pas trop lent ;
- recommencer dans chaque laboratoire, car il n'existait pas de bibliothèque libre et standard.

Exemple : un flou moyenneur 3 × 3 écrit en C, comme avant OpenCV.

```
/* Flou moyenneur 3 x 3 sur une image en niveaux de gris.
   src et dst sont des tableaux de largeur x hauteur octets. */
void flou3x3(const unsigned char *src, unsigned char *dst,
             int largeur, int hauteur)
{
    for (int y = 1; y < hauteur - 1; y++) {          /* lignes  */
        for (int x = 1; x < largeur - 1; x++) {      /* colonnes */
            int somme = 0;
            for (int j = -1; j <= 1; j++)            /* voisinage 3 x 3 */
                for (int i = -1; i <= 1; i++)
                    somme += src[(y + j) * largeur + (x + i)];
            dst[y * largeur + x] = somme / 9;
        }
    }
    /* Il reste à traiter les bords, à lire et écrire le fichier image,
       et à gérer les images couleur... */
}
```

Le même traitement avec OpenCV (Python) :

```
import cv2

img = cv2.imread("photo.jpg")       # lecture de n'importe quel format
courant
flou = cv2.blur(img, (3, 3))        # bords, couleurs et optimisation gérés
cv2.imwrite("flou.jpg", flou)
```

	Avant OpenCV	Avec OpenCV
Lire une image	Décoder le format à la main	`cv2.imread`
Écrire un filtre	Quatre boucles imbriquées	Une fonction
Bords de l'image	À gérer soi-même	Gérés automatiquement
Performance	Dépend du programmeur	Code C++ optimisé
Partage	Chaque labo a son code	Bibliothèque commune et libre

1.4 La naissance d'OpenCV

En 1999, chez Intel, une équipe animée par Gary Bradski lance une bibliothèque de vision par ordinateur. Son but est d'accélérer la recherche et l'industrie en offrant une base commune, libre et optimisée. Les grandes étapes sont les suivantes :

- **2000** : première version publique, présentée à la conférence CVPR ;

- **2006** : version 1.0, écrite en C ;
- **2009** : version 2, interface C++ et accès depuis Python ;
- **2012** : la fondation à but non lucratif OpenCV.org prend le relais ;
- **2015 et 2018** : versions 3 et 4 ; un module de réseaux de neurones (DNN) apparaît fin 2016 ;
- **2026** : version 5, utilisée pour vérifier les exemples de ce cours.

Aujourd'hui, OpenCV regroupe plus de 2 500 algorithmes, est utilisable en Python, C++ et Java, et fonctionne sur ordinateur, téléphone et cartes embarquées.

> **À retenir** — OpenCV n'a pas inventé la vision par ordinateur. Il a rassemblé dans une même bibliothèque libre des algorithmes publiés depuis des décennies (Sobel, Otsu, Canny…), en les rendant rapides, fiables et faciles à réutiliser.

1.5 La vision par ordinateur aujourd'hui

Domaine	Exemple d'application	Techniques du cours
Photographie	Réglage automatique de l'exposition, flou d'arrière-plan	Histogramme, flous
Médecine	Repérage de cellules ou de tumeurs sur des images médicales	Seuillage, segmentation
Robotique	Un robot repère et saisit un objet	Contours, formes, détection d'objets
Vidéosurveillance	Comptage de personnes, détection d'intrusion	Vidéo, YOLO
Conduite autonome	Détection des lignes, piétons et panneaux	Contours, réseaux de neurones
Industrie	Contrôle qualité des pièces sur une chaîne	Template matching, comptage, points caractéristiques

> **Question de réflexion** — Pourquoi reconnaître un chat est-il facile pour un enfant de trois ans, mais difficile pour un ordinateur ? Pensez à tout ce qui peut changer d'une photo à l'autre : lumière, angle, distance, arrière-plan, race du chat.

Exercices

1. Citez trois difficultés rencontrées par les programmeurs avant l'apparition d'OpenCV.

2. Dans le code C ci-dessus, combien d'additions faut-il pour flouter une image de 1 000 × 1 000 pixels ?
3. Choisissez une application de la vision par ordinateur de votre quotidien et décrivez les étapes qu'elle doit réaliser.

Chapitre 2 — Installation et premiers programmes

Objectifs : installer l'environnement ; lire, afficher, dessiner et enregistrer une image ; lire une vidéo ou une webcam.

Prérequis : bases de Python (variables, `if`, boucles `for` et `while`).

2.1 Installer et vérifier

```
pip install opencv-python numpy matplotlib
```

```
import cv2
import numpy as np

print("OpenCV :", cv2.__version__)
print("NumPy  :", np.__version__)
```

Si les deux versions s'affichent, l'installation est correcte. Les exemples de ce cours ont été vérifiés avec OpenCV 5.0 et fonctionnent aussi avec OpenCV 4.

Bibliothèque	Rôle
`opencv-python` (import `cv2`)	Lire, transformer, analyser images et vidéos
`numpy`	Représenter une image comme un tableau de nombres
`matplotlib`	Afficher des images et des graphiques, surtout dans un notebook

2.2 Premier programme : lire, afficher, enregistrer

```
import cv2

img = cv2.imread("photo.jpg")          # 1. lire l'image

if img is None:                         # 2. vérifier la lecture
    print("Erreur : image introuvable")
else:
    print("Taille :", img.shape)        # 3. afficher ses dimensions
    cv2.imshow("Ma photo", img)         # 4. l'afficher dans une fenêtre
    cv2.waitKey(0)                      # 5. attendre une touche
    cv2.destroyAllWindows()
    cv2.imwrite("copie.png", img)       # 6. l'enregistrer dans un autre
format
```

> **Erreur fréquente** — `cv2.imread` ne provoque **aucune erreur** si le fichier n'existe pas : elle renvoie `None`. L'erreur apparaît plus loin, avec un message peu clair. Il faut donc toujours tester `if img is None`.

2.3 Afficher dans un notebook (Jupyter, Colab)

Les fenêtres `cv2.imshow` ne fonctionnent pas dans un notebook. On utilise Matplotlib, sans oublier de convertir l'ordre des couleurs (voir chapitre 3).

```
import cv2
import matplotlib.pyplot as plt

img = cv2.imread("photo.jpg")
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)   # BGR -> RGB pour matplotlib

plt.imshow(img_rgb)
plt.title("Ma photo")
plt.axis("off")
plt.show()
```

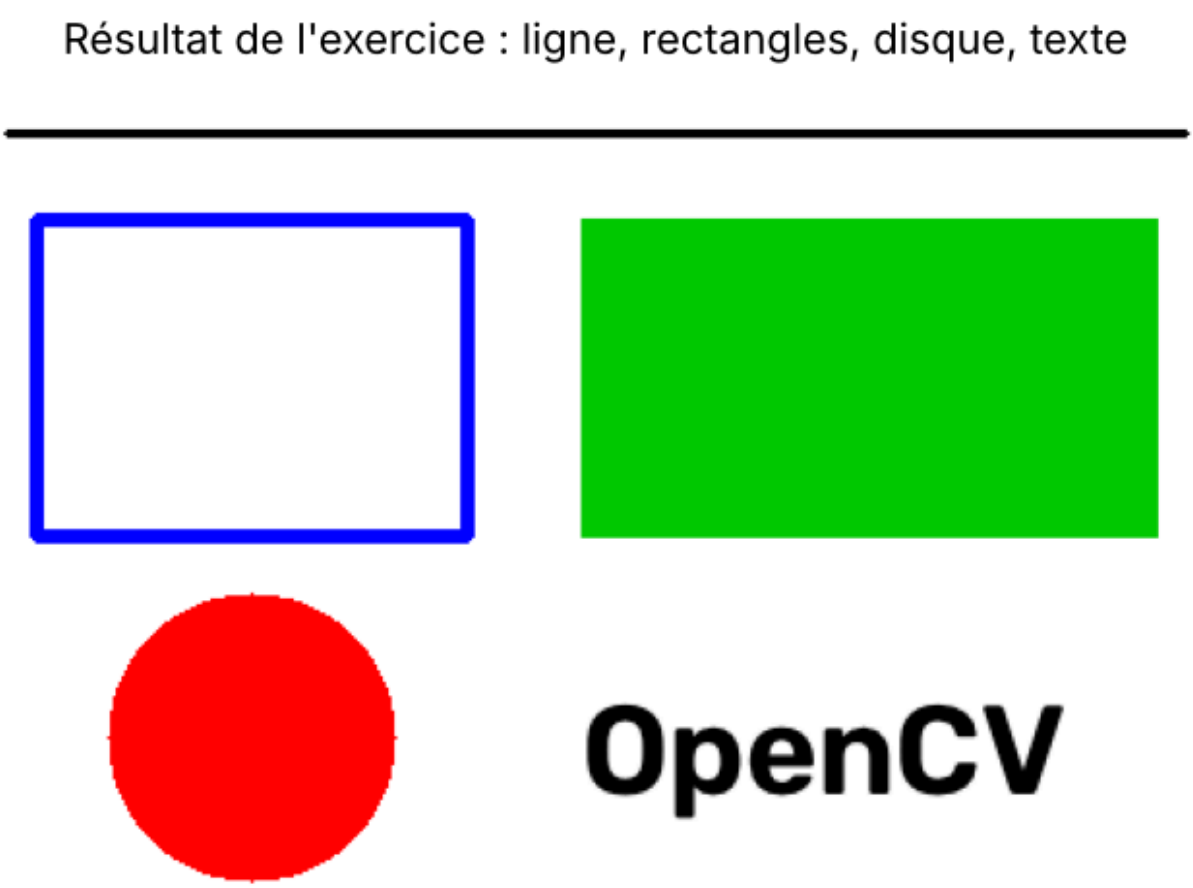
2.4 Dessiner sur une image

```
import numpy as np
import cv2

img = np.full((300, 450, 3), 255, dtype=np.uint8)   # fond blanc

cv2.line(img, (20, 20), (430, 20), (0, 0, 0), 2)
cv2.rectangle(img, (30, 50), (180, 160), (255, 0, 0), 3)
cv2.rectangle(img, (220, 50), (420, 160), (0, 200, 0), -1)   # -1 = rempli
cv2.circle(img, (105, 230), 50, (0, 0, 255), -1)
cv2.putText(img, "OpenCV", (220, 250), cv2.FONT_HERSHEY_SIMPLEX, 1.5, (0, 0,
0), 3)

cv2.imwrite("formes.png", img)
```



Les couleurs sont données dans l'ordre (Bleu, Vert, Rouge) : `(255, 0, 0)` est donc du **bleu**, et `(0, 0, 255)` du **rouge**. Les positions sont données en (x, y), c'est-à-dire (colonne, ligne).

2.5 Lire une vidéo ou une webcam

Une vidéo est une suite d'images. On les lit une par une dans une boucle.

```
import cv2

camera = cv2.VideoCapture(0)          # 0 = webcam ; ou "video.mp4"
while True:
    ok, image = camera.read()         # ok vaut False s'il n'y a plus d'image
    if not ok:
        break
    cv2.imshow("Webcam", image)
    if cv2.waitKey(1) == ord("q"):    # touche q pour quitter
        break
camera.release()
cv2.destroyAllWindows()
```

> **À retenir** — Un programme OpenCV suit presque toujours le même schéma : **lire** l'image (ou la vidéo), la **traiter**, puis **afficher** ou **enregistrer** le résultat.

Exercices

1. Lire une photo, afficher ses dimensions, puis l'enregistrer au format PNG.
2. Dessiner le drapeau du Bénin (300 × 450 pixels) : une bande verte verticale à gauche (40 % de la largeur), une bande jaune en haut à droite et une bande rouge en bas à droite.
3. Mini-défi : afficher la webcam en ajoutant le texte « Cours INF1522 » sur chaque image.

Corrigés

1. `img = cv2.imread("photo.jpg")`, puis `print(img.shape)`, puis `cv2.imwrite("photo.png", img)`.
2. Avec des rectangles remplis (couleurs en BGR : vert `(0, 135, 0)`, jaune `(0, 210, 255)`, rouge `(0, 0, 230)`) :

```
import numpy as np
import cv2

drapeau = np.zeros((300, 450, 3), dtype=np.uint8)
cv2.rectangle(drapeau, (0, 0), (179, 299), (0, 135, 0), -1)      # vert
cv2.rectangle(drapeau, (180, 0), (449, 149), (0, 210, 255), -1)  # jaune
cv2.rectangle(drapeau, (180, 150), (449, 299), (0, 0, 230), -1)  # rouge
cv2.imwrite("benin.png", drapeau)
```

3. Dans la boucle de la section 2.5, ajouter avant `cv2.imshow` : `cv2.putText(image, "Cours INF1522", (20, 40), cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 255, 0), 2)`.

Compétences acquises : installer OpenCV ; lire, afficher, enregistrer une image ; dessiner ; lire un flux vidéo.

Chapitre 3 — Représentation numérique et parcours des pixels

Objectifs : comprendre comment un ordinateur stocke une image ; maîtriser l'indexation `img[y, x]` ; parcourir une image avec des boucles puis sans boucle ; éviter les pièges de type de données.

Prérequis : chapitre 2 ; notion de tableau (liste de listes).

3.1 Le pixel et la matrice

Un **pixel** (de l'anglais *picture element*) est le plus petit point d'une image. Une image numérique est une **matrice** de pixels : un tableau à deux dimensions avec H lignes (la hauteur) et W colonnes (la largeur). Chaque case contient un nombre qui indique l'intensité lumineuse du pixel.

Pour une image en niveaux de gris codée sur 8 bits, chaque nombre va de **0 (noir)** à **255 (blanc)**, soit $2^8 = 256$ niveaux possibles.



Trois points à observer sur ce schéma :

- l'**origine** (0, 0) est en **haut à gauche**, et non en bas à gauche comme en mathématiques ;
- l'axe **x** va vers la **droite** (colonnes) et l'axe **y** va vers le **bas** (lignes) ;

- un programme parcourt l'image **ligne par ligne** : de gauche à droite, puis de haut en bas, comme on lit un texte.

Représentation mathématique d'une image. Mathématiquement, une image en niveaux de gris est une **fonction** f qui associe à chaque position (x, y) une intensité :

```
f : (x, y)  →  f(x, y)        avec 0 ⩽ x < W,  0 ⩽ y < H,  0 ⩽ f(x, y) ⩽ 255

Exemple : f(5, 2) = 220   (le pixel de la colonne 5, ligne 2 du schéma)
```

Une image couleur est une fonction qui renvoie trois valeurs : f(x, y) = (B, G, R).

La scène réelle est continue : la lumière varie en tout point et peut prendre n'importe quelle intensité. Pour la rendre numérique, l'appareil photo effectue deux opérations :

- l'**échantillonnage** : on ne mesure la lumière qu'en un nombre fini de points, les pixels (plus il y en a, plus la **résolution** est grande) ;
- la **quantification** : on arrondit chaque mesure à un nombre entier, ici de 0 à 255 (8 bits).

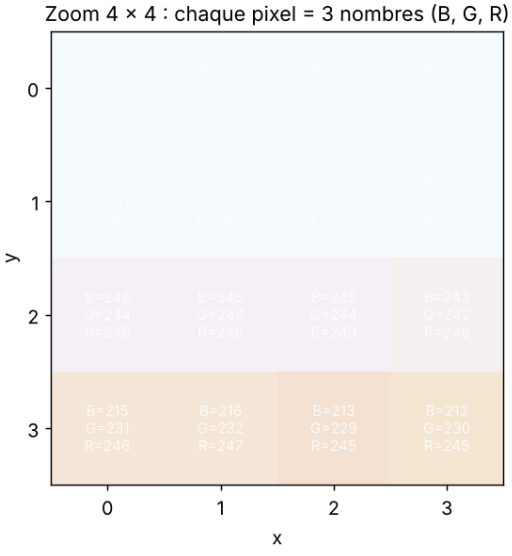
Voir l'image comme une fonction est très utile : **transformer une image, c'est appliquer une opération mathématique à cette fonction** (chapitre 4).

3.2 Niveaux de gris et couleur

Type d'image	Forme du tableau	Valeur d'un pixel	Exemple
Niveaux de gris	(H, W)	Un nombre de 0 à 255	`128` (gris moyen)
Couleur	(H, W, 3)	Trois nombres (B, G, R)	`(0, 0, 255)` (rouge)

Une image couleur superpose trois **canaux** : un pour le bleu, un pour le vert, un pour le rouge. Chaque couleur est obtenue par mélange de ces trois lumières.







> **Erreur fréquente** — OpenCV range les canaux dans l'ordre **BGR** (bleu, vert, rouge), alors que Matplotlib et la plupart des autres outils attendent **RGB**. Une image OpenCV affichée directement avec Matplotlib a ses rouges et ses bleus inversés. Solution : `cv2.cvtColor(img, cv2.COLOR_BGR2RGB)`.

3.3 Dimensions, canaux et type de données

```
import cv2

img = cv2.imread("photo.jpg")
gris = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

print("Couleur :", img.shape, img.dtype)     # (400, 600, 3) uint8
print("Gris    :", gris.shape, gris.dtype)   # (400, 600) uint8
print("Nombre de pixels :", gris.size)       # 400 x 600 = 240000
print("Min et max :", gris.min(), gris.max())
```

Attribut	Signification	Exemple
`img.shape`	(hauteur, largeur) ou (hauteur, largeur, canaux)	`(400, 600, 3)`
`img.dtype`	Type de chaque valeur	`uint8` : entier non signé sur 8 bits, de 0 à 255
`img.size`	Nombre total de valeurs	400 × 600 × 3 = 720 000

3.4 Accéder à un pixel : img[y, x]

En géométrie, on donne un point par (x, y). Mais un tableau NumPy s'indexe par **[ligne, colonne]**, c'est-à-dire **[y, x]**.

```
import cv2

img = cv2.imread("photo.jpg")
x, y = 200, 100

b, g, r = img[y, x]                 # attention : [ligne, colonne] = [y, x]
print("Pixel (x=200, y=100) : B =", b, " G =", g, " R =", r)

img[y, x] = (0, 0, 255)             # le pixel devient rouge
img[0:50, 0:100] = (255, 0, 0)      # un bloc de 50 lignes x 100 colonnes
devient bleu
```

Contexte	Ordre des coordonnées	Exemple
Tableau NumPy	`[y, x]` (ligne, colonne)	`img[100, 200]`
Fonctions de dessin OpenCV	`(x, y)` (colonne, ligne)	`cv2.circle(img, (200, 100), 5, ...)`
`img.shape`	`(hauteur, largeur)`	`(400, 600, 3)`
`cv2.resize`	`(largeur, hauteur)`	`cv2.resize(img, (600, 400))`

> **À retenir** — Pour lire ou modifier un pixel : `img[y, x]`. Pour dessiner ou redimensionner : `(x, y)` et `(largeur, hauteur)`. Cette inversion est la source d'erreur la plus fréquente chez les débutants.

3.5 Parcourir l'image avec des boucles

Pour comprendre ce que fait un algorithme, on peut parcourir l'image pixel par pixel. Exemple : calculer le **négatif** d'une image grise (chaque valeur v devient 255 − v).

```
import cv2
import numpy as np

gris = cv2.imread("photo.jpg", cv2.IMREAD_GRAYSCALE)
hauteur, largeur = gris.shape
negatif = np.zeros_like(gris)

for y in range(hauteur):            # de haut en bas (lignes)
    for x in range(largeur):        # de gauche à droite (colonnes)
        negatif[y, x] = 255 - gris[y, x]
```

La boucle extérieure porte sur les lignes (y), la boucle intérieure sur les colonnes (x) : c'est exactement le sens de parcours du schéma de la section 3.1.

3.6 Le traitement vectorisé avec NumPy

NumPy sait appliquer une opération à **tous les pixels à la fois**, sans boucle Python : c'est le **traitement vectorisé**. Le calcul est alors fait par du code compilé, beaucoup plus rapide.

```
import time
import cv2
import numpy as np

gris = cv2.imread("photo.jpg", cv2.IMREAD_GRAYSCALE)
h, w = gris.shape

debut = time.perf_counter()
negatif1 = np.zeros_like(gris)
for y in range(h):
    for x in range(w):
        negatif1[y, x] = 255 - gris[y, x]
t_boucles = time.perf_counter() - debut

debut = time.perf_counter()
negatif2 = 255 - gris                  # une seule ligne, sans boucle
t_numpy = time.perf_counter() - debut

print("Boucles :", round(t_boucles * 1000, 1), "ms")
print("NumPy   :", round(t_numpy * 1000, 3), "ms")
print("Résultats identiques :", np.array_equal(negatif1, negatif2))
```

Mesures réelles obtenues sur une image de 400 × 600 pixels (240 000 pixels) ; les valeurs exactes dépendent de l'ordinateur :

Méthode	Temps mesuré	Lignes de code
Double boucle Python	environ 60 à 80 ms	4
NumPy vectorisé (`255 - gris`)	environ 0,01 à 0,1 ms	1
OpenCV (`cv2.bitwise_not(gris)`)	environ 0,03 ms	1

Le calcul vectorisé est ici des centaines, voire des milliers de fois plus rapide. Sur une vidéo de 30 images par seconde, la double boucle serait trop lente pour suivre le rythme.

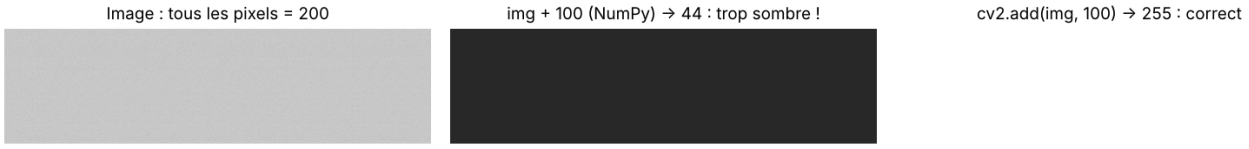
> **À retenir** — Les boucles servent à **comprendre** un algorithme. En pratique, on utilise les opérations NumPy ou les fonctions OpenCV, qui travaillent sur toute l'image d'un coup.

3.7 Piège : le débordement des entiers uint8

Un pixel `uint8` ne peut pas dépasser 255. Avec NumPy, un calcul qui dépasse « fait le tour » et repart de 0 ; avec OpenCV, la valeur est bloquée à 255 (on parle de **saturation**).

```
import numpy as np
import cv2

gris = np.full((2, 3), 200, dtype=np.uint8)   # petite image grise : tous les pixels valent 200
print(gris + 100)           # 44 partout : 300 - 256 = 44, le calcul « fait le tour »
print(cv2.add(gris, 100))   # 255 partout : OpenCV bloque à 255 (saturation)
```



> **Erreur fréquente** — Éclaircir une image avec `img + 100` donne des taches sombres là où l'image était claire. Utiliser `cv2.add`, `cv2.convertScaleAbs` (chapitre 4) ou convertir d'abord en entiers plus grands.

Exercices

1. Une image couleur a pour forme `(480, 640, 3)`. Quelles sont sa largeur et sa hauteur ? Combien contient-elle de valeurs ?
2. Quelle instruction lit la valeur du pixel situé à la colonne 50 et à la ligne 20 d'une image grise `gris` ?
3. Que vaut `np.uint8(250) + np.uint8(10)` ? Et `cv2.add(np.uint8([[250]]), np.uint8([[10]]))` ?
4. Mini-défi : avec une double boucle, puis avec NumPy, mettre à 0 tous les pixels dont la valeur est inférieure à 100. Comparer les temps.

Corrigés

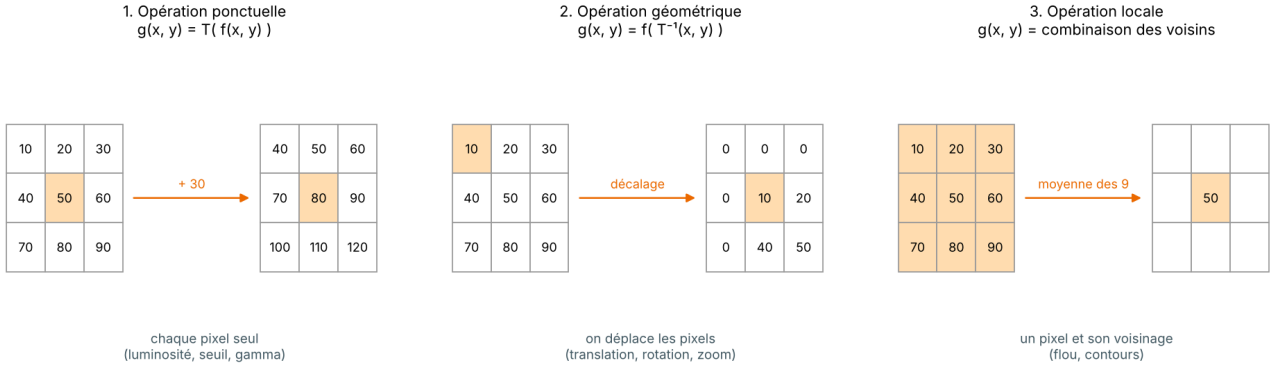
1. Hauteur ..., largeur... Nombre de valeurs : 480 × 640 × 3 = 921 600.
2. `gris[20, 50]` : d'abord la ligne (y = 20), puis la colonne (x = 50).

3. Avec NumPy : 260 − 256 = **4** (débordement ; NumPy affiche aussi un avertissement « overflow »). Avec OpenCV : **255** (saturation).
4. Version vectorisée en une ligne : `resultat = gris.copy()` puis `resultat[resultat < 100] = 0`. L'expression `resultat < 100` crée un tableau de vrai/faux, appelé **masque booléen**, qui sélectionne les pixels à modifier.

Chapitre 4 — Manipulation et transformations d'images

Objectifs : déplacer et déformer une image ; modifier sa luminosité et son contraste ; changer d'espace colorimétrique ; binariser une image ; combiner des images avec les opérations bit à bit

Comment fait-on une transformation mathématique sur une image ? Puisqu'une image est une fonction f(x, y) (chapitre 3), la transformer consiste à calculer une nouvelle image g(x, y) à partir de f. Selon ce que le calcul utilise, on distingue trois familles :



Famille	Formule	Ce qui change	Exemples
Ponctuelle (photométrique)	g(x, y) = T(f(x, y))	La **valeur** de chaque pixel, calculée à partir de lui seul	Luminosité, contraste, gamma, seuillage
Géométrique	g(x, y) = f(T^{-1}(x, y))	La **position** des pixels	Translation, rotation, redimensionnement
Locale (voisinage)	g(x, y) = combinaison de f autour de (x, y)	La valeur, calculée à partir des **voisins**	Flou, détection de contours

Pour une transformation géométrique, OpenCV raisonne « à l'envers » : pour chaque pixel (x, y) de l'image d'arrivée, il cherche d'où il vient dans l'image de départ, avec la transformation inverse T^{-1}. Ainsi, aucun pixel d'arrivée ne reste vide par oubli.

4.1 Transformations géométriques

Intuition : on déplace chaque pixel vers une nouvelle position, comme si l'on faisait glisser, tourner ou étirer une photo imprimée.

Principe mathématique. Une translation ou une rotation se décrit par une petite matrice M de taille 2 × 3. Le pixel (x, y) va en (x′, y′) :

```
Translation de (tx, ty)            Rotation d'un angle θ autour de l'origine
x' = x + tx                        x' = x · cos θ − y · sin θ
y' = y + ty                        y' = x · sin θ + y · cos θ

M = [ 1  0  tx ]                   M = [ cos θ  −sin θ  0 ]
    [ 0  1  ty ]                       [ sin θ   cos θ  0 ]
```

Pour tourner autour du centre de l'image, OpenCV calcule lui-même la bonne matrice avec `cv2.getRotationMatrix2D`.

Transformation	Fonction OpenCV	Paramètres principaux
Recadrage	découpage NumPy `img[y1:y2, x1:x2]`	Lignes puis colonnes
Redimensionnement	`cv2.resize`	(largeur, hauteur) ou facteurs `fx`, `fy` ; méthode d'interpolation
Translation	`cv2.warpAffine`	Matrice M, taille de sortie
Symétrie (miroir)	`cv2.flip`	1 = horizontale, 0 = verticale, −1 = les deux
Rotation	`cv2.getRotationMatrix2D` + `cv2.warpAffine`	Centre, angle en degrés, échelle
Perspective	`cv2.getPerspectiveTransform` + `cv2.warpPerspective`	4 points de départ, 4 points d'arrivée

```
import cv2
import numpy as np

img = cv2.imread("photo.jpg")
h, w = img.shape[:2]

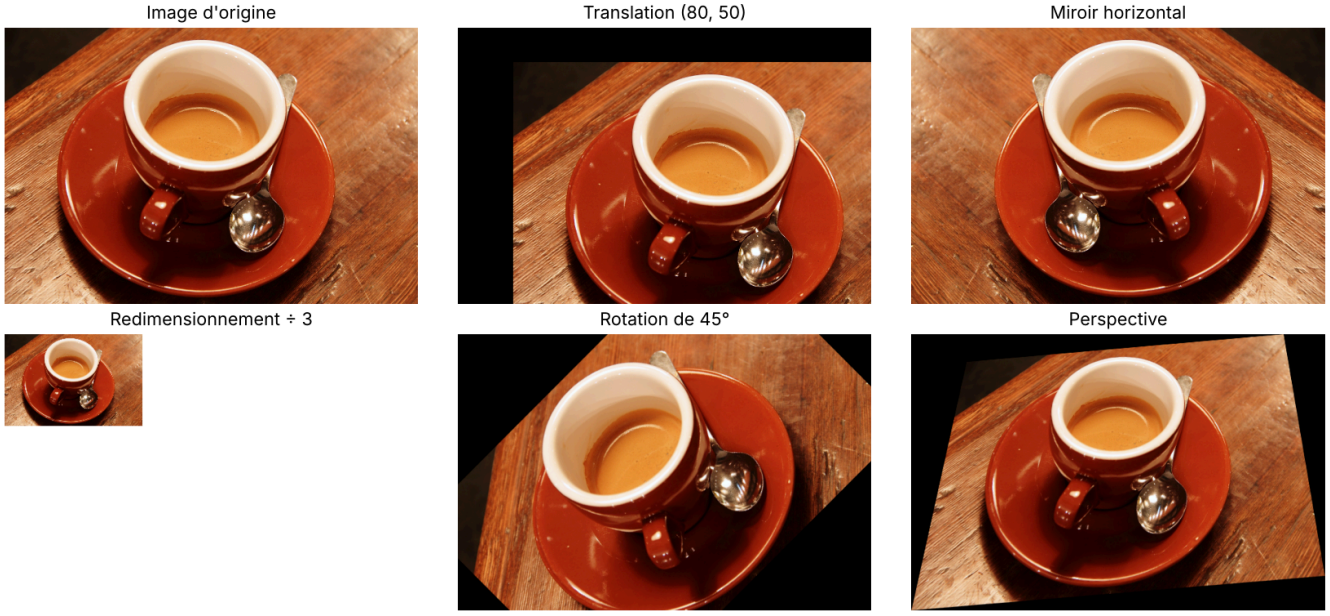
recadree = img[100:300, 150:450]                      # [lignes, colonnes]
petite = cv2.resize(img, (w // 3, h // 3))           # (largeur, hauteur)
grande = cv2.resize(img, None, fx=2, fy=2, interpolation=cv2.INTER_LINEAR)
```

```
M = np.float32([[1, 0, 80],                              # translation : 80 px à
droite
                [0, 1, 50]])                             #               50 px
vers le bas
decalee = cv2.warpAffine(img, M, (w, h))

miroir = cv2.flip(img, 1)                                # symétrie horizontale

M = cv2.getRotationMatrix2D((w / 2, h / 2), 45, 1)   # centre, angle, échelle
tournee = cv2.warpAffine(img, M, (w, h))

coins = np.float32([[0, 0], [w, 0], [0, h], [w, h]])
nouveaux = np.float32([[80, 40], [w - 60, 0], [0, h], [w, h - 50]])
M = cv2.getPerspectiveTransform(coins, nouveaux)
perspective = cv2.warpPerspective(img, M, (w, h))
```



Interprétation : les zones noires apparaissent là où aucun pixel de l'image d'origine n'arrive. Lors d'un agrandissement, OpenCV doit **inventer** les pixels manquants par **interpolation** : il les calcule à partir des pixels voisins.

Limites : agrandir une image ne crée pas de détails, cela la rend plus floue ; une rotation fait perdre les coins de l'image.

4.2 Redimensionner : agrandir ou réduire une image

Intuition : agrandir une image de 100 × 100 à 400 × 400 pixels oblige à **inventer** 150 000 pixels qui n'existaient pas. Réduire oblige à **résumer** plusieurs pixels en un seul. La façon de calculer ces pixels s'appelle l'**interpolation**.

Principe mathématique. Pour un agrandissement d'un facteur s, le pixel (x, y) de la nouvelle image vient de la position (x / s, y / s) de l'ancienne. Cette position tombe en général **entre** quatre pixels A, B, C, D.

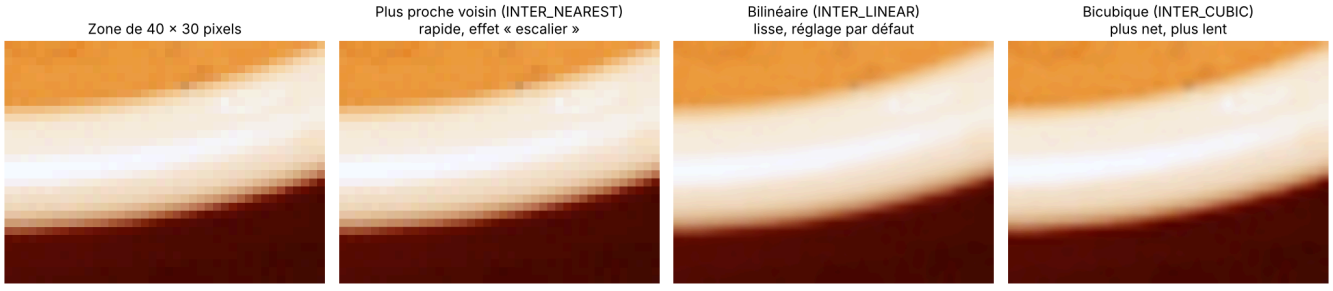
Méthode	Fonction	Principe	Usage conseillé
Plus proche voisin	`cv2.INTER_NEAREST`	Copier la valeur du pixel le plus proche	Masques et étiquettes (ne crée pas de nouvelles valeurs)
Bilinéaire	`cv2.INTER_LINEAR` (par défaut)	Moyenne pondérée des 4 voisins, selon la distance	Agrandissement courant
Bicubique	`cv2.INTER_CUBIC`	Utilise 16 voisins ; résultat plus net	Agrandissement de qualité
Par zone	`cv2.INTER_AREA`	Moyenne de tous les pixels couverts	**Réduction**

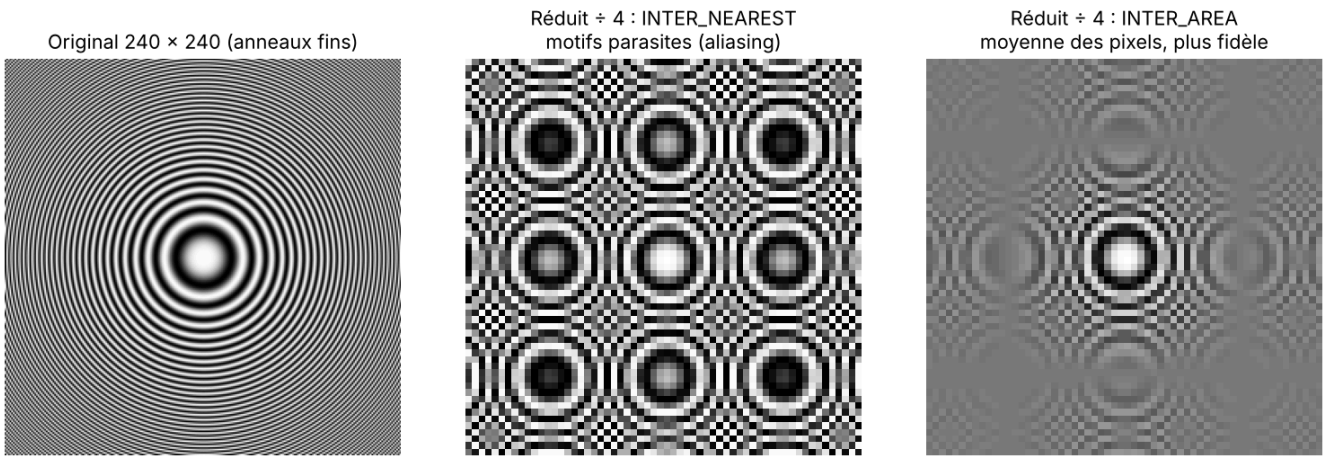
```
import cv2

img = cv2.imread("photo.jpg")

# Agrandir 4 fois avec trois méthodes
plus_proche = cv2.resize(img, None, fx=4, fy=4,
interpolation=cv2.INTER_NEAREST)
bilineaire = cv2.resize(img, None, fx=4, fy=4,
interpolation=cv2.INTER_LINEAR)
bicubique = cv2.resize(img, None, fx=4, fy=4, interpolation=cv2.INTER_CUBIC)

# Réduire 4 fois : INTER_AREA est recommandé
reduite = cv2.resize(img, None, fx=0.25, fy=0.25,
interpolation=cv2.INTER_AREA)
print(bilineaire.shape, reduite.shape)
```





Interprétation : en agrandissement, le plus proche voisin donne des « escaliers », le bilinéaire et le bicubique des transitions douces. En réduction, sauter des pixels (plus proche voisin) fait apparaître de faux motifs : c'est le **repliement** (*aliasing*). `INTER_AREA` fait la moyenne des pixels et l'évite.

> **Erreur fréquente** — Croire qu'agrandir une image la rend plus détaillée. L'interpolation ne fait que calculer des valeurs intermédiaires : aucun détail absent de l'image d'origine ne peut apparaître. C'est pourquoi « zoomer et améliorer » comme dans les séries policières est impossible avec ces méthodes (voir aussi la section 5.6).

4.3 Transformations photométriques : luminosité, contraste, gamma

Intuition : on garde les pixels à leur place, mais on change leur valeur, comme les réglages « luminosité » et « contraste » d'un téléphone.

Principe mathématique. Chaque pixel f devient g :

$$g(x, y) = \alpha \cdot f(x, y) + \beta \qquad (\alpha : \text{contraste}, \ \beta : \text{luminosité})$$

$$g = 255 \times \left(\frac{f}{255}\right)^{\gamma} \qquad \text{(correction gamma)}$$

- β > 0 éclaircit l'image, β < 0 l'assombrit ;
- α > 1 augmente le contraste (les écarts entre pixels sont agrandis), α < 1 le diminue ;
- γ < 1 éclaircit surtout les zones sombres, γ > 1 les assombrit.

L'**histogramme** d'une image compte combien de pixels ont chaque valeur de 0 à 255.

C'est un graphique qui indique combien de pixels possèdent chaque niveau de luminosité ou chaque valeur de couleur dans une image.

Il montre d'un coup d'œil si l'image est sombre, claire ou peu contrastée. L'**égalisation d'histogramme** étale automatiquement les valeurs sur toute la plage.

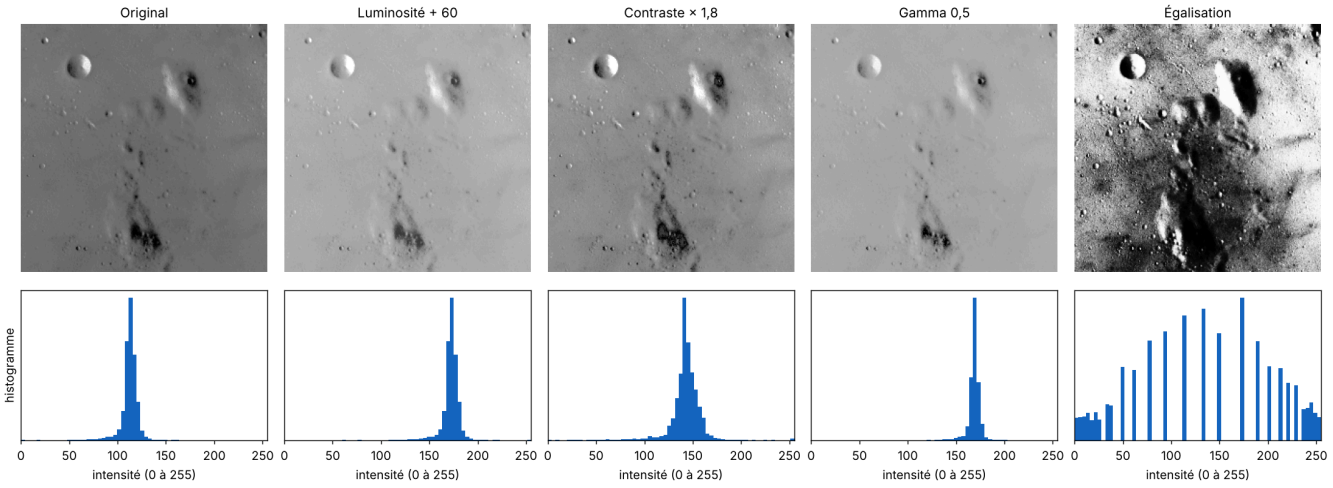
```
import cv2
import numpy as np

gris = cv2.imread("photo.jpg", cv2.IMREAD_GRAYSCALE)

plus_clair = cv2.convertScaleAbs(gris, alpha=1.0, beta=60)       # luminosité
+ 60
plus_contraste = cv2.convertScaleAbs(gris, alpha=1.8, beta=-60)  # contraste
x 1,8

gamma = 0.5
corrigee = (255 * (gris / 255) ** gamma).astype(np.uint8)        # correction
gamma

egalisee = cv2.equalizeHist(gris)                                #
égalisation
```



Interprétation : la luminosité décale l'histogramme vers la droite sans l'élargir ; le contraste l'élargit ; l'égalisation l'étale sur toute la plage et fait ressortir des détails invisibles au départ. `cv2.convertScaleAbs` bloque automatiquement les résultats entre 0 et 255.

Limites : une zone déjà blanche (255) ou noire (0) a perdu ses détails : aucun réglage ne peut les retrouver. L'égalisation peut aussi amplifier le bruit.

4.4 Espaces colorimétriques : gris, BGR, RGB et HSV

Un **espace colorimétrique** est une façon de décrire une couleur par des nombres.

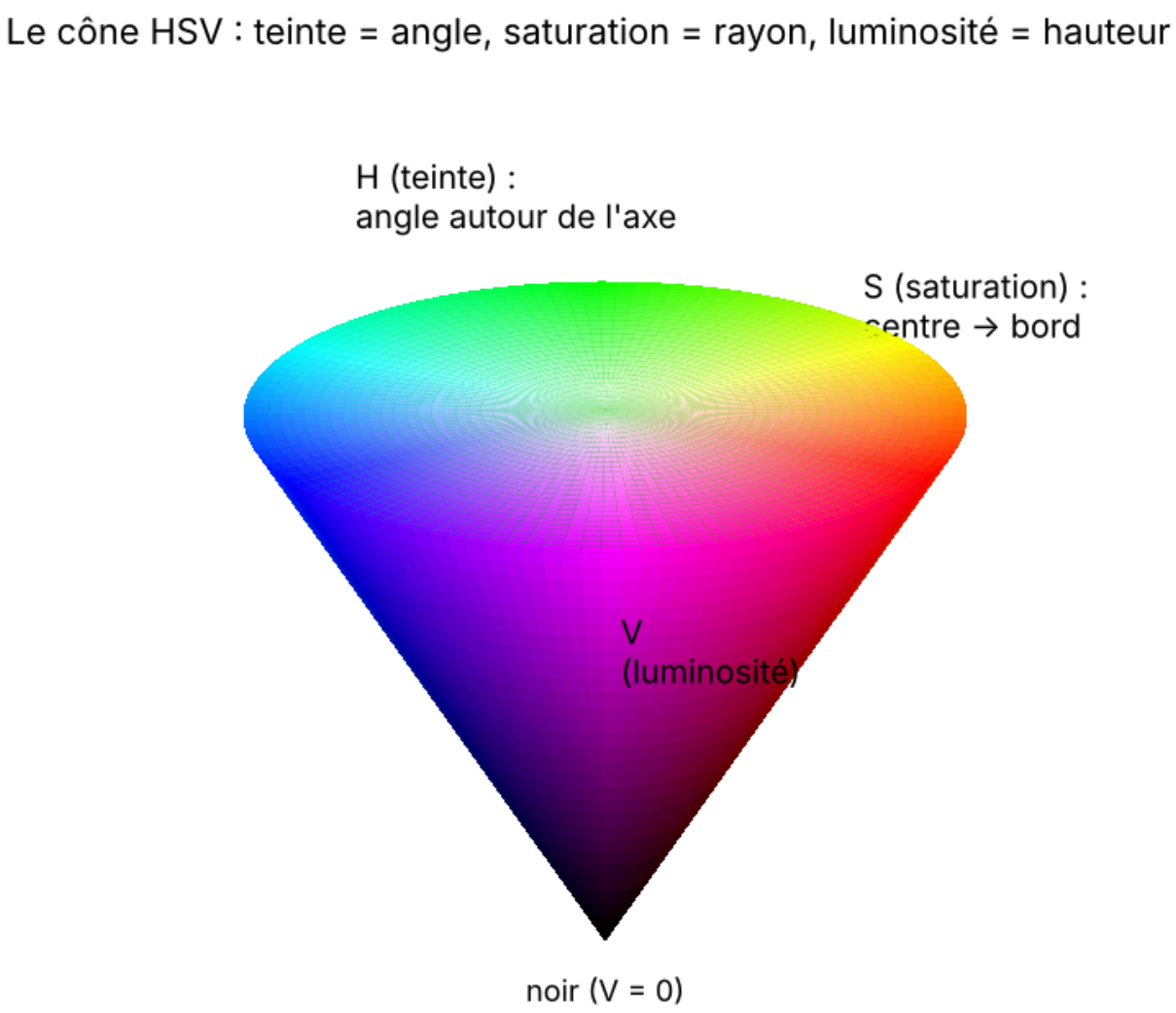
Espace	Composantes	Usage typique
BGR	Bleu, Vert, Rouge	Format natif d'OpenCV
RGB	Rouge, Vert, Bleu	Matplotlib, réseaux de neurones, Web
Niveaux de gris	Une seule intensité	Contours, seuillage, calculs plus rapides
HSV	Teinte, Saturation, Luminosité	Repérer un objet par sa couleur (chapitre 6)

Le passage au gris est une moyenne **pondérée** des trois canaux ; le vert compte le plus car l'œil humain y est le plus sensible :

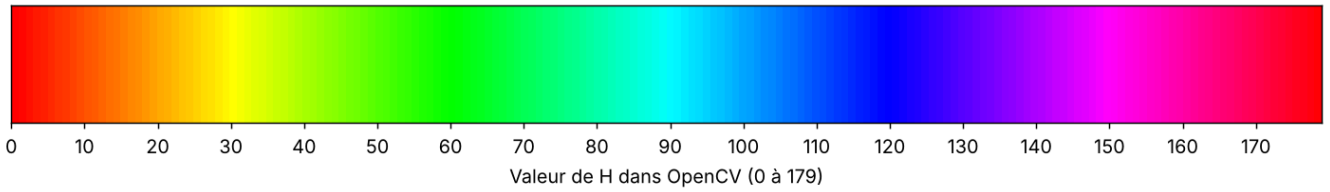
$$\text{gris} = 0{,}299\,R + 0{,}587\,G + 0{,}114\,B$$

L'espace **HSV** se représente comme un **cône** :

- **H (Hue, teinte)** : l'angle autour de l'axe, qui donne la couleur (rouge, jaune, vert, bleu…) ;
- **S (Saturation)** : la distance à l'axe, du gris au centre jusqu'à la couleur pure au bord ;
- **V (Value, luminosité)** : la hauteur, du noir à la pointe jusqu'à la pleine lumière en haut.



Dans OpenCV, H va de 0 à 179 (l'angle de 0 à 360° divisé par 2, pour tenir sur un octet), S et V vont de 0 à 255.



```
import cv2

img = cv2.imread("photo.jpg")
gris = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)

h, s, v = cv2.split(hsv)            # séparer les trois canaux
print("Teinte : de", h.min(), "à", h.max())
```

Image — Une même image en BGR, en niveaux de gris et ses trois canaux H, S, V

Interprétation : dans le canal H, la tasse et la soucoupe rouges ont une teinte homogène, alors que leurs valeurs B, G et R varient beaucoup avec les reflets. C'est pourquoi HSV facilite la détection d'un objet par sa couleur.

4.5 Le seuillage simple et adaptatif

Intuition : on transforme l'image en noir et blanc pour séparer l'objet du fond. Chaque pixel devient blanc s'il dépasse un seuil T, noir sinon.

```
g(x, y) = 255   si f(x, y) > T
g(x, y) = 0     sinon
```

Méthode	Comment T est choisi	Fonction
Seuil simple	Par le programmeur	`cv2.threshold(..., T, 255, cv2.THRESH_BINARY)`
Otsu	Automatiquement, pour toute l'image	`cv2.THRESH_BINARY + cv2.THRESH_OTSU`
Adaptatif	Un seuil différent pour chaque zone, calculé à partir des voisins	`cv2.adaptiveThreshold`

```
import cv2

page = cv2.imread("page.png", cv2.IMREAD_GRAYSCALE)

T, simple = cv2.threshold(page, 127, 255, cv2.THRESH_BINARY)
T_otsu, otsu = cv2.threshold(page, 0, 255, cv2.THRESH_BINARY +
cv2.THRESH_OTSU)
adaptatif = cv2.adaptiveThreshold(page, 255, cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
                                  cv2.THRESH_BINARY, 25, 10)
print("Seuil choisi par Otsu :", T_otsu)
```

Pour le seuil adaptatif, `25` est la taille (impaire) du voisinage utilisé pour calculer le seuil local, et `10` une constante retirée de la moyenne locale.

Image — Page de texte éclairée de façon inégale : seuil simple, Otsu et seuils adaptati…

Interprétation : avec un seul seuil pour toute la page, la partie gauche, plus sombre, devient entièrement noire. Le seuil adaptatif s'adapte à l'éclairage local et rend le texte lisible partout.

Limites : le seuillage ne regarde que l'intensité d'un pixel. Il échoue si l'objet et le fond ont des intensités proches.

4.6 La méthode d'Otsu en détail

La méthode d'Otsu (Nobuyuki Otsu, 1979) choisit **automatiquement** le meilleur seuil T. C'est l'une des méthodes les plus utilisées en traitement d'images.

Intuition : un seuil T coupe les pixels en deux classes, les sombres (⩽ T) et les clairs (> T). Un bon seuil donne deux classes **homogènes à l'intérieur** et **bien séparées entre elles**. Otsu essaie les 256 seuils possibles et garde celui qui sépare le mieux les deux classes.

Principe mathématique. Pour chaque seuil T, on calcule :

```
w0 = proportion de pixels ⩽ T          m0 = moyenne des pixels ⩽ T
w1 = proportion de pixels > T          m1 = moyenne des pixels > T

Variance inter-classes :  σ²_B(T) = w0 × w1 × (m0 − m1)²
```

Le seuil d'Otsu est le T qui rend σ²_B(T) **maximale**. La formule récompense deux choses : des moyennes très éloignées (m0 − m1 grand) et des classes de tailles comparables (w0 × w1 grand). Maximiser cette variance revient à minimiser la variance à l'intérieur de chaque classe.

Exemple à la main. Une petite image de 25 pixels a des niveaux de gris de 0 à 9, répartis ainsi :

Niveau	0	1	2	3	4	5	6	7	8	9
Nombre de pixels	1	4	5	3	1	1	2	4	3	1

T	Pixels ⩽ T	Pixels > T	m0	m1	σ²_B(T)
1	5	20	0,80	5,05	2,89
2	10	15	1,40	6,07	5,23
3	13	12	1,77	6,83	6,40
4	**14**	**11**	**1,93**	**7,09**	**6,57**
5	15	10	2,13	7,30	6,41
6	17	8	2,59	7,62	5,52
7	21	4	3,43	8,25	3,12

Détail pour T = 4 : w0 = 14 / 25 = 0,56 ; w1 = 11 / 25 = 0,44 ; m0 = (0×1 + 1×4 + 2×5 + 3×3 + 4×1) / 14 = 27 / 14 ≈ 1,93 ; m1 = (5×1 + 6×2 + 7×4 + 8×3 + 9×1) / 11 = 78 / 11 ≈ 7,09 ; σ²_B = 0,56 × 0,44 × (7,09 − 1,93)² ≈ 6,57. C'est la plus grande valeur : **le seuil d'Otsu est T = 4**.

Le même calcul en Python, comparé à OpenCV :

```
import cv2
import numpy as np

gris = cv2.imread("pieces.png", cv2.IMREAD_GRAYSCALE)
hist = np.bincount(gris.ravel(), minlength=256)      # nombre de pixels par
niveau
p = hist / hist.sum()                                  # proportions
niveaux = np.arange(256)

meilleur_T, meilleure_var = 0, 0
for T in range(256):
    w0 = p[:T + 1].sum()                               # poids de la classe 0
(<= T)
    w1 = p[T + 1:].sum()                               # poids de la classe 1
(> T)
    if w0 == 0 or w1 == 0:
        continue
    m0 = (niveaux[:T + 1] * p[:T + 1]).sum() / w0      # moyenne de la classe
0
    m1 = (niveaux[T + 1:] * p[T + 1:]).sum() / w1      # moyenne de la classe
1
```

```
    var_inter = w0 * w1 * (m0 - m1) ** 2               # variance inter-
classes
    if var_inter > meilleure_var:
        meilleur_T, meilleure_var = T, var_inter

T_opencv, _ = cv2.threshold(gris, 0, 255, cv2.THRESH_BINARY +
cv2.THRESH_OTSU)
print("Seuil calculé à la main :", meilleur_T, " seuil OpenCV :", T_opencv)
# 107 et 107
```

Image — Histogramme coupé en deux classes, courbe de la variance inter-classes avec son…

Interprétation : le seuil d'Otsu (107) tombe dans le creux entre les deux « bosses » de l'histogramme, l'une pour le fond sombre, l'autre pour les pièces claires.

Limites : Otsu suppose que l'histogramme a **deux bosses** bien distinctes. Il échoue quand l'objet est très petit par rapport au fond, ou quand l'éclairage n'est pas uniforme : sur l'image ci-dessus, le haut, plus sombre, est mal séparé. On utilise alors le seuil adaptatif (section 4.5) ou une correction de l'éclairage (chapitre 6.5).

> **À retenir** — Otsu teste tous les seuils et garde celui qui maximise $w0 \times w1 \times (m0 - m1)^2$: deux classes de tailles comparables et aux moyennes les plus éloignées possible.

4.7 Opérations bit à bit (bitwise)

Intuition : on combine deux images noir et blanc case par case, comme on combine des conditions avec ET, OU, NON.

Opération	Fonction	Le pixel est blanc si…
ET	`cv2.bitwise_and(a, b)`	il est blanc dans **les deux** images
OU	`cv2.bitwise_or(a, b)`	il est blanc dans **au moins une** image
OU exclusif	`cv2.bitwise_xor(a, b)`	il est blanc dans **une seule** image
NON	`cv2.bitwise_not(a)`	il était noir (inversion)

```
import numpy as np
import cv2

rectangle = np.zeros((200, 200), np.uint8)
cv2.rectangle(rectangle, (25, 25), (175, 175), 255, -1)
cercle = np.zeros((200, 200), np.uint8)
cv2.circle(cercle, (100, 100), 100, 255, -1)

et = cv2.bitwise_and(rectangle, cercle)
ou = cv2.bitwise_or(rectangle, cercle)
xor = cv2.bitwise_xor(rectangle, cercle)
non = cv2.bitwise_not(rectangle)
```

Image — Les quatre opérations bit à bit appliquées à un rectangle et un cercle

Usage principal : appliquer un masque. Avec un masque binaire, `bitwise_and` garde la partie de l'image située sous le blanc du masque et met le reste en noir.

```
img = cv2.imread("photo.jpg")
h, w = img.shape[:2]

masque = np.zeros((h, w), np.uint8)
cv2.circle(masque, (300, 200), 140, 255, -1)          # disque blanc

resultat = cv2.bitwise_and(img, img, mask=masque)
```

Image — Application d'un masque circulaire à une photo avec bitwise_and

> **À retenir** — Géométrique = on déplace les pixels ; photométrique = on change leurs valeurs ; seuillage = on décide pixel par pixel « objet » ou « fond » ; bitwise = on combine des masques.

Exercices

1. Faire tourner une image de 180° avec `cv2.getRotationMatrix2D`, puis comparer avec `cv2.flip(img, -1)`.
2. Un pixel vaut 100. Que vaut-il après `convertScaleAbs` avec α = 2 et β = 30 ? Et un pixel qui vaut 150 ?
3. Pourquoi H va-t-il de 0 à 179 et non de 0 à 359 dans OpenCV ?

4. Photographier une feuille de cours avec un téléphone, puis comparer seuil simple, Otsu et seuil adaptatif.
5. Mini-défi : créer un masque rectangulaire et ne garder que le centre d'une photo avec `bitwise_and`.

Corrigés

1. Les deux images sont presque identiques : une rotation de 180° équivaut à deux symétries. Il reste un léger décalage d'un demi-pixel, car le centre exact des pixels est ((w − 1) / 2, (h − 1) / 2) ; avec ce centre, les deux résultats sont rigoureusement égaux. Pour 90°, préférer `cv2.rotate(img, cv2.ROTATE_90_CLOCKWISE)`, qui échange largeur et hauteur sans couper l'image.
2. 2 × 100 + 30 = **230**. Pour 150 : 2 × 150 + 30 = 330, bloqué à **255** : le pixel devient blanc et ses détails sont perdus.
3. Une valeur `uint8` ne dépasse pas 255. OpenCV divise donc l'angle par 2 pour qu'il tienne sur un octet : 360° deviennent 180 valeurs (0 à 179).
4. Le seuil simple et Otsu donnent de grandes zones noires dans les parties mal éclairées ; le seuil adaptatif donne un texte net partout. On peut ajuster la taille du voisinage (impaire) et la constante.
5. `masque = np.zeros((h, w), np.uint8)`, puis `cv2.rectangle(masque, (w // 4, h // 4), (3 * w // 4, 3 * h // 4), 255, -1)`, puis `cv2.bitwise_and(img, img, mask=masque)`.

Compétences acquises : appliquer et paramétrer les transformations géométriques ; ajuster luminosité, contraste et gamma ; lire un histogramme ; convertir entre BGR, RGB, gris et HSV ; choisir une méthode de seuillage ; utiliser un masque.

Chapitre 5 — Filtres, convolutions et détection des contours

Objectifs : comprendre la convolution ; choisir un filtre de lissage ; calculer le gradient d'une image avec Sobel ; détecter des contours avec le Laplacien et Canny ; retrouver un motif dans une image.

Prérequis : chapitres 3 et 4 ; produit et somme de nombres ; notion intuitive de dérivée (pente d'une courbe).

5.1 La convolution : un noyau qui glisse sur l'image

Intuition : on pose une petite grille de coefficients, le **noyau** (en anglais *kernel*), sur un pixel et ses voisins. On multiplie chaque pixel par le coefficient qui le recouvre, on additionne tout, et le résultat devient la nouvelle valeur du pixel central. Puis on déplace le noyau d'un pixel et on recommence, partout dans l'image.

Exemple détaillé sur un pixel. L'image ci-dessous a une partie sombre (10) à gauche et une partie claire (80) à droite. Le noyau est celui de Sobel, qui mesure les variations horizontales.

Image — Convolution pas à pas : fenêtre 3 × 3, noyau, produits case par case, somme

```
Étape 1 — fenêtre 3 x 3 autour du pixel (2, 2)     Étape 2 — noyau
   10  10  80                                          -1   0   1
   10  10  80                                          -2   0   2
   10  10  80                                          -1   0   1

Étape 3 — produits case par case
   10 x (-1) = -10     10 x 0 = 0     80 x 1 =  80
   10 x (-2) = -20     10 x 0 = 0     80 x 2 = 160
   10 x (-1) = -10     10 x 0 = 0     80 x 1 =  80

Étape 4 — somme : -10 - 20 - 10 + 80 + 160 + 80 = 280
```

Le résultat est grand (280) car on est sur un bord : à droite, c'est beaucoup plus clair qu'à gauche. Dans une zone uniforme (que des 10), la somme vaudrait 0.

Formule générale pour un noyau K de taille 3 × 3 :

Image — Formule de la convolution avec un noyau 3 × 3

Le calcul à la main en Python, puis avec OpenCV :

```
import numpy as np
import cv2

image = np.array([[10, 10, 10, 80, 80],
                  [10, 10, 10, 80, 80],
                  [10, 10, 10, 80, 80],
                  [10, 10, 10, 80, 80],
                  [10, 10, 10, 80, 80]], dtype=np.float32)

noyau = np.array([[-1, 0, 1],
                  [-2, 0, 2],
                  [-1, 0, 1]], dtype=np.float32)

h, w = image.shape
sortie = np.zeros((h - 2, w - 2), dtype=np.float32)
for y in range(1, h - 1):
```

```
    for x in range(1, w - 1):
        fenetre = image[y - 1:y + 2, x - 1:x + 2]      # voisinage 3 x 3
        sortie[y - 1, x - 1] = np.sum(fenetre * noyau)  # multiplier puis
additionner
print(sortie)                    # trois colonnes : 0, 280, 280

avec_opencv = cv2.filter2D(image, -1, noyau)   # même calcul, optimisé
print(avec_opencv[1:-1, 1:-1])                 # mêmes valeurs
```

> **À retenir** — Un noyau est une « recette » : ses coefficients décident de ce que le filtre fait. Des coefficients tous positifs et égaux font une **moyenne** (flou). Des coefficients positifs d'un côté et négatifs de l'autre mesurent une **différence** (contour).

Calculer la nouvelle valeur d'un pixel : et les pixels du bord ? Pour calculer la nouvelle valeur d'un pixel, le filtre a besoin de tous ses voisins. Au bord de l'image, une partie de ces voisins n'existe pas : le pixel en haut à gauche n'a ni voisin au-dessus ni voisin à gauche. Il faut donc **inventer une bordure** autour de l'image avant d'appliquer le filtre. Trois solutions sont courantes :

Solution	Principe	Constante OpenCV	Effet
Ajouter des zéros	Les pixels manquants valent 0 (ou une valeur choisie)	`cv2.BORDER_CONSTANT`	Simple, mais assombrit les bords après un flou
Répliquer les bords	On recopie le dernier pixel : `aaa|abcd|ddd`	`cv2.BORDER_REPLICATE`	Bords naturels, légèrement étirés
Réfléchir l'image	On prolonge l'image comme dans un miroir : `cba|abcd|dcb`	`cv2.BORDER_REFLECT` ; variante `cv2.BORDER_REFLECT_101` (défaut d'OpenCV) qui ne répète pas le bord : `dcb|abcd|cba`	Transitions les plus douces

Image — Une image 3 × 3 prolongée de deux pixels de chaque côté : zéros, réplication, r…

```
import cv2
import numpy as np

m = np.array([[1, 2, 3],
              [4, 5, 6],
              [7, 8, 9]], dtype=np.uint8)

# Ajouter 2 pixels de bordure de chaque côté pour voir les trois solutions
print(cv2.copyMakeBorder(m, 2, 2, 2, 2, cv2.BORDER_CONSTANT, value=0))
print(cv2.copyMakeBorder(m, 2, 2, 2, 2, cv2.BORDER_REPLICATE))
print(cv2.copyMakeBorder(m, 2, 2, 2, 2, cv2.BORDER_REFLECT))

# Les filtres d'OpenCV acceptent le paramètre borderType
img = cv2.imread("photo.jpg")
flou_zeros = cv2.blur(img, (41, 41), borderType=cv2.BORDER_CONSTANT)
flou_replique = cv2.blur(img, (41, 41), borderType=cv2.BORDER_REPLICATE)
flou_defaut = cv2.blur(img, (41, 41))          # BORDER_REFLECT_101 par
défaut
```

Image — Un flou très fort avec les trois gestions de bord : les zéros créent un cadre s…

> **À retenir** — La gestion des bords n'a d'effet que sur une bande de la largeur du demi-noyau (2 pixels pour un noyau 5 × 5). Elle devient visible avec de grands noyaux. Le choix par défaut d'OpenCV, la réflexion, convient dans la plupart des cas.

5.2 Le flou et la réduction du bruit

Intuition : un capteur d'appareil photo produit du **bruit**, de petites variations aléatoires. Remplacer chaque pixel par une moyenne de ses voisins atténue ces variations.

```
Moyenneur 3 x 3 (÷ 9)      Gaussien 3 x 3 (÷ 16)
  1  1  1                     1  2  1
  1  1  1                     2  4  2
  1  1  1                     1  2  1
```

Filtre	Fonction	Principe	Utile pour	Limite
Moyenneur	`cv2.blur`	Moyenne simple des voisins	Lissage rapide	Floute les bords

Filtre	Fonction	Principe	Utile pour	Limite
Gaussien	`cv2.GaussianBlur`	Moyenne pondérée, le centre compte plus	Bruit léger, avant Canny	Floute aussi les bords
Médian	`cv2.medianBlur`	Valeur médiane des voisins (pas une convolution)	Bruit « poivre et sel »	Plus lent sur de grands noyaux
Bilatéral	`cv2.bilateralFilter`	Moyenne des voisins de couleur proche	Lisser en gardant les bords	Beaucoup plus lent

```
import cv2
import numpy as np

img = cv2.imread("photo.jpg")

moyenne = cv2.blur(img, (5, 5))                # noyau 5 x 5
gauss = cv2.GaussianBlur(img, (5, 5), 0)       # 0 : écart-type calculé
automatiquement
mediane = cv2.medianBlur(img, 5)               # taille impaire obligatoire
bilateral = cv2.bilateralFilter(img, 9, 75, 75)

noyau = np.ones((5, 5), np.float32) / 25       # le moyenneur écrit « à la
main »
moyenne2 = cv2.filter2D(img, -1, noyau)        # donne le même résultat que
cv2.blur
```

Image — Plus le noyau est grand, plus le flou est fort ; le filtre bilatéral garde les …

Image — Sur un bruit « poivre et sel », le filtre médian est de loin le plus efficace

Interprétation : le moyenneur et le gaussien étalent les points blancs et noirs au lieu de les supprimer. Le médian les élimine, car une valeur extrême n'est jamais la médiane de son voisinage.

5.3 Le gradient d'une image et les filtres de Sobel

Intuition : un **contour** est un endroit où l'intensité change brusquement. Le **gradient** mesure ce changement, comme la pente d'une route mesure à quelle vitesse on monte.

Image — Intensité le long d'une ligne de l'image et sa dérivée : les pics correspondent…

Les **filtres de Sobel** estiment cette dérivée dans deux directions :

```
Sobel X (variations de gauche à droite)     Sobel Y (variations de haut en
bas)
  -1   0   1                                  -1  -2  -1
  -2   0   2                                   0   0   0
  -1   0   1                                   1   2   1
```

Sobel X réagit aux **bords verticaux**, Sobel Y aux **bords horizontaux**. On combine les deux résultats Gx et Gy pour obtenir la force et la direction du bord :

Image — Norme et direction du gradient à partir de Gx et Gy

5.4 Le Laplacien

Le **Laplacien** est une dérivée seconde : il mesure comment la pente elle-même change. Il réagit aux bords dans toutes les directions en une seule opération, mais il est très sensible au bruit : on floute toujours l'image avant.

```
Noyau du Laplacien
   0   1   0
   1  -4   1
   0   1   0
```

5.5 Le détecteur de Canny

Canny (1986) enchaîne quatre étapes pour obtenir des contours fins et continus :

1. **lisser** l'image avec un filtre gaussien ;
2. **calculer le gradient** avec Sobel (force et direction) ;
3. **amincir** les bords : on ne garde que les pixels dont le gradient est maximal dans la direction du gradient ;
4. **seuiller par hystérésis** avec deux seuils : au-dessus du seuil haut, le pixel est un bord ; entre les deux seuils, il n'est gardé que s'il touche un bord déjà retenu.

```
import cv2

gris = cv2.imread("photo.jpg", cv2.IMREAD_GRAYSCALE)
gris = cv2.GaussianBlur(gris, (3, 3), 0)

gx = cv2.Sobel(gris, cv2.CV_64F, 1, 0, ksize=3)    # dérivée selon x
gy = cv2.Sobel(gris, cv2.CV_64F, 0, 1, ksize=3)    # dérivée selon y
norme = cv2.magnitude(gx, gy)                      # racine(gx² + gy²)

laplacien = cv2.Laplacian(gris, cv2.CV_64F)
canny = cv2.Canny(gris, 50, 150)                   # seuil bas, seuil haut

affichage = cv2.convertScaleAbs(norme)             # retour à des valeurs 0-
255 pour afficher
```

Image — Sobel X, Sobel Y, norme du gradient, Laplacien et Canny sur la même photo

Interprétation : Sobel X fait ressortir le trépied et le dos du photographe (bords verticaux) ; Sobel Y, l'horizon et les épaules (bords horizontaux). La norme réunit les deux. Canny donne des traits d'un pixel d'épaisseur, plus faciles à exploiter, et ignore une grande partie de la texture de l'herbe.

Image — Effet des seuils de Canny : trop bas, adaptés, trop hauts

> **Erreur fréquente** — Calculer Sobel avec une sortie `uint8` (`cv2.Sobel(gris, -1, 1, 0)`) supprime toutes les valeurs négatives, donc la moitié des bords (passage du clair au sombre). Il faut demander `cv2.CV_64F`, puis revenir à 0-255 avec `cv2.convertScaleAbs` seulement pour l'affichage.

> **Question de réflexion** — Le mot « gradient » reviendra au chapitre 9 pour entraîner des réseaux de neurones. Ici, il mesure comment l'**intensité** varie quand on se déplace dans l'image. Que pourrait-il mesurer quand on modifie les **paramètres** d'un modèle ?

5.6 Améliorer une image floue : contraste, netteté, réduction du flou

Une photo floue (mise au point ratée, mouvement) a perdu ses variations rapides d'intensité : les détails fins. Trois approches existent, qui n'ont pas du tout la même ambition.

Approche	Technique	Ce qu'elle fait	Ce qu'elle ne fait pas
1. Améliorer le contraste	Égalisation d'histogramme, `convertScaleAbs` (section 4.3)	Rend plus visibles les différences entre zones sombres et claires	Ne récupère pas les détails perdus par le flou
2. Renforcer la netteté	Filtre passe-haut, masque flou (*unsharp masking*)	Accentue les contours pour donner une **impression** de netteté	Ne reconstitue pas nécessairement les vrais détails ; amplifie le bruit
3. Réduire le flou	Déconvolution, méthodes de restauration, modèles d'IA spécialisés	Cherche à **récupérer** les détails atténués	Résultat limité par le type de flou et l'information encore présente

Renforcer la netteté avec le masque flou. Malgré son nom, cette technique rend l'image plus nette. Une version floue de l'image contient les grandes formes ; la différence **image − version floue** contient donc les détails. On ajoute ces détails, amplifiés, à l'image :

```
nette = image + k × (image − image_floue)   =   (1 + k) × image − k × image_floue
```

Avec k = 2, cela donne `cv2.addWeighted(image, 3, image_floue, -2, 0)`.

Filtre passe-haut. Un noyau dont la somme vaut 1, avec un centre fort et des voisins négatifs, garde l'image et ajoute ses contours :

```
 -1  -1  -1
 -1   9  -1        somme = 9 − 8 = 1 : les zones uniformes ne changent pas
 -1  -1  -1
```

Réduire le flou par déconvolution. Un flou est une convolution de l'image nette par un noyau, appelé **réponse impulsionnelle** (en anglais *PSF*, *point spread function*). Si l'on connaît ou estime ce noyau, on peut chercher l'image nette qui, floutée, redonnerait l'image observée. L'algorithme de **Richardson-Lucy** fait cette recherche par itérations successives.

```
import cv2
import numpy as np
from skimage import restoration          # pip install scikit-image

gris = cv2.imread("camera.png", cv2.IMREAD_GRAYSCALE)
```

```
floue = cv2.GaussianBlur(gris, (9, 9), 2)                  # on simule une
photo floue

# 1. Améliorer le contraste
egalisee = cv2.equalizeHist(floue)

# 2a. Renforcer la netteté : masque flou
flou_fort = cv2.GaussianBlur(floue, (0, 0), 3)
nette = cv2.addWeighted(floue, 3.0, flou_fort, -2.0, 0)

# 2b. Renforcer la netteté : filtre passe-haut
noyau = np.array([[-1, -1, -1],
                  [-1,  9, -1],
                  [-1, -1, -1]], dtype=np.float32)
passe_haut = cv2.filter2D(floue, -1, noyau)

# 3. Réduire le flou : déconvolution de Richardson-Lucy
psf = cv2.getGaussianKernel(9, 2) @ cv2.getGaussianKernel(9, 2).T   # le flou
supposé
restauree = restoration.richardson_lucy(floue / 255.0, psf, num_iter=30)
restauree = (np.clip(restauree, 0, 1) * 255).astype(np.uint8)
```

Image — Image floue, contraste amélioré, masque flou, filtre passe-haut et déconvolution

Interprétation : l'égalisation rend l'image plus contrastée mais toujours aussi floue. Le masque flou et le filtre passe-haut accentuent les contours (visage, caméra). La déconvolution récupère davantage de détails, car elle « défait » le flou au lieu de seulement souligner les bords. Elle fonctionne bien ici parce que le flou a été simulé et que l'on connaît exactement son noyau. Sur une vraie photo, il faut l'estimer, et le bruit limite fortement le résultat.

> **Erreur fréquente** — Renforcer trop la netteté (k ou noyau trop forts). On obtient des halos clairs et sombres autour des contours et un bruit très visible.

5.7 Rechercher un motif : le template matching

Le **template matching** cherche où se trouve un petit motif (le **modèle**) dans une grande image. Le principe est proche de la convolution : on fait glisser le modèle sur toute l'image et on calcule à chaque position un **score de ressemblance**. La meilleure position est celle du score maximal.

```
import cv2

image = cv2.imread("photo.jpg")
modele = image[150:220, 280:380].copy()          # un morceau découpé de
l'image
h, w = modele.shape[:2]

gris_image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
gris_modele = cv2.cvtColor(modele, cv2.COLOR_BGR2GRAY)

carte = cv2.matchTemplate(gris_image, gris_modele, cv2.TM_CCOEFF_NORMED)
_, score_max, _, position = cv2.minMaxLoc(carte)
print("Score :", round(score_max, 2), " position :", position)   # score 1 =
identique

x, y = position
cv2.rectangle(image, (x, y), (x + w, y + h), (0, 255, 0), 3)
```

Image — Le modèle, la carte de ressemblance (zone claire = ressemblance forte) et la po…

Limites : le template matching ne fonctionne que si l'objet a la même taille, la même orientation et le même éclairage que le modèle. Un objet tourné ou plus petit n'est pas retrouvé : c'est l'une des motivations des réseaux de neurones.

Exercices

1. Appliquer à la main le noyau Sobel X au pixel central de la fenêtre ci-dessous. Y a-t-il un bord vertical ?

```
50  50  50
50  50  50
50  50  50
```

2. Calculer à la main le filtre moyenneur et le filtre médian 3 × 3 au centre de cette matrice. Lequel élimine la valeur 255 ?

```
12  15  14
13 255  16
11  14  13
```

3. Que détecte le noyau ci-dessous ? Le vérifier avec `cv2.filter2D`.

```
-1  -1  -1
 0   0   0
 1   1   1
```

4. Mini-défi : découper un morceau d'une photo, l'enregistrer, puis le retrouver par template matching. Le retrouve-t-on encore si l'on réduit la photo de moitié ?

Corrigés

1. Tous les produits s'annulent : (−50 − 100 − 50) + (50 + 100 + 50) = **0**. Zone uniforme, aucun bord.
2. Moyenne : 363 / 9 ≈ **40**. Médiane : on trie 11, 12, 13, 13, 14, 14, 15, 16, 255 ; la valeur du milieu est **14**. Le médian élimine la valeur aberrante, le moyenneur la répartit sur le voisinage.
3. C'est un détecteur de **bords horizontaux** (variation de haut en bas), proche de Sobel Y mais sans le poids 2 au centre (on l'appelle filtre de Prewitt).
4. Sur l'image d'origine, le score vaut 1 à la bonne position. Après réduction de moitié, le motif n'a plus la même taille : le score maximal chute et la position trouvée est souvent fausse. Il faudrait tester plusieurs échelles du modèle.

Compétences acquises : calculer une convolution à la main et avec OpenCV ; choisir un filtre selon le type de bruit ; interpréter Sobel X et Y, le Laplacien et Canny ; utiliser le template matching et connaître ses limites.

Chapitre 6 — Segmentation et opérations morphologiques

Objectifs : isoler un objet par sa couleur ; nettoyer un masque avec la morphologie ; extraire et mesurer des contours ; enchaîner plusieurs traitements pour compter des objets.

Prérequis : chapitre 4 (HSV, seuillage, bitwise) et chapitre 5 (flou, contours).

Segmenter une image, c'est la découper en régions : d'un côté les pixels de l'objet, de l'autre ceux du fond. Le résultat est un **masque** binaire (blanc = objet, noir = fond).

6.1 Segmenter par la couleur avec HSV

Avec `cv2.inRange`, on garde les pixels dont H, S et V sont tous compris entre une valeur minimale et une valeur maximale.

```
import cv2

img = cv2.imread("pomme.jpg")
hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)

#                      H   S    V            H    S    V
masque = cv2.inRange(hsv, (0, 120, 70), (10, 255, 255))

rouge = cv2.bitwise_and(img, img, mask=masque)   # garder seulement les
pixels rouges
```

Le seuil S ⩾ 120 écarte les couleurs ternes ou grises ; V ⩾ 70 écarte les pixels trop sombres, dont la teinte n'est pas fiable.

6.2 Érosion et dilatation

Intuition : un masque brut contient souvent des points isolés et de petits trous. La morphologie mathématique les corrige en faisant glisser un petit carré, l'**élément structurant**, sur le masque.

- **Érosion** : un pixel reste blanc seulement si **tous** ses voisins sont blancs. Les zones blanches rétrécissent et les petits points disparaissent.
- **Dilatation** : un pixel devient blanc si **au moins un** voisin est blanc. Les zones blanches grossissent et les petits trous se bouchent.

Exemple sur une petite matrice (élément structurant carré 3 × 3) :

```
Masque            Après érosion      Après dilatation
0 0 0 0 0         0 0 0 0 0          1 1 1 1 1
0 1 1 1 0         0 0 0 0 0          1 1 1 1 1
0 1 1 1 0    →    0 0 1 0 0          1 1 1 1 1
0 1 1 1 0         0 0 0 0 0          1 1 1 1 1
0 0 0 0 0         0 0 0 0 0          1 1 1 1 1
```

Seul le pixel central a ses 8 voisins blancs : c'est le seul qui survit à l'érosion. À l'inverse, tous les pixels touchent au moins un pixel blanc : la dilatation remplit toute la grille.

```
import cv2
import numpy as np

noyau = np.ones((3, 3), np.uint8)                  # élément structurant 3 x
3

erode = cv2.erode(masque, noyau, iterations=2)     # plus d'itérations =
effet plus fort
dilate = cv2.dilate(masque, noyau, iterations=2)

ouverture = cv2.morphologyEx(masque, cv2.MORPH_OPEN, noyau)    # érosion puis
dilatation
fermeture = cv2.morphologyEx(masque, cv2.MORPH_CLOSE, noyau)   # dilatation
puis érosion
```

Image — Érosion et dilatation avec 1 et 3 itérations sur un texte bruité

Opération	Enchaînement	Effet	Usage
Érosion	—	Rétrécit les zones blanches	Supprimer les petits points, séparer des objets collés
Dilatation	—	Agrandit les zones blanches	Boucher les trous, relier des morceaux
Ouverture	érosion puis dilatation	Enlève les petits points blancs, garde la taille	Nettoyer le bruit
Fermeture	dilatation puis érosion	Bouche les petits trous noirs, garde la taille	Remplir l'objet

Image — Effet des quatre opérations morphologiques sur un masque bruité

6.3 Contours et mesures

Une fois le masque propre, `cv2.findContours` trouve le bord de chaque région blanche. On peut ensuite mesurer chaque contour.

Fonction	Résultat
`cv2.findContours(masque, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)`	Liste des contours extérieurs
`cv2.contourArea(c)`	Aire du contour, en pixels
`cv2.boundingRect(c)`	Rectangle englobant (x, y, largeur, hauteur)
`cv2.minEnclosingCircle(c)`	Plus petit cercle englobant (centre, rayon)
`cv2.drawContours(img, contours, -1, couleur, épaisseur)`	Dessin des contours

6.4 Exemple : détecter une pomme rouge

```
import cv2
import numpy as np

img = cv2.imread("pomme.jpg")
hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)

# 1. Masque du rouge
masque = cv2.inRange(hsv, (0, 120, 70), (10, 255, 255))

# 2. Nettoyage
noyau = np.ones((5, 5), np.uint8)
masque = cv2.morphologyEx(masque, cv2.MORPH_OPEN, noyau)
masque = cv2.morphologyEx(masque, cv2.MORPH_CLOSE, noyau)

# 3. Contours
contours, _ = cv2.findContours(masque, cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_SIMPLE)

# 4. Une boîte autour de chaque contour assez grand
for c in contours:
    if cv2.contourArea(c) > 1000:
        x, y, w, h = cv2.boundingRect(c)
        cv2.rectangle(img, (x, y), (x + w, y + h), (0, 255, 0), 2)

cv2.imwrite("pomme_detectee.jpg", img)
```

Image — Résultat avec un seul masque : seule une partie de la pomme est détectée

Le résultat est **incomplet** : une partie de la pomme n'entre pas dans le masque. L'exercice guidé 6.6 permet de trouver pourquoi.

6.5 Enchaîner les traitements : compter des pièces de monnaie

Une application réelle combine presque toujours plusieurs étapes. Sur la photo ci-dessous, le haut est plus sombre que le bas : un seul seuil global ne sépare pas correctement les pièces du fond.

Image — Seuil global d'Otsu et seuil adaptatif sur une image mal éclairée

Une solution simple consiste à **estimer l'éclairage du fond** avec un flou très fort, puis à le **retirer** de l'image avant de seuiller.

```
import cv2
import numpy as np

gris = cv2.imread("pieces.png", cv2.IMREAD_GRAYSCALE)

fond = cv2.GaussianBlur(gris, (0, 0), 40)       # 1. estimer l'éclairage du
fond
corrigee = cv2.subtract(gris, fond)              # 2. retirer le fond

_, masque = cv2.threshold(corrigee, 0, 255, cv2.THRESH_BINARY +
cv2.THRESH_OTSU)  # 3. seuil

masque = cv2.morphologyEx(masque, cv2.MORPH_OPEN, np.ones((3, 3), np.uint8))
# 4. nettoyage
masque = cv2.morphologyEx(masque, cv2.MORPH_CLOSE, np.ones((7, 7), np.uint8))

contours, _ = cv2.findContours(masque, cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_SIMPLE)  # 5. contours

resultat = cv2.cvtColor(gris, cv2.COLOR_GRAY2BGR)
nombre = 0
for c in contours:
    if cv2.contourArea(c) > 300:                 # ignorer les petites
taches
        (x, y), rayon = cv2.minEnclosingCircle(c)
```

```
        cv2.circle(resultat, (int(x), int(y)), int(rayon), (0, 255, 0), 2)
        nombre += 1

print("Nombre de pièces :", nombre)              # 24
```

Image — Les six étapes du comptage de pièces : original, fond estimé, correction, seuil…

Interprétation : chaque étape règle un problème précis. La correction du fond rend le seuil d'Otsu efficace partout ; la morphologie supprime les taches et bouche les trous dus aux reliefs des pièces ; le filtre sur l'aire élimine les derniers fragments. Les 24 pièces sont trouvées.

> **À retenir** — Une chaîne de traitement classique : **prétraiter** (gris, flou, correction d'éclairage) → **segmenter** (seuil ou couleur) → **nettoyer** (morphologie) → **analyser** (contours, aires) → **afficher**.

> **Question de réflexion** — Tous les réglages de ce chapitre (plages HSV, taille des noyaux, aire minimale) ont été choisis à la main pour **ces** images. Que se passe-t-il avec une autre photo, un autre éclairage, des pièces qui se touchent ? Les chapitres 7 et 8 proposent d'autres approches.

6.6 Exercice guidé : obtenir la pomme complète

1. Afficher la valeur HSV d'un pixel de la partie gauche de la pomme, puis d'un pixel de la partie droite :

```
print(hsv[150, 110])    # partie gauche, détectée
print(hsv[150, 180])    # partie droite, non détectée
```

2. Comparer les deux valeurs de H. Sur la barre des teintes (chapitre 4.4), à quelle couleur correspond la valeur trouvée à droite ?
3. En déduire pourquoi un rouge peut avoir une teinte très petite **ou** très grande. Penser à la forme du cône : la teinte est un angle.
4. Construire un second masque avec `cv2.inRange` pour la partie manquante.
5. Combiner les deux masques avec une opération bit à bit du chapitre 4.7. Laquelle faut-il choisir : AND, OR ou XOR ?
6. Relancer la détection et vérifier que la boîte entoure toute la pomme.

Exercices

1. Une érosion avec un élément structurant 3 × 3 est appliquée à un carré blanc de 5 × 5 pixels. Quelle est la taille du carré obtenu ?
2. On applique une ouverture à un masque qui contient un objet et des points blancs isolés d'un pixel. Que deviennent les points ? Et l'objet ?
3. Adapter le code de la pomme pour détecter une balle jaune (indice : `np.uint8([[[0, 255, 255]]])` est du jaune pur en BGR).
4. Mini-défi : dans le comptage de pièces, afficher l'aire de chaque pièce à côté de son cercle, puis compter séparément les grandes et les petites pièces.

Corrigés

Exercice guidé 6.6. À gauche, H vaut environ 3 ; à droite, environ 176. Les deux sont rouges : le rouge se trouve à la fois au début (0°) et à la fin (360°) du cercle des teintes, donc près de 0 **et** près de 179 dans OpenCV. Il faut deux masques, combinés par **OR** (un pixel est rouge s'il est dans l'une **ou** l'autre plage) :

```
masque1 = cv2.inRange(hsv, (0, 120, 70), (10, 255, 255))
masque2 = cv2.inRange(hsv, (170, 120, 70), (179, 255, 255))
masque = cv2.bitwise_or(masque1, masque2)
```

La boîte entoure alors toute la pomme (159 × 159 pixels au lieu de 81 × 159). AND ne garderait aucun pixel, car une teinte ne peut pas être à la fois inférieure à 10 et supérieure à 170.

1. Un carré de **3 × 3** : l'érosion retire une couche d'un pixel tout autour.
2. Les points isolés **disparaissent** (l'érosion les efface et la dilatation ne peut pas les recréer) ; l'objet **garde à peu près sa taille** (l'érosion le rétrécit, la dilatation le rétablit).
3. Le jaune pur a une teinte H = 30. On peut prendre `cv2.inRange(hsv, (20, 100, 100), (35, 255, 255))`, puis le même nettoyage et la même recherche de contours.
4. Dans la boucle, ajouter `cv2.putText(resultat, str(int(cv2.contourArea(c))), (int(x), int(y)), cv2.FONT_HERSHEY_SIMPLEX, 0.4, (0, 0, 255), 1)` ; puis comparer chaque aire à un seuil choisi en regardant les valeurs affichées.

Compétences acquises : construire un masque de couleur ; appliquer érosion, dilatation, ouverture et fermeture ; extraire, mesurer et dessiner des contours ; construire une chaîne de traitement complète.

Chapitre 7 — Identifier un objet dans une image

Objectifs : connaître les principales méthodes pour retrouver un objet dans une image ; reconnaître une forme géométrique à partir de son contour ; retrouver un objet par ses points caractéristiques avec ORB ; savoir quand passer aux réseaux de neurones.

Prérequis : chapitres 4 à 6 (seuillage, contours, segmentation, morphologie).

7.1 Les principales méthodes

Méthode	Principe	Points forts	Limites	Où dans le cours
Couleur (segmentation HSV)	Garder les pixels d'une plage de couleur	Simple, très rapide	Tout objet de même couleur est détecté ; sensible à l'éclairage	Chapitre 6
Forme et caractéristiques géométriques	Trouver les contours, puis reconnaître cercle, rectangle, triangle…	Simple, interprétable	Échoue quand les objets se chevauchent ou se touchent	Section 7.2
Comparaison de points caractéristiques	Retrouver dans l'image des points qui ressemblent à ceux d'une image de référence	Résiste à la rotation, au changement d'échelle, à l'occultation partielle	Il faut une image de référence ; objets peu texturés difficiles	Section 7.3
Réseaux de neurones	Apprendre l'apparence d'une catégorie à partir d'exemples	Reconnaît une **catégorie** (toutes les voitures), très robuste	Beaucoup de données, calcul lourd	Chapitres 10 et 11 (YOLO)

7.2 Reconnaître un objet par sa forme

Intuition : on isole chaque objet (seuillage), on trouve son contour, puis on utilise la **géométrie** du contour pour décider de quelle forme il s'agit.

Caractéristiques géométriques utiles :

Caractéristique	Calcul OpenCV	Ce qu'elle indique
Nombre de sommets	`cv2.approxPolyDP` simplifie le contour en polygone	3 = triangle, 4 = quadrilatère, beaucoup = forme arrondie

Caractéristique	Calcul OpenCV	Ce qu'elle indique
Rapport largeur / hauteur	`cv2.boundingRect`	Proche de 1 : carré ; sinon rectangle
Circularité	$4\pi \times$ aire / périmètre²	Vaut 1 pour un cercle parfait, moins pour les autres formes
Aire	`cv2.contourArea`	Éliminer les petites taches

```
import cv2

img = cv2.imread("formes.png")
gris = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
_, masque = cv2.threshold(gris, 240, 255, cv2.THRESH_BINARY_INV)   # objets
en blanc
contours, _ = cv2.findContours(masque, cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_SIMPLE)

for c in contours:
    perimetre = cv2.arcLength(c, True)
    sommets = cv2.approxPolyDP(c, 0.03 * perimetre, True)    # polygone
simplifié
    n = len(sommets)
    if n == 3:
        nom = "triangle"
    elif n == 4:
        x, y, w, h = cv2.boundingRect(sommets)
        nom = "carré" if 0.9 < w / h < 1.1 else "rectangle"
    else:
        nom = "cercle"
    x, y, w, h = cv2.boundingRect(c)
    cv2.putText(img, nom, (x, y + h + 20), cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0,
0, 0), 2)
    print(nom, n)
```

Le paramètre `0.03 * perimetre` fixe la tolérance de simplification : plus il est grand, plus le polygone obtenu a peu de sommets.

Image — À gauche, chaque forme séparée est reconnue ; à droite, trois formes qui se tou…

Limite : quand les objets se **chevauchent** ou se **touchent**, le seuillage les fusionne en une seule région et le contour ne correspond plus à aucune forme. La méthode est fiable pour des objets bien séparés sur un fond uniforme : pièces sur un tapis roulant, panneaux,

formes dessinées.

7.3 Comparer des points caractéristiques

Intuition : pour retrouver un objet précis (une couverture de livre, un logo, un badge), on dispose d'une **image de référence** de cet objet. On repère dans la référence des **points caractéristiques** : des endroits très reconnaissables, comme les coins et les zones texturées. On les cherche ensuite dans l'**image à analyser**. Si beaucoup de points se retrouvent avec une disposition cohérente, l'objet est présent : ce sont les **correspondances**.

```
Image de référence  →  points + descripteurs  ┐
                                               ├→  correspondances  →
vérification géométrique  →  objet localisé
Image à analyser    →  points + descripteurs  ┘
```

Chaque point est décrit par un **descripteur** : une signature numérique de son voisinage. Deux points se correspondent si leurs descripteurs se ressemblent.

ORB (*Oriented FAST and Rotated BRIEF*) est une méthode libre, rapide et incluse dans OpenCV. Elle fonctionne en trois étapes :

1. **FAST : détecter les points clés.** Pour chaque pixel candidat, on regarde les 16 pixels d'un petit cercle autour de lui. Si au moins 9 pixels consécutifs du cercle sont tous nettement plus clairs, ou tous nettement plus sombres, que le centre, le pixel est un coin : un point clé. Le test est fait sur plusieurs échelles de l'image, pour trouver les points même si l'objet change de taille.
2. **Orientation : déterminer la direction.** On calcule le « centre de gravité » des intensités autour du point. La direction du point vers ce centre donne l'orientation du point. On pourra ainsi comparer des points même si l'objet est tourné.
3. **BRIEF orienté : décrire chaque point.** Dans un carré de 31 × 31 pixels autour du point, tourné selon son orientation, on compare 256 paires de pixels choisies à l'avance : « le pixel A est-il plus clair que le pixel B ? ». Chaque réponse donne un bit (1 ou 0). ORB construit ainsi une **signature binaire** de 256 bits (32 octets) à partir de comparaisons d'intensité entre des paires de pixels autour du point.

Pour comparer deux signatures binaires, on compte les bits différents : c'est la **distance de Hamming**. Plus elle est petite, plus les points se ressemblent.

```
import cv2
import numpy as np

reference = cv2.imread("reference.jpg", cv2.IMREAD_GRAYSCALE)   # l'objet
seul
scene = cv2.imread("scene.jpg", cv2.IMREAD_GRAYSCALE)           # l'image à
```

```
analyser

# 1. Détecter et décrire les points caractéristiques
orb = cv2.ORB_create(nfeatures=1000)
kp1, desc1 = orb.detectAndCompute(reference, None)
kp2, desc2 = orb.detectAndCompute(scene, None)

# 2. Chercher, pour chaque point, ses deux meilleurs candidats
matcher = cv2.BFMatcher(cv2.NORM_HAMMING)
paires = matcher.knnMatch(desc1, desc2, k=2)

# 3. Test du ratio : garder un point seulement si le meilleur candidat est
nettement meilleur que le second
bonnes = [m for m, n in paires if m.distance < 0.75 * n.distance]
print("Bonnes correspondances :", len(bonnes))

# 4. Vérification géométrique : les points doivent suivre une même
déformation
if len(bonnes) >= 10:
    src = np.float32([kp1[m.queryIdx].pt for m in bonnes]).reshape(-1, 1, 2)
    dst = np.float32([kp2[m.trainIdx].pt for m in bonnes]).reshape(-1, 1, 2)
    H, masque = cv2.findHomography(src, dst, cv2.RANSAC, 5.0)
    print("Correspondances cohérentes :", int(masque.sum()))

    # Dessiner le cadre de l'objet retrouvé dans la scène
    h, w = reference.shape
    coins = np.float32([[0, 0], [w, 0], [w, h], [0, h]]).reshape(-1, 1, 2)
    cadre = cv2.perspectiveTransform(coins, H)
    resultat = cv2.cvtColor(scene, cv2.COLOR_GRAY2BGR)
    cv2.polylines(resultat, [np.int32(cadre)], True, (0, 255, 0), 3)
else:
    print("Objet non trouvé")
```

Image — Points ORB détectés, puis correspondances entre la référence et une scène tourn…

Résultats mesurés avec la même référence, face à une scène qui contient l'objet (tourné de 25° et réduit à 70 %) et face à une photo qui ne le contient pas (la tasse de café) :

Étape	Objet présent	Objet absent
Correspondances brutes (meilleur candidat mutuel de chaque point)	186	173

Étape	Objet présent	Objet absent
Après le test du ratio	27	3
Après la vérification géométrique	26	trop peu pour conclure

> **Erreur fréquente** — Conclure que l'objet est présent parce qu'il y a beaucoup de correspondances. Chaque point trouve toujours un « meilleur candidat », même dans une image qui ne contient pas l'objet : on obtient ici 173 correspondances brutes avec une photo sans aucun rapport. Avoir beaucoup de correspondances ne garantit pas que l'objet est présent. Il faut filtrer (test du ratio) et vérifier que les points suivent une même déformation géométrique.

7.4 Exposé : la méthode SIFT

SIFT (*Scale-Invariant Feature Transform*, David Lowe, 2004) est l'ancêtre célèbre d'ORB. Plus précis mais plus lent, longtemps protégé par un brevet (expiré en 2020), il est disponible dans OpenCV avec `cv2.SIFT_create()`.

Sujet d'exposé (par groupe, 15 minutes), à préparer à partir de l'article original de Lowe et de la documentation d'OpenCV :

1. Quel problème SIFT résout-il ? Que signifie « invariant à l'échelle » ?
2. Comment SIFT détecte-t-il les points clés (différences de gaussiennes, extrema dans l'espace des échelles) ?
3. Comment calcule-t-il l'orientation et le descripteur de 128 valeurs (histogrammes d'orientation des gradients) ?
4. Démonstration : refaire l'exemple de la section 7.3 avec `cv2.SIFT_create()` et `cv2.NORM_L2`, puis comparer avec ORB : nombre de correspondances, temps de calcul, résistance à la rotation et au changement d'échelle.
5. Tableau comparatif SIFT / ORB et conclusion : quand choisir l'un ou l'autre ?

7.5 Les réseaux de neurones

Les trois méthodes précédentes reposent sur des règles ou des caractéristiques choisies par l'humain. Elles retrouvent **un** objet précis, mais ne savent pas reconnaître une **catégorie** d'objets : toutes les voitures, tous les visages, toutes les pommes, quelles que soient leur couleur et leur forme exacte. Les réseaux de neurones apprennent eux-mêmes les caractéristiques utiles à partir de milliers d'exemples (chapitres 8 à 10). Dans ce cours, nous utiliserons **YOLO**, qui sera l'objet du TP et du mini-projet (chapitre 11).

Exercices

1. Quelle est la circularité d'un carré de côté a ? Est-elle plus grande ou plus petite que celle d'un cercle ?
2. Dans le code de la section 7.2, que se passe-t-il si l'on remplace `0.03 * perimetre` par `0.2 * perimetre` ?
3. Deux signatures binaires valent `10110010` et `10011011`. Quelle est leur distance de Hamming ?
4. Pourquoi ORB calcule-t-il une orientation pour chaque point ?
5. Mini-défi : photographier la couverture d'un livre, puis le même livre posé sur une table encombrée. Le retrouver avec ORB et dessiner son cadre.

Corrigés

1. Aire = a^2, périmètre = $4a$. Circularité = $4\pi a^2 / (4a)^2 = \pi / 4 \approx$ **0,79**, plus petite que celle d'un cercle (1).
2. La tolérance devient très grande : les contours sont simplifiés à l'extrême. Un cercle peut alors donner un polygone de 4 sommets ou moins et être pris pour un rectangle ou un triangle.
3. On compare bit à bit : 1–1, 0–0, 1–0, 1–1, 0–1, 0–0, 1–1, 0–1. Trois bits diffèrent : distance = **3**.
4. Pour rendre la description indépendante de la rotation : les paires de pixels sont comparées dans un repère tourné selon l'orientation du point. Un même coin, photographié tourné, garde ainsi la même signature.
5. Utiliser la couverture comme `reference.jpg`, la photo de la table comme `scene.jpg`, et le code de la section 7.3. Si le livre est trop petit ou flou dans la scène, augmenter `nfeatures` ou rapprocher l'appareil.

Compétences acquises : choisir une méthode d'identification adaptée ; reconnaître des formes géométriques ; expliquer les étapes d'ORB ; retrouver et localiser un objet par ses points caractéristiques ; connaître les limites de chaque approche.

Chapitre 8 — Introduction à l'apprentissage automatique

Objectifs : distinguer un algorithme classique d'un modèle appris ; connaître le vocabulaire de l'apprentissage automatique ; entraîner et évaluer un premier classifieur d'images.

Prérequis : chapitres 3 à 6 ; installer `scikit-learn` (`pip install scikit-learn`).

8.1 Deux façons de résoudre un problème

Dans les chapitres 4 à 6, c'est **le programmeur** qui a fixé toutes les règles : le seuil T = 127, la plage de teintes du rouge, la taille du noyau, l'aire minimale d'une pièce. Cette approche fonctionne bien quand les règles sont simples et les images contrôlées.

Mais comment écrire à la main les règles qui distinguent un chat d'un chien, ou un « 4 » manuscrit d'un « 9 » ? Il y a trop de cas. L'**apprentissage automatique** (en anglais *machine learning*) inverse la démarche : on donne à la machine des **exemples avec la bonne réponse**, et elle trouve elle-même les règles.

Image — Algorithme classique : l'humain écrit les règles ; apprentissage automatique : …

	Algorithme classique	Modèle d'apprentissage automatique	Réseau de neurones
Qui fixe les règles ?	Le programmeur	L'algorithme, à partir des données	L'algorithme, à partir des données
Ce qui est appris	Rien	Quelques paramètres	Des milliers à des milliards de **poids**
Besoin de données	Aucun ou très peu	Des centaines d'exemples	Des milliers à des millions d'exemples
Exemple dans ce cours	Seuil d'Otsu, Canny (ch. 4–6)	k plus proches voisins (ce chapitre)	CNN, YOLO (ch. 9–11)

Un **réseau de neurones** est un modèle d'apprentissage automatique particulier : il est formé de petites unités de calcul, les **neurones**, organisées en **couches** et reliées par des **poids** (chapitre 9). Quand il comporte de nombreuses couches, on parle d'**apprentissage profond** (*deep learning*).

8.2 Le vocabulaire essentiel

Terme	Définition	Exemple
Données (exemples)	Ce que le modèle observe	Des images de chiffres
Caractéristiques (*features*)	Les nombres qui décrivent un exemple	Les 64 pixels d'une image 8 × 8
Étiquette (*label*)	La bonne réponse associée à un exemple	« 7 »

Terme	Définition	Exemple
Modèle	La fonction qui transforme les caractéristiques en prédiction	Un réseau de neurones
Paramètres	Les nombres que le modèle ajuste pendant l'apprentissage	Les poids d'un réseau
Entraînement	La phase où le modèle ajuste ses paramètres sur des exemples	`modele.fit(...)`
Prédiction (inférence)	L'utilisation du modèle sur une nouvelle donnée	`modele.predict(...)`

Types de tâches

- **Classification** : prédire une catégorie (chat ou chien, chiffre de 0 à 9).
- **Régression** : prédire un nombre (l'âge d'une personne sur une photo).
- **Détection** : prédire des catégories **et** des positions (boîtes autour des objets).
- **Segmentation** : prédire la catégorie de **chaque pixel**.

Quand chaque exemple d'entraînement est accompagné de sa bonne réponse, on parle d'**apprentissage supervisé**. C'est le cas dans tout ce cours.

8.3 Premier modèle : reconnaître des chiffres manuscrits

On utilise un jeu de données fourni avec scikit-learn : 1 797 images de chiffres de 8 × 8 pixels, chacune avec son étiquette.

Image — Exemples de chiffres manuscrits et leur représentation sous forme de 64 nombres

Le modèle choisi est celui des **k plus proches voisins** (k-NN). Son principe est intuitif : pour classer une nouvelle image, on cherche les k images d'entraînement qui lui ressemblent le plus (les 64 pixels les plus proches), et on choisit la réponse majoritaire parmi elles.

```
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier

chiffres = load_digits()                                    # 1 797 images de
8 x 8 pixels
X = chiffres.images.reshape(len(chiffres.images), -1)     # chaque image
devient 64 nombres
y = chiffres.target                                         # les bonnes
réponses (0 à 9)

# 80 % pour l'entraînement, 20 % pour le test
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=0)

modele = KNeighborsClassifier(n_neighbors=3)
modele.fit(X_train, y_train)                                # apprentissage

exactitude = modele.score(X_test, y_test)                   # évaluation sur
des images inédites
print("Exactitude sur le test :", round(exactitude * 100, 1), "%")   #
environ 98 %
print("Prédictions :", modele.predict(X_test[:5]))
print("Vraies réponses :", y_test[:5])
```

Interprétation : sans aucune règle écrite à la main sur la forme des chiffres, le modèle reconnaît correctement environ 98 % des images de test. Il a « appris » uniquement à partir d'exemples.

Limites : k-NN compare les pixels un par un. Si le chiffre est décalé de quelques pixels, agrandi ou tourné, la comparaison échoue. Sur de vraies photos (millions de pixels, fonds variés), il faut un modèle capable d'apprendre lui-même les bonnes caractéristiques : un réseau de neurones.

8.4 Un modèle appris dans OpenCV : la détection de visages

OpenCV contient déjà des modèles entraînés. Le détecteur de visages de Viola et Jones (2001) a été entraîné sur des milliers d'images de visages et de non-visages. Il utilise des motifs simples de zones claires et sombres (les caractéristiques de Haar), organisés en **cascade** : les fenêtres de l'image qui ne ressemblent clairement pas à un visage sont rejetées très vite, ce qui rend la détection rapide.

```
import cv2

img = cv2.imread("astronaute.jpg")
gris = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

detecteur = cv2.CascadeClassifier("haarcascade_frontalface_default.xml")
visages = detecteur.detectMultiScale(gris, scaleFactor=1.1, minNeighbors=5)

print("Visages trouvés :", len(visages))
for (x, y, w, h) in visages:
    cv2.rectangle(img, (x, y), (x + w, y + h), (0, 255, 0), 2)
cv2.imwrite("visages.jpg", img)
```

Image — Détection de visage par cascade de Haar sur une photo d'astronaute

Le fichier `haarcascade_frontalface_default.xml` contient les paramètres appris. Avec OpenCV 4, il est fourni avec la bibliothèque (`cv2.data.haarcascades + "haarcascade_frontalface_default.xml"`). Avec OpenCV 5, il faut le télécharger depuis le dépôt GitHub d'OpenCV (dossier `data/haarcascades`) et le placer à côté du programme.

Paramètres : `scaleFactor=1.1` réduit l'image de 10 % à chaque passage pour trouver des visages de tailles différentes ; `minNeighbors=5` exige plusieurs détections proches pour confirmer un visage (plus la valeur est grande, moins il y a de fausses alarmes).

> **À retenir** — Un modèle appris n'est pas magique : il ne vaut que par ses données d'entraînement. Le détecteur de Haar, entraîné sur des visages de face, détecte mal les visages de profil ou partiellement cachés.

> **Question de réflexion** — Un modèle de reconnaissance de visages entraîné presque uniquement sur des photos de personnes à peau claire fonctionnera-t-il aussi bien sur des personnes à peau foncée ? Quelles conséquences pratiques et éthiques cela peut-il avoir ?

Exercices

1. Pour chacune des tâches suivantes, dire s'il s'agit de classification, régression, détection ou segmentation : (a) estimer l'âge d'une personne ; (b) dire si une radio montre une fracture ; (c) entourer chaque voiture d'une image ; (d) colorier en rouge chaque pixel de route.

2. Dans le code des chiffres, remplacer `n_neighbors=3` par 1, puis 15. L'exactitude change-t-elle ?
3. Que se passe-t-il si l'on évalue le modèle avec `modele.score(X_train, y_train)` ? Pourquoi ce résultat est-il trompeur ?
4. Mini-défi : appliquer le détecteur de visages à une photo de groupe, puis faire varier `minNeighbors` de 1 à 10.

Corrigés

1. (a) régression ; (b) classification ; (c) détection ; (d) segmentation.
2. Les résultats restent proches de 97 à 99 % sur ce jeu de données facile ; un k très grand finit par mélanger des chiffres différents et fait baisser l'exactitude. Le choix de k se fait sur un ensemble de **validation**, pas sur le test.
3. On obtient un score très élevé (100 % avec k = 1, car chaque image est sa propre plus proche voisine). Il ne mesure pas la capacité à reconnaître des images nouvelles.
4. Avec `minNeighbors=1`, de fausses détections apparaissent (motifs pris pour des visages) ; avec une valeur élevée, certains vrais visages sont manqués. Il faut trouver un compromis.

Compétences acquises : distinguer approche classique et approche apprise ; utiliser le vocabulaire du domaine ; séparer données d'entraînement et de test ; entraîner et évaluer un classifieur ; utiliser un modèle pré-entraîné d'OpenCV.

Chapitre 9 — Réseaux de neurones et descente de gradient

Objectifs : décrire un neurone artificiel et un réseau ; calculer une propagation avant ; comprendre la fonction de perte, le gradient et la descente de gradient ; expliquer la rétropropagation ; reconnaître le surapprentissage.

Prérequis : chapitre 8 ; dérivée d'une fonction simple (par exemple, la dérivée de $(\theta - 3)^2$ est $2(\theta - 3)$) ; installer PyTorch (`pip install torch`).

9.1 L'intuition : apprendre en corrigeant ses erreurs

Imaginez un randonneur perdu dans le brouillard sur une colline, qui veut descendre dans la vallée. Il ne voit pas la vallée, mais il sent la **pente** sous ses pieds. À chaque pas, il avance dans la direction qui descend le plus. Pas après pas, il finit par atteindre le fond.

Un réseau de neurones apprend exactement ainsi :

- l'**altitude** est l'erreur du réseau (la **perte**) ;
- la **position** du randonneur, ce sont les paramètres du réseau (ses **poids**) ;
- la **pente** est le **gradient** ;
- la **longueur du pas** est le **taux d'apprentissage**.

Le réseau fait une prédiction, mesure son erreur, regarde dans quel sens modifier chaque poids pour réduire cette erreur, fait un petit pas dans ce sens, et recommence des milliers de fois.

9.2 Le neurone artificiel

Un neurone reçoit des **entrées** x_1, x_2, … (par exemple des valeurs de pixels). Chaque entrée est multipliée par un **poids** w, qui indique son importance. On ajoute un **biais** b, qui décale le résultat. Enfin, une **fonction d'activation** σ transforme la somme en sortie.

Image — Schéma d'un neurone artificiel : entrées, poids, biais, somme et activation

Image — Formules du neurone : somme pondérée puis activation sigmoïde

Symbole	Nom	Rôle
x_i	Entrée	Une donnée fournie au neurone
w_i	Poids	Importance de l'entrée (appris)
b	Biais	Décalage, comme l'ordonnée à l'origine d'une droite (appris)
z	Somme pondérée	$w_1x_1 + w_2x_2 + \dots + b$
σ	Fonction d'activation	Rend le neurone non linéaire
$\hat{y}$	Sortie (prédiction)	σ(z)

Exemple de calcul. Entrées x = (0,8 ; 0,2 ; 0,5), poids w = (0,4 ; −0,6 ; 0,9), biais b = −0,1.

```
z = 0,4 × 0,8 + (−0,6) × 0,2 + 0,9 × 0,5 + (−0,1)
  = 0,32 − 0,12 + 0,45 − 0,10
  = 0,55
ŷ = σ(0,55) = 1 / (1 + e^(−0,55)) ≈ 0,634
```

```
import numpy as np

x = np.array([0.8, 0.2, 0.5])        # entrées
w = np.array([0.4, -0.6, 0.9])       # poids
b = -0.1                             # biais

z = np.dot(w, x) + b                 # somme pondérée
y = 1 / (1 + np.exp(-z))             # activation sigmoïde
print("z =", round(z, 3), " sortie =", round(y, 3))   # z = 0.55  sortie =
0.634
```

Les fonctions d'activation. Sans elles, empiler des neurones reviendrait à une seule opération linéaire, incapable de représenter des formes complexes.

Image — Trois fonctions d'activation : sigmoïde, tanh et ReLU

Fonction	Formule	Sortie	Usage
Sigmoïde	$1 / (1 + e^{-z})$	entre 0 et 1	Probabilité d'une classe
Tanh	tanh(z)	entre −1 et 1	Réseaux récurrents
ReLU	max(0, z)	0 ou z	Couches cachées des réseaux modernes

9.3 Le réseau de neurones et la propagation avant

Un réseau organise les neurones en **couches** : une couche d'entrée, une ou plusieurs **couches cachées**, une couche de sortie. Chaque neurone d'une couche reçoit les sorties de tous les neurones de la couche précédente.

Image — Un réseau de neurones : couche d'entrée, deux couches cachées, couche de sortie

La **propagation avant** (*forward propagation*) consiste à faire passer les données de la couche d'entrée vers la sortie, couche par couche, pour obtenir une prédiction.

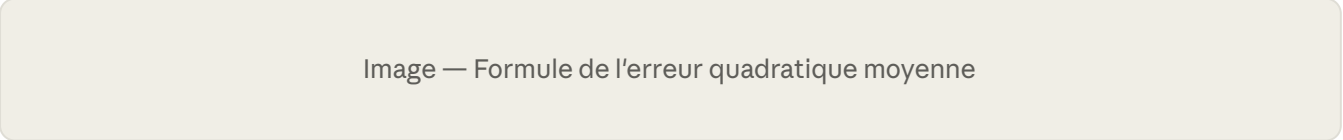
```
import torch
import torch.nn as nn

reseau = nn.Sequential(
    nn.Linear(4, 5),    # 4 entrées -> 5 neurones
    nn.ReLU(),
    nn.Linear(5, 4),    # 5 -> 4 neurones
    nn.ReLU(),
    nn.Linear(4, 2)     # 4 -> 2 sorties
)

x = torch.rand(1, 4)                # un exemple de 4 valeurs
print(reseau(x))                    # propagation avant : 2 valeurs en
sortie
print("Nombre de paramètres :", sum(p.numel() for p in reseau.parameters()))
# 59
```

9.4 La fonction de perte

La **fonction de perte** (*loss*) mesure l'écart entre la prédiction ŷ et la bonne réponse y. Plus elle est petite, meilleur est le modèle. Pour prédire un nombre, on utilise souvent l'**erreur quadratique moyenne** :



Exemple : le modèle prédit 2 et 4 alors que les bonnes réponses sont 3 et 3. L = ((2 − 3)² + (4 − 3)²) / 2 = (1 + 1) / 2 = 1.

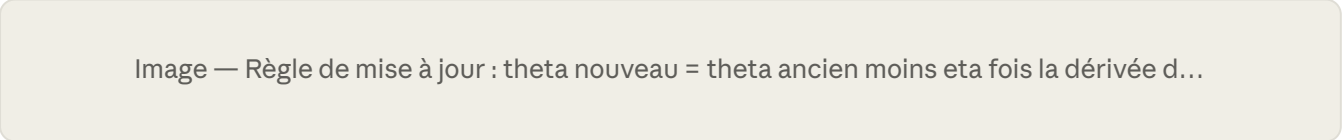
Pour une classification, on utilise plutôt l'**entropie croisée** (*cross-entropy*), qui pénalise fortement une prédiction confiante mais fausse. Le principe reste le même : un nombre qui mesure l'erreur.

9.5 Le gradient et la descente de gradient

Le **gradient** ∂L/∂θ est la dérivée de la perte par rapport à un paramètre θ. Il indique :

- par son **signe**, dans quel sens la perte augmente quand θ augmente ;
- par sa **valeur**, à quel point la perte est sensible à ce paramètre.

Pour diminuer la perte, on déplace donc chaque paramètre dans le **sens opposé** au gradient. C'est la **descente de gradient** :



Symbole	Signification
θ (thêta)	Un paramètre du modèle : un poids ou un biais
L	La perte, l'erreur à réduire
∂L/∂θ	Le gradient : la pente de L quand on fait varier θ
η (êta)	Le taux d'apprentissage : la longueur du pas

Pourquoi soustraire ? Si la pente est positive, la perte augmente quand θ augmente : il faut donc **diminuer** θ. Si la pente est négative, il faut **augmenter** θ. Le signe moins fait exactement cela dans les deux cas.

Exemple. Perte $L(\theta) = (\theta - 3)^2 + 1$, minimale pour θ = 3. Gradient : $\partial L/\partial\theta = 2(\theta - 3)$. Départ θ = −1, η = 0,2.

```
étape 0 : gradient = 2 × (−1 − 3) = −8    →  θ = −1 − 0,2 × (−8)   = 0,60
étape 1 : gradient = 2 × (0,6 − 3) = −4,8 →  θ = 0,6 − 0,2 × (−4,8) = 1,56
étape 2 : gradient = 2 × (1,56 − 3) = −2,88 → θ = 1,56 + 0,576      = 2,136
...      : θ se rapproche de 3
```

Image — Descente de gradient sur une courbe de perte : les points se rapprochent du min…

```
theta = -1.0          # point de départ
eta = 0.2             # taux d'apprentissage

for etape in range(10):
    perte = (theta - 3) ** 2 + 1
    gradient = 2 * (theta - 3)          # dérivée de la perte
    theta = theta - eta * gradient      # mise à jour
    print(etape, round(theta, 3), round(perte, 3))
# theta : 0.6, 1.56, 2.136, 2.482, ... 2.976 -> se rapproche de 3
```

9.6 Le taux d'apprentissage

Image — Même problème avec trois taux d'apprentissage : trop petit, bien choisi, trop g…

η	Comportement
Trop petit	L'apprentissage est très lent, il faut beaucoup d'étapes
Bien choisi	La perte diminue rapidement et se stabilise au minimum
Trop grand	Les pas dépassent le minimum ; la perte oscille ou augmente : on dit que l'apprentissage **diverge**

9.7 La rétropropagation

Un réseau peut avoir des millions de poids. La **rétropropagation du gradient** (*backpropagation*) calcule efficacement le gradient de la perte pour **chacun** d'eux. Elle part de l'erreur en sortie et remonte le réseau couche par couche, en appliquant la **règle de dérivation en chaîne** :

Image — Règle de dérivation en chaîne pour un poids w

Chaque facteur se calcule localement, dans une seule couche. On multiplie ces facteurs de la sortie vers l'entrée : d'où le nom « rétro »-propagation.

Image — La boucle d'entraînement : propagation avant, perte, rétropropagation, mise à j…

Exemple complet à la main (NumPy). On cherche w et b tels que y = w · x + b colle aux données, qui suivent en réalité y = 2x + 1. Les gradients de l'erreur quadratique moyenne sont calculés à la main.

```
import numpy as np

# Données : la vraie relation est y = 2x + 1 (que le modèle ne connaît pas)
x = np.array([0, 1, 2, 3, 4, 5], dtype=float)
y = 2 * x + 1

w, b = 0.0, 0.0       # paramètres au départ
eta = 0.02            # taux d'apprentissage

for epoque in range(1000):
    y_pred = w * x + b                    # 1. propagation avant
    erreur = y_pred - y
    perte = np.mean(erreur ** 2)          # 2. perte
```

```
    grad_w = np.mean(2 * erreur * x)       # 3. gradients (dérivées calculées
à la main)
    grad_b = np.mean(2 * erreur)
    w = w - eta * grad_w                   # 4. mise à jour
    b = b - eta * grad_b
    if epoque % 200 == 0:
        print(f"époque {epoque:4d}  perte = {perte:.4f}  w = {w:.3f}  b =
{b:.3f}")

print("Résultat : w =", round(w, 2), " b =", round(b, 2))   # w = 2.0  b =
1.0
```

Le même apprentissage avec PyTorch. PyTorch calcule les gradients automatiquement : `perte.backward()` réalise la rétropropagation, `optimiseur.step()` la mise à jour.

```
import torch

x = torch.tensor([0., 1., 2., 3., 4., 5.])
y = 2 * x + 1

w = torch.tensor(0.0, requires_grad=True)    # PyTorch suivra les calculs sur
w et b
b = torch.tensor(0.0, requires_grad=True)
optimiseur = torch.optim.SGD([w, b], lr=0.02)

for epoque in range(1000):
    y_pred = w * x + b                  # propagation avant
    perte = ((y_pred - y) ** 2).mean()  # perte
    optimiseur.zero_grad()              # remettre les gradients à zéro
    perte.backward()                    # rétropropagation : calcule les
gradients
    optimiseur.step()                   # mise à jour : w = w - lr * gradient

print(round(w.item(), 2), round(b.item(), 2))   # 2.0 1.0
```

9.8 Retour aux images : un neurone qui apprend un seuil

Au chapitre 4, le seuil T = 127 était choisi par le programmeur. Un neurone peut l'**apprendre** à partir de pixels étiquetés « fond » (0) ou « objet » (1).

```
import numpy as np

pixels = np.array([30, 45, 60, 80, 95, 110, 140, 160, 175, 190, 210, 230])
etiquettes = np.array([0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1])   # 0 = fond, 1 =
objet

x = pixels / 255.0          # normalisation entre 0 et 1
w, b = 0.0, 0.0
eta = 1.0

for epoque in range(5000):
    y_pred = 1 / (1 + np.exp(-(w * x + b)))          # sigmoïde
    w = w - eta * np.mean((y_pred - etiquettes) * x)  # gradient de
l'entropie croisée
    b = b - eta * np.mean(y_pred - etiquettes)

seuil = -b / w * 255         # la sortie vaut 0,5 quand w * x + b = 0
print("Seuil appris :", round(seuil))   # environ 125
```

Image — Un neurone apprend seul le seuil entre pixels de fond et pixels d'objet

Interprétation : le neurone place sa frontière (sortie = 0,5) entre les deux groupes de pixels, sans qu'on lui donne le seuil. Un réseau profond fait la même chose avec des millions de poids, et apprend des frontières beaucoup plus complexes : la forme d'un chat, d'une voiture ou d'un visage.

9.9 Époques, lots et itérations

Terme	Définition
Lot (*batch*)	Petit groupe d'exemples traités ensemble avant une mise à jour (par exemple 32 images)
Itération	Une mise à jour des poids, calculée sur un lot
Époque (*epoch*)	Un passage complet sur toutes les données d'entraînement

Exemple : 1 000 images d'entraînement, lots de 100 images. Une époque compte 1 000 / 100 = 10 itérations. Un entraînement de 20 époques fait donc 200 mises à jour des poids.

Travailler par lots est un compromis : un lot est assez grand pour donner une bonne estimation du gradient, et assez petit pour tenir en mémoire et permettre des mises à jour fréquentes.

9.10 Le surapprentissage

Un modèle **surapprend** (*overfitting*) quand il apprend par cœur ses exemples d'entraînement, y compris leur bruit, au lieu d'en retenir la règle générale. Son erreur d'entraînement est très faible, mais son erreur sur de nouvelles données est grande.

Image — Sous-apprentissage, bon apprentissage et surapprentissage sur les mêmes données

Comment le détecter ? On suit la perte sur l'ensemble de **validation** pendant l'entraînement : si elle remonte alors que la perte d'entraînement continue de baisser, le modèle surapprend.

Comment l'éviter ? Plus de données ; l'**augmentation de données** (créer de nouvelles images par rotation, miroir, changement de luminosité : exactement les transformations du chapitre 4) ; un modèle plus simple ; l'arrêt de l'entraînement au bon moment.

9.11 Deux gradients à ne pas confondre

Image — Gradient d'une image (variation d'intensité) et gradient d'une perte (variation…

	Gradient d'une image (chapitre 5)	Gradient de la perte (ce chapitre)
Ce qu'on dérive	L'intensité des pixels	L'erreur du modèle
Par rapport à quoi	La position (x, y) dans l'image	Les paramètres (poids, biais)
Calculé avec	Les filtres de Sobel	La rétropropagation
Sert à	Trouver les contours	Ajuster les poids pour réduire l'erreur

Le mot est le même parce que l'idée mathématique est la même : une dérivée, qui mesure comment une grandeur varie quand on en modifie une autre.

> **À retenir** — Apprendre = répéter quatre étapes : **prédire** (propagation avant), **mesurer l'erreur** (perte), **calculer les gradients** (rétropropagation), **corriger les poids** (descente de gradient).

Exercices

1. Un neurone a pour entrées $x = (1 ; 2)$, poids $w = (0{,}5 ; -0{,}25)$, biais $b = 0$. Calculer z, puis la sortie avec ReLU et avec la sigmoïde.
2. Avec $L(\theta) = (\theta - 3)^2 + 1$, $\theta = 5$ et $\eta = 0{,}1$, calculer une étape de descente de gradient. La perte a-t-elle diminué ?
3. Combien de paramètres (poids et biais) possède une couche `nn.Linear(4, 5)` ? Vérifier le total de 59 pour le réseau de la section 8.3.
4. Un jeu de 5 000 images est entraîné avec des lots de 50 images pendant 10 époques. Combien y a-t-il d'itérations ?
5. Un modèle a une exactitude de 99 % sur l'entraînement et de 70 % sur la validation. Que se passe-t-il ? Proposer deux remèdes.
6. Mini-défi : dans le code de la descente de gradient, essayer $\eta = 0{,}05$, puis $1{,}05$. Décrire ce qui se passe.

Corrigés

1. $z = 0{,}5 \times 1 + (-0{,}25) \times 2 + 0 =$ **0**. ReLU(0) = **0** ; sigmoïde(0) = 1 / (1 + 1) = **0,5**.
2. Gradient $= 2 \times (5 - 3) = 4$. Nouveau $\theta = 5 - 0{,}1 \times 4 =$ **4,6**. Perte avant : $(5 - 3)^2 + 1 = 5$; après : $(4{,}6 - 3)^2 + 1 =$ **3,56**. Elle a diminué.
3. Chacun des 5 neurones a 4 poids et 1 biais : $5 \times 4 + 5 =$ **25**. Total : $(4 \times 5 + 5) + (5 \times 4 + 4) + (4 \times 2 + 2) = 25 + 24 + 10 =$ **59**.
4. 5 000 / 50 = 100 itérations par époque, donc $100 \times 10 =$ **1 000 itérations**.
5. **Surapprentissage** : le modèle a mémorisé ses exemples. Remèdes : ajouter des données ou faire de l'augmentation de données ; simplifier le modèle ; arrêter l'entraînement quand la perte de validation remonte.
6. Avec $\eta = 0{,}05$, θ progresse lentement vers 3 (il n'y est pas encore après 10 étapes). Avec $\eta = 1{,}05$, θ saute d'un côté à l'autre du minimum en s'en éloignant : la perte augmente, l'apprentissage diverge.

Compétences acquises : calculer la sortie d'un neurone ; décrire un réseau et compter ses paramètres ; appliquer la règle de mise à jour ; expliquer la rétropropagation ; choisir un taux d'apprentissage ; détecter et limiter le surapprentissage.

Chapitre 10 — Deep learning appliqué aux images : OpenCV et les réseaux de neurones

Objectifs : comprendre ce qu'un réseau convolutif apprend ; préparer une image avec OpenCV avant de l'envoyer à un modèle ; classifier une image et détecter des objets avec un modèle pré-entraîné ; exploiter les prédictions avec OpenCV.

Prérequis : chapitres 5 (convolution), 8 et 9 ; installer `torch`, `torchvision` et `ultralytics`.

10.1 Des caractéristiques choisies aux caractéristiques apprises

	Méthodes classiques (ch. 4–6)	Deep learning
Caractéristiques	Choisies par l'humain : couleur HSV, contours, aire	Apprises par le réseau à partir des données
Réglages	Seuils et noyaux fixés à la main	Millions de poids ajustés par descente de gradient
Données nécessaires	Très peu	Beaucoup d'images étiquetées
Robustesse	Sensible à l'éclairage, à l'angle, au fond	Bonne, si les données d'entraînement sont variées
Explicabilité	Chaque étape se comprend	Le raisonnement interne est difficile à interpréter
Calcul	Léger	Lourd, souvent sur carte graphique (GPU)

Les deux approches ne s'opposent pas : on les **combine**. OpenCV charge, prépare et affiche les images ; le réseau de neurones fait la reconnaissance.

Image — La chaîne de traitement : OpenCV charge et prépare, le modèle prédit, OpenCV ex…

10.2 Le réseau de neurones convolutif (CNN)

Un **CNN** (*Convolutional Neural Network*) applique des convolutions comme celles du chapitre 5, mais les coefficients de ses noyaux sont des **poids appris** par descente de gradient. Les premières couches apprennent à détecter des bords et des couleurs, un peu comme Sobel ; les couches plus profondes combinent ces motifs simples en motifs complexes : un œil, une roue, un visage.

Couche	Rôle
Convolution	Extraire des motifs locaux avec des noyaux appris
ReLU	Garder les valeurs positives : max(0, x)
Max pooling	Réduire la taille en gardant le maximum de chaque zone 2 × 2
Couche dense	Combiner les motifs pour décider
Softmax	Transformer les scores en probabilités

Image — Architecture d'un petit CNN pour reconnaître des chiffres manuscrits

```
import torch
import torch.nn as nn

modele = nn.Sequential(
    nn.Conv2d(1, 16, kernel_size=3, padding=1),   # 16 noyaux 3 x 3 appris
    nn.ReLU(),
    nn.MaxPool2d(2),                               # 28 x 28 -> 14 x 14
    nn.Conv2d(16, 32, kernel_size=3, padding=1),
    nn.ReLU(),
    nn.MaxPool2d(2),                               # 14 x 14 -> 7 x 7
    nn.Flatten(),
    nn.Linear(32 * 7 * 7, 10)                      # 10 classes : chiffres 0
à 9
)

images = torch.rand(8, 1, 28, 28)       # un lot de 8 images grises 28 x 28
print(modele(images).shape)             # torch.Size([8, 10]) : 10 scores par
image
```

10.3 Préparer une image pour un réseau de neurones

Un réseau attend ses données dans un format précis, fixé lors de son entraînement. OpenCV sert à transformer l'image dans ce format.

Étape	Pourquoi	Avec OpenCV / NumPy
Conversion BGR → RGB	Les réseaux sont entraînés en RGB	`cv2.cvtColor(img, cv2.COLOR_BGR2RGB)`

Étape	Pourquoi	Avec OpenCV / NumPy
Redimensionnement	La taille d'entrée est fixe (par exemple 224 × 224)	`cv2.resize(img, (224, 224))`
Mise à l'échelle	Valeurs entre 0 et 1 au lieu de 0 et 255	`img / 255.0`
Normalisation	Valeurs centrées, comme pendant l'entraînement	`(img - moyenne) / ecart_type`
Réorganisation	PyTorch attend (canaux, hauteur, largeur)	`np.transpose(img, (2, 0, 1))`
Dimension du lot	Le réseau traite des lots d'images	`np.expand_dims(img, 0)`

Image — Formule de normalisation d'un pixel

Les valeurs μ = (0,485 ; 0,456 ; 0,406) et σ = (0,229 ; 0,224 ; 0,225) sont la moyenne et l'écart-type des canaux R, G, B du jeu de données ImageNet, sur lequel la plupart des modèles publics ont été entraînés.

```
import cv2
import numpy as np

img = cv2.imread("chat.jpg")                          # 1. lecture (BGR, 0-255)
img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)            # 2. BGR -> RGB
img = cv2.resize(img, (224, 224))                     # 3. taille attendue par
le modèle
img = img.astype(np.float32) / 255.0                  # 4. valeurs entre 0 et 1

moyenne = np.array([0.485, 0.456, 0.406])             # 5. normalisation par
canal
ecart_type = np.array([0.229, 0.224, 0.225])
img = (img - moyenne) / ecart_type

img = np.transpose(img, (2, 0, 1))                    # 6. (H, W, C) -> (C, H,
W)
lot = np.expand_dims(img, axis=0)                     # 7. ajouter la dimension
du lot
print(lot.shape)                                      # (1, 3, 224, 224)
```

Image — Les étapes de préparation d'une image : lecture BGR, conversion RGB, redimensio…

OpenCV propose aussi une fonction qui réalise une partie de ces étapes en une ligne :

```
import cv2

img_bgr = cv2.imread("chat.jpg")
lot = cv2.dnn.blobFromImage(img_bgr, scalefactor=1 / 255.0, size=(224, 224),
swapRB=True)
print(lot.shape)    # (1, 3, 224, 224) ; swapRB=True fait la conversion BGR -
> RGB
```

> **Erreur fréquente** — Oublier la conversion BGR → RGB ou utiliser une autre normalisation que celle de l'entraînement. Le programme fonctionne sans erreur… mais les prédictions sont mauvaises, car le réseau reçoit des images différentes de celles qu'il a appris à reconnaître.

10.4 Classifier une image avec un modèle pré-entraîné

Un modèle **pré-entraîné** a déjà appris sur un grand jeu de données ; on peut l'utiliser directement. Ici, ResNet-18, entraîné sur ImageNet (1 000 classes d'objets et d'animaux).

```
import cv2
import numpy as np
import torch
from torchvision import models

poids = models.ResNet18_Weights.DEFAULT
modele = models.resnet18(weights=poids)      # télécharge les poids la
première fois
modele.eval()                                # mode « utilisation », pas «
entraînement »

# Préparation de l'image avec OpenCV (section 9.3)
img = cv2.imread("chat.jpg")
img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
img = cv2.resize(img, (224, 224)).astype(np.float32) / 255.0
img = (img - np.array([0.485, 0.456, 0.406])) / np.array([0.229, 0.224,
0.225])
tenseur = torch.from_numpy(img.transpose(2, 0, 1)).float().unsqueeze(0)

# Prédiction
with torch.no_grad():                        # pas de calcul de gradient
```

```
    scores = modele(tenseur)
probas = torch.softmax(scores, dim=1)
proba, indice = probas.max(dim=1)
print(poids.meta["categories"][indice.item()], round(proba.item(), 2))
```

Le même travail avec la bibliothèque Ultralytics, qui fait la préparation toute seule :

```
from ultralytics import YOLO

classifieur = YOLO("yolov8n-cls.pt")         # classifieur pré-entraîné sur
ImageNet
resultat = classifieur("chat.jpg")[0]
print(classifieur.names[resultat.probs.top1],
round(float(resultat.probs.top1conf), 2))
```

Résultat obtenu sur la photo du chat utilisée dans ce cours (cinq classes les plus probables) :

Classe prédite	Probabilité
Egyptian cat	0,84
tabby (chat tigré)	0,10
tiger cat	0,05
lynx	0,02
cheetah (guépard)	0,00

Interprétation : le modèle hésite entre plusieurs races de chats tigrés, ce qui est raisonnable vu la photo. Les probabilités sont données par la fonction softmax et leur somme vaut 1.

10.5 Détecter des objets avec YOLO

YOLO (*You Only Look Once*) est un CNN de **détection** : il trouve plusieurs objets dans une image et donne pour chacun une boîte, une classe et un score de confiance. Il regarde l'image **une seule fois** : il la découpe en grille et prédit pour chaque case des boîtes candidates. Deux filtres gardent ensuite les bonnes boîtes :

1. le **seuil de confiance** supprime les boîtes dont le score est trop faible ;
2. la **suppression des non-maxima** (NMS) garde, parmi plusieurs boîtes qui se chevauchent fortement sur le même objet, uniquement la plus confiante.

Image — YOLO : la grille, les boîtes candidates, puis le résultat après seuil et NMS

Le modèle pré-entraîné connaît les 80 classes du jeu de données COCO (personne, voiture, chat, tasse, vélo…).

```
from ultralytics import YOLO

modele = YOLO("yolov8n.pt")                    # version « nano », téléchargée la première fois
resultat = modele("photo.jpg", conf=0.5)[0]    # conf : seuil de confiance

for boite in resultat.boxes:
    classe = modele.names[int(boite.cls)]
    score = float(boite.conf)
    x1, y1, x2, y2 = boite.xyxy[0].tolist()     # coins de la boîte
    print(classe, round(score, 2), int(x1), int(y1), int(x2), int(y2))

resultat.save("photo_annotee.jpg")             # image avec les boîtes dessinées
```

Image — Détections réelles de YOLOv8n sur trois images de test : chat, tasse et table, …

Sur une vidéo ou une webcam, on combine la boucle de lecture du chapitre 2 et la détection :

```
import cv2
from ultralytics import YOLO

modele = YOLO("yolov8n.pt")
video = cv2.VideoCapture("rue.mp4")            # 0 pour la webcam

while True:
    ok, image = video.read()                    # OpenCV : lecture
    if not ok:
        break
    resultat = modele(image, conf=0.5, verbose=False)[0]   # modèle : prédiction
    image_annotee = resultat.plot()             # boîtes et étiquettes dessinées
    nombre = len(resultat.boxes)
    cv2.putText(image_annotee, f"Objets : {nombre}", (20, 40),
                cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 0, 255), 2)        # OpenCV
```

```
: affichage
    cv2.imshow("YOLO", image_annotee)
    if cv2.waitKey(1) == ord("q"):
        break

video.release()
cv2.destroyAllWindows()
```

10.6 Quelques applications

Tâche	Ce que prédit le modèle	Exemple d'outil	Domaine
Classification	Une classe par image	ResNet, `yolov8n-cls`	Tri de photos, diagnostic sur radio
Détection de visages	Une boîte par visage	Cascade de Haar (ch. 8), détecteurs par réseau	Appareil photo, contrôle d'accès
Détection d'objets	Boîtes, classes et scores	YOLO	Vidéosurveillance, conduite autonome, robotique
Segmentation	Une classe par pixel	`yolov8n-seg`, U-Net	Médecine, cartographie, retouche photo

> **Question de réflexion** — Au chapitre 6, il a fallu régler à la main les plages de couleur de la pomme. Comment YOLO pourrait-il apprendre à détecter des pommes, même vertes ou à moitié cachées ? C'est l'objet du mini-projet du chapitre 11.

Exercices

1. Un réseau attend des images de 224 × 224 en RGB. Une photo lue par OpenCV a la forme (480, 640, 3). Indiquer les deux transformations indispensables et la forme finale du tenseur envoyé au réseau.
2. Un pixel rouge vaut 255 dans le canal R. Que vaut-il après mise à l'échelle puis normalisation avec $\mu = 0{,}485$ et $\sigma = 0{,}229$?
3. Dans le premier exemple YOLO, remplacer `conf=0.5` par `conf=0.2`, puis par `conf=0.9`. Que constate-t-on ?

4. Mini-défi : avec la webcam, n'afficher que les personnes (`modele.names[int(boite.cls)] == "person"`) et compter combien il y en a à chaque image.

Corrigés

1. `cv2.cvtColor(img, cv2.COLOR_BGR2RGB)` puis `cv2.resize(img, (224, 224))` ; après normalisation, réorganisation et ajout du lot, la forme est **(1, 3, 224, 224)**.
2. 255 / 255 = 1, puis (1 − 0,485) / 0,229 ≈ **2,25**.
3. Avec 0,2, des objets moins sûrs apparaissent, dont des fausses détections ; avec 0,9, seuls les objets très nets restent, et certains vrais objets disparaissent (sur la photo de la tasse, la table à 0,72 n'est plus affichée).
4. Dans la boucle, parcourir `resultat.boxes`, garder les boîtes de classe « person », les dessiner avec `cv2.rectangle` et afficher leur nombre avec `cv2.putText`.

Compétences acquises : expliquer ce qu'apprend un CNN ; préparer une image pour un réseau ; utiliser un classifieur et un détecteur pré-entraînés ; combiner OpenCV et un modèle de deep learning dans une même application.

Chapitre 11 — Mini-projet : entraîner son propre détecteur YOLO

Objectifs : constituer et annoter un jeu de données ; entraîner YOLO par transfert d'apprentissage ; évaluer le modèle ; l'intégrer dans un programme OpenCV qui traite une vidéo.

Prérequis : l'ensemble de la Partie I, en particulier les chapitres 4 (transformations, utiles pour l'augmentation de données), 9 (entraînement, surapprentissage) et 10 (YOLO pré-entraîné).

Ce mini-projet réunit toutes les compétences de la partie : OpenCV pour lire et afficher, l'apprentissage pour reconnaître, et la démarche expérimentale pour évaluer.

Un modèle préentraîné ne connaît que ses 80 classes. Pour détecter un objet nouveau (mangue, casque de moto, bouteille plastique), on entraîne YOLO sur ses propres images annotées.

11.1 Le transfert d'apprentissage

On ne part pas de zéro : on reprend un modèle déjà entraîné, qui sait détecter bords, textures et formes, et on l'ajuste sur nos images. Quelques centaines d'images suffisent alors.

11.2 Les étapes du projet

1. **Choisir un problème** (je vais definir!!!)

2. **Collecter les images** : 150 à 300 par classe, variées (angle, distance, lumière, fond).
3. **Annoter** : tracer une boîte autour de chaque objet avec un outil comme Label Studio, CVAT ou Roboflow.
4. **Répartir** les images : 70 % entraînement, 20 % validation, 10 % test.
5. **Entraîner**, puis **évaluer** sur les images de test.
6. **Intégrer** le modèle dans un programme.
7. Je vais fournir une base de code :)

11.3 Le format d'annotation YOLO

Un fichier texte par image, une ligne par objet. Les coordonnées sont divisées par la taille de l'image, donc comprises entre 0 et 1.

```
<classe> <x_centre> <y_centre> <largeur> <hauteur>
0 0.512 0.430 0.210 0.330
```

Exemple de conversion : une boîte de (100, 50) à (300, 250) dans une image 640 × 480.

```
x_centre = (100 + 300) / 2 / 640 = 0,3125
y_centre = (50 + 250) / 2 / 480 = 0,3125
largeur  = (300 − 100) / 640     = 0,3125
hauteur  = (250 − 50) / 480      = 0,4167
```

11.4 Organisation des fichiers

```
dataset/
├── data.yaml
├── images/   train/  val/  test/
└── labels/   train/  val/  test/
```

Fichier `data.yaml` :

```
path: dataset
train: images/train
val: images/val
test: images/test
names:
  0: casque
  1: sans_casque
```

11.5 Entraîner et évaluer

```
from ultralytics import YOLO

modele = YOLO("yolov8n.pt")                          # point de départ
préentraîné
modele.train(data="dataset/data.yaml", epochs=50, imgsz=640)

# Le meilleur modèle est enregistré dans runs/detect/train/weights/best.pt
meilleur = YOLO("runs/detect/train/weights/best.pt")
metriques = meilleur.val(data="dataset/data.yaml", split="test")
print("mAP50 :", metriques.box.map50)
```

Mesure	Question	Formule
Précision	Parmi les détections, combien sont justes ?	VP / (VP + FP)
Rappel	Parmi les vrais objets, combien sont trouvés ?	VP / (VP + FN)
mAP50	Qualité globale, une détection compte si sa boîte recouvre au moins la moitié de l'objet	moyenne sur les classes

VP = vrais positifs, FP = faux positifs, FN = faux négatifs (objets manqués).

Exemple : 10 casques dans les images de test. Le modèle fait 8 détections, dont 6 justes. Précision = 6 / 8 = 0,75 ; rappel = 6 / 10 = 0,60.

11.6 Intégrer le modèle dans un programme

On place la détection dans une fonction, ce qui permet de la réutiliser.

```
# detection.py
from ultralytics import YOLO

modele = YOLO("runs/detect/train/weights/best.pt")

def detecter(image):
    """Renvoie la liste des noms de classes détectées dans l'image."""
    resultat = modele(image, conf=0.5)[0]
    return [modele.names[int(b.cls)] for b in resultat.boxes]
```

```
# application.py
import cv2
from detection import detecter

cap = cv2.VideoCapture("trafic.mp4")
while True:
    ok, frame = cap.read()
    if not ok:
        break
    classes = detecter(frame)
    print("Sans casque :", classes.count("sans_casque"))
cap.release()
```

Projet

Par binôme : choisir un objet, annoter au moins 150 images, entraîner un modèle, mesurer précision, rappel et mAP50, puis présenter une démonstration sur une vidéo.

Compétences acquises : constituer et annoter un jeu de données ; entraîner un détecteur par transfert d'apprentissage ; interpréter précision, rappel et mAP ; intégrer un modèle entraîné dans une application OpenCV.

Partie II — Traitement automatique du langage naturel

Le TALN apprend aux machines à analyser, comprendre et produire du langage humain : traduction, résumé, analyse de sentiments, assistants conversationnels. Cette partie va du texte brut jusqu'aux modèles neuronaux de traduction.

Les grandes étapes du TALN

Un système de TALN transforme un texte brut en décision ou en nouveau texte, en passant par quatre étapes. Chaque étape s'appuie sur la précédente.

Image — Les quatre étapes du TALN appliquées à la phrase « Fatou a payé les frais d'ins…

Étape	Question posée	Où dans le cours
1. Segmentation	Où commencent et finissent les phrases et les mots ?	NLTK, spaCy (chapitre 12)
2. Analyse syntaxique	Quelle est la catégorie de chaque mot, et qui dépend de qui ?	spaCy (chapitre 12)

Étape	Question posée	Où dans le cours
3. Analyse sémantique	Que signifient les mots ? Quelles entités, quelles relations ?	entités nommées, embeddings (chapitres 12 et 14)
4. Compréhension	Que faut-il faire de ce texte ?	classification, traduction, Transformers (chapitres 15 à 18)

Les modèles récents ne séparent plus toujours ces étapes : un Transformer apprend la syntaxe et le sens en même temps. Les distinguer reste utile pour comprendre un système et pour savoir où une erreur se produit.

Les tâches de compréhension

« Comprendre » un texte n'a de sens que pour une tâche précise. Les exemples ci-dessous viennent d'un service de scolarité qui reçoit des questions d'étudiants.

Tâche	Ce que fait la machine	Exemple
Classification de texte	Attribue une catégorie à un texte	« Quand payer les frais ? » → paiement
Extraction d'informations	Repère des éléments précis : noms, dates, montants	« Fatou a payé 25 000 FCFA le 3 octobre » → PERSONNE, MONTANT, DATE
Réponse aux questions	Trouve la réponse dans un texte donné	« Quand commencent les examens ? » → « en janvier »
Analyse des intentions	Devine ce que veut l'utilisateur	« Je veux annuler mon inscription » → ANNULER_INSCRIPTION
Recherche sémantique	Retrouve les documents de même sens, même sans mot commun	« aide financière » retrouve le document sur les bourses
Résumé automatique	Produit une version courte d'un texte long	un règlement de 10 pages → 5 lignes

Le chapitre 18 montre comment mesurer la qualité de chacune de ces tâches.

Le rôle des réseaux de neurones dans le langage

Les méthodes classiques reposent sur des règles écrites à la main ou sur des comptages de mots (TF-IDF, chapitre 13). Elles ne voient ni les synonymes ni l'ordre des mots. Les réseaux de neurones apprennent ces régularités à partir d'exemples, en quatre progrès successifs :

1. représenter le sens d'un mot par un vecteur (embeddings, chapitre 14) ;
2. lire une phrase dans l'ordre en gardant une mémoire (RNN et LSTM, chapitre 15) ;
3. produire une phrase à partir d'une autre (Seq2Seq et traduction, chapitre 16) ;
4. relier directement tous les mots d'un texte entre eux (attention et Transformers, chapitre 17).

Chaque chapitre corrige une limite du précédent : c'est le fil conducteur de cette partie.

Chapitre 12 — Prétraitement et analyse syntaxique

Avant tout modèle, un texte doit être découpé, nettoyé et analysé grammaticalement.

12.1 Les étapes du prétraitement

Étape	Rôle	Exemple
Segmentation	Découper le texte en phrases	« Il pleut. Je reste. » → 2 phrases
Tokenisation	Découper une phrase en mots (tokens)	« Le chat dort » → [Le, chat, dort]
Normalisation	Minuscules, suppression de la ponctuation	« Bonjour ! » → « bonjour »
Mots vides	Retirer les mots très fréquents	le, de, et, à…
Racinisation (stemming)	Couper la fin des mots	« mangeaient » → « mang »
Lemmatisation	Retrouver la forme du dictionnaire	« mangeaient » → « manger »

12.2 Exemples avec NLTK

```
import nltk
nltk.download("punkt")
nltk.download("punkt_tab")
nltk.download("stopwords")

from nltk.tokenize import sent_tokenize, word_tokenize
from nltk.corpus import stopwords
from nltk.stem.snowball import FrenchStemmer

texte = "Les étudiants révisent le cours. Ils préparent l'examen de TALN !"

# Segmentation en phrases
```

```
print(sent_tokenize(texte, language="french"))

# Tokenisation en mots
mots = word_tokenize(texte, language="french")
print(mots)

# Normalisation et suppression des mots vides
mots_vides = stopwords.words("french")
mots = [m.lower() for m in mots if m.isalpha() and m.lower() not in
mots_vides]
print(mots)

# Racinisation
stemmer = FrenchStemmer()
print([stemmer.stem(m) for m in mots])
```

Fréquence des mots :

```
from nltk import FreqDist

freq = FreqDist(word_tokenize("le chat mange le poisson et le chien mange",
language="french"))
print(freq.most_common(3))     # [('le', 3), ('mange', 2), ('chat', 1)]
```

Les n-grammes :

```
from nltk import ngrams

phrase = ["le", "chat", "mange", "le", "poisson"]
print(list(ngrams(phrase, 2)))   # bigrammes
```

12.3 Exemples avec spaCy

spaCy fait en une seule passe la tokenisation, la lemmatisation et l'analyse syntaxique.

```
import spacy
nlp = spacy.load("fr_core_news_sm")

doc = nlp("Les chats mangeaient les poissons du marché.")

for token in doc:
    print(token.text, token.lemma_, token.pos_)
```

Résultat attendu :

Mot	Lemme	Catégorie (POS)
Les	le	DET (déterminant)
chats	chat	NOUN (nom)
mangeaient	manger	VERB (verbe)
les	le	DET
poissons	poisson	NOUN
du	de	ADP (préposition)
marché	marché	NOUN

12.4 Analyse syntaxique : les dépendances

L'analyse en dépendances relie chaque mot au mot dont il dépend. On obtient un arbre dont la racine est le verbe principal.

```
doc = nlp("Le professeur enseigne le TALN.")

for token in doc:
    print(token.text, "--", token.dep_, "-->", token.head.text)
```

```
              enseigne  (racine)
             /         \
         nsubj          obj
           /              \
    professeur            TALN
        |                   |
       det                 det
        |                   |
        Le                  le
```

Visualiser l'arbre dans un notebook :

```
from spacy import displacy
displacy.render(doc, style="dep")
```

12.5 Reconnaissance d'entités nommées

```
doc = nlp("Fatou est partie de Cotonou pour Paris avec Air France en mars
2026.")

for entite in doc.ents:
    print(entite.text, "→", entite.label_)
# Fatou → PER   Cotonou → LOC   Paris → LOC   Air France → ORG ...
```

Les résultats exacts dépendent du modèle : le petit modèle `fr_core_news_sm` peut faire des erreurs, ce qui est un bon sujet de discussion.

12.6 La notion de vocabulaire

Le **vocabulaire** est la liste de tous les tokens qu'un modèle connaît, chacun associé à un numéro. C'est le pont entre le texte et les nombres : un modèle ne voit jamais « chat », il voit l'indice 3.

On le construit à partir du corpus d'entraînement :

1. tokeniser tous les textes ;
2. compter les tokens, et souvent ne garder que les plus fréquents ;
3. donner un numéro à chaque token ;
4. ajouter des **jetons spéciaux**.

Jeton spécial	Rôle
`<pad>`	Remplissage : toutes les phrases d'un lot doivent avoir la même longueur
`<unk>`	Mot inconnu, absent du vocabulaire
`<debut>`, `<fin>`	Début et fin de phrase, utiles en traduction (chapitre 16)

```
from collections import Counter

corpus = ["le chat mange le poisson",
          "le chien mange la viande",
          "le chat dort"]

compteur = Counter(mot for phrase in corpus for mot in phrase.split())
print(compteur.most_common(3))

vocab = {"<pad>": 0, "<unk>": 1}           # jetons spéciaux
for mot in compteur:
    vocab[mot] = len(vocab)
print("Taille du vocabulaire :", len(vocab))

def encoder(phrase):
    return [vocab.get(mot, vocab["<unk>"]) for mot in phrase.split()]

print(encoder("le chat mange la souris"))   # « souris » est inconnu -> <unk>
```

Résultat :

```
[('le', 4), ('chat', 2), ('mange', 2)]
Taille du vocabulaire : 10
[2, 3, 4, 7, 1]
```

« souris » n'apparaît pas dans le corpus : il devient `<unk>` (indice 1) et le modèle perd cette information. C'est le problème des **mots hors vocabulaire** (OOV, *out of vocabulary*).

Choisir la taille du vocabulaire est un compromis. Trop petit, il produit beaucoup de `<unk>`. Trop grand, chaque mot rare a peu d'exemples, et les couches d'embedding et de sortie grossissent : 50 000 mots × 300 dimensions = 15 millions de paramètres pour la seule table d'embeddings. Les modèles modernes découpent donc les mots rares en sous-mots (section 17.6) : BERT utilise un vocabulaire d'environ 30 000 tokens, GPT-2 de 50 257.

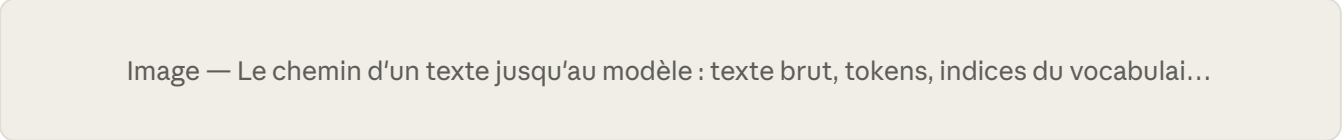
À vous : encoder « le chien dort sur le lit » avec ce vocabulaire. Réponse : [2, 6, 9, 1, 2, 1], soit deux mots inconnus.

Exercices

1. Comparer la racinisation NLTK et la lemmatisation spaCy sur : chevaux, mangeaient, sommes, meilleures.
2. Extraire tous les sujets (`token.dep_ == "nsubj"`) d'un texte de presse.
3. Compter les lieux (`LOC`) cités dans un article.

Chapitre 13 — Représenter le texte par des nombres

Un modèle ne lit que des nombres : il faut transformer mots et documents en vecteurs.



Ce chapitre traite des premières étapes de ce chemin avec des méthodes de comptage ; le chapitre 14 remplace les comptages par des vecteurs appris.

13.1 One-Hot Encoding

Chaque mot devient un vecteur de 0 avec un seul 1 à sa position dans le vocabulaire.

Mot	chat	chien	poisson
chat	1	0	0
chien	0	1	0
poisson	0	0	1

Limites : un vocabulaire de 50 000 mots donne des vecteurs de 50 000 cases, et tous les mots sont à la même distance les uns des autres (« chat » n'est pas plus proche de « chien » que de « voiture »).

13.2 Sac de mots (Bag of Words)

Un document est représenté par le nombre d'occurrences de chaque mot, sans tenir compte de l'ordre.

```
from sklearn.feature_extraction.text import CountVectorizer

corpus = ["le chat mange le poisson",
          "le chien mange la viande",
          "le chat dort"]

bow = CountVectorizer()
X = bow.fit_transform(corpus)
print(bow.get_feature_names_out())
print(X.toarray())
```

Limite : « le chien mord l'homme » et « l'homme mord le chien » ont le même vecteur. Les **n-grammes** (`CountVectorizer(ngram_range=(1, 2))`) conservent un peu d'ordre en comptant aussi les paires de mots.

13.3 TF-IDF : la formule

TF-IDF donne un poids élevé à un mot **fréquent dans un document** mais **rare dans le corpus**.

Image — Formules de TF, IDF et TF-IDF

N est le nombre de documents et df(t) le nombre de documents qui contiennent le mot t.

13.4 Calcul pas à pas

Corpus de N = 3 documents :

- D1 : « le chat mange le poisson » (5 mots)
- D2 : « le chien mange la viande » (5 mots)
- D3 : « le chat dort » (3 mots)

Étape 1 — TF : occurrences divisées par la longueur du document.

Mot	TF dans D1	TF dans D2	TF dans D3
le	2/5 = 0,400	1/5 = 0,200	1/3 = 0,333
chat	1/5 = 0,200	0	1/3 = 0,333
mange	1/5 = 0,200	1/5 = 0,200	0
poisson	1/5 = 0,200	0	0
chien	0	1/5 = 0,200	0
la	0	1/5 = 0,200	0
viande	0	1/5 = 0,200	0
dort	0	0	1/3 = 0,333

Étape 2 — IDF : dans combien de documents le mot apparaît-il ?

Mot	df	N / df	IDF = $\log_{10}$(N / df)
le	3	1	0
chat, mange	2	1,5	0,176
poisson, chien, la, viande, dort	1	3	0,477

Étape 3 — TF-IDF = TF × IDF.

Mot	D1	D2	D3
le	0,400 × 0 = 0	0,200 × 0 = 0	0,333 × 0 = 0
chat	0,200 × 0,176 = 0,035	0	0,333 × 0,176 = 0,059
mange	0,200 × 0,176 = 0,035	0,200 × 0,176 = 0,035	0
poisson	0,200 × 0,477 = 0,095	0	0
chien	0	0,200 × 0,477 = 0,095	0
la	0	0,200 × 0,477 = 0,095	0
viande	0	0,200 × 0,477 = 0,095	0
dort	0	0	0,333 × 0,477 = 0,159

Image — La matrice TF-IDF du mini-corpus, plus la case est foncée plus le mot est carac…

Interprétation :

- « le » vaut 0 : il apparaît partout, il ne distingue aucun document.
- « dort » est le mot le plus caractéristique de D3 (0,159).
- « chat » pèse plus dans D3 que dans D1, car D3 est plus court.

13.5 Le même calcul en Python

```
import math

corpus = ["le chat mange le poisson", "le chien mange la viande", "le chat
dort"]
docs = [d.split() for d in corpus]
N = len(docs)

def tf(mot, doc):
    return doc.count(mot) / len(doc)

def idf(mot):
    df = sum(1 for doc in docs if mot in doc)
    return math.log10(N / df)

for mot in ["le", "chat", "dort"]:
    print(mot, [round(tf(mot, d) * idf(mot), 3) for d in docs])
# le   [0.0, 0.0, 0.0]
# chat [0.035, 0.0, 0.059]
# dort [0.0, 0.0, 0.159]
```

Avec scikit-learn :

```
from sklearn.feature_extraction.text import TfidfVectorizer

tfidf = TfidfVectorizer()
X = tfidf.fit_transform(corpus)
print(tfidf.get_feature_names_out())
print(X.toarray().round(3))
```

scikit-learn utilise une formule lissée, IDF = ln((1 + N) / (1 + df)) + 1, puis normalise chaque document. Les valeurs diffèrent donc du calcul manuel, mais l'ordre d'importance des mots est le même.

13.6 Limite

Pour TF-IDF, « voiture » et « automobile » sont deux mots sans aucun lien. Les embeddings du chapitre 14 résolvent ce problème.

Exercices

1. Ajouter D4 « le poisson dort » et recalculer l'IDF de « poisson » et de « dort ».
2. Calculer la similarité cosinus entre D1 et D3 (réponse : environ 0,11, seul « chat » est commun).

Chapitre 14 — Embeddings : représenter le sens des mots

Un embedding est un vecteur court (50 à 300 nombres) appris de sorte que des mots employés dans les mêmes contextes aient des vecteurs proches.

Image — Trois familles de représentations : comptages, embeddings statiques, embeddings…

Le chapitre 13 a couvert la première famille. Ce chapitre traite la deuxième ; la troisième arrive à la section 14.5 et au chapitre 17.

14.1 L'idée : un mot se reconnaît à ses voisins

« Le ___ miaule sur le toit » : le contexte suffit à deviner « chat ». C'est l'**hypothèse distributionnelle** : deux mots qui apparaissent dans les mêmes contextes ont des sens proches.

14.2 Word2Vec

Word2Vec entraîne un petit réseau à prédire les mots voisins :

- **CBOW** : prédire le mot central à partir de ses voisins ;
- **Skip-gram** : prédire les voisins à partir du mot central.

```
Phrase  : le  chat  mange  le  poisson      (fenêtre de 2 mots)
Paires skip-gram pour « mange » :
          (mange, le) (mange, chat) (mange, le) (mange, poisson)
```

Image — CBOW prédit le mot central à partir de ses quatre voisins ; Skip-gram prédit le…

Dans les deux cas, la tâche de prédiction n'est qu'un prétexte : ce qui nous intéresse, ce sont les embeddings appris au passage (la boîte verte). CBOW est plus rapide ; Skip-gram donne de meilleurs vecteurs pour les mots rares.

Exemple 1 — Entraîner Word2Vec avec gensim et chercher les mots proches.

```
from gensim.models import Word2Vec

phrases = [["le", "chat", "boit", "du", "lait"],
           ["le", "chien", "mange", "de", "la", "viande"],
           ["le", "chat", "chasse", "la", "souris"],
           ["le", "chien", "chasse", "le", "chat"],
           ["la", "voiture", "roule", "sur", "la", "route"],
           ["le", "camion", "roule", "sur", "la", "route"]]

modele = Word2Vec(phrases, vector_size=50, window=3, min_count=1, sg=1,
epochs=200)

print(modele.wv["chat"][:5])                    # les 5 premiers nombres du
vecteur
print(modele.wv.most_similar("voiture", topn=3))  # les 3 mots les plus
proches
print(modele.wv.similarity("chat", "chien"))      # similarité cosinus
```

Sur un corpus aussi petit, les résultats changent d'une exécution à l'autre. Avec des millions de phrases, les voisins deviennent très pertinents.

Exemple 2 — Visualiser des embeddings Word2Vec avec t-SNE. t-SNE projette des vecteurs de 50 dimensions sur un plan en conservant au mieux les voisinages.

```
import matplotlib.pyplot as plt
from gensim.models import Word2Vec
from sklearn.manifold import TSNE

# 1. Corpus d'exemple
sentences = [
    ["the", "cat", "drinks", "milk"],
    ["the", "dog", "eats", "meat"],
    ["the", "cat", "chases", "the", "mouse"],
    ["the", "dog", "chases", "the", "ball"],
    ["a", "car", "drives", "on", "the", "road"],
    ["a", "truck", "drives", "on", "the", "road"],
    ["the", "king", "rules", "the", "kingdom"],
    ["the", "queen", "rules", "the", "kingdom"],
    ["the", "apple", "is", "a", "fruit"],
    ["the", "banana", "is", "a", "fruit"]
]

# 2. Entraîner Word2Vec
model = Word2Vec(sentences=sentences, vector_size=50, window=3,
                 min_count=1, workers=1, seed=42, epochs=200)
```

```
# 3. Récupérer les mots et leurs vecteurs
words = list(model.wv.index_to_key)
embeddings = model.wv[words]

# 4. Projeter en 2D avec t-SNE
tsne = TSNE(n_components=2, perplexity=min(5, len(words) - 1),
            random_state=42, init="random", learning_rate="auto")
embeddings_2d = tsne.fit_transform(embeddings)

# 5. Afficher
plt.figure(figsize=(12, 8))
plt.scatter(embeddings_2d[:, 0], embeddings_2d[:, 1], color="steelblue")
for i, word in enumerate(words):
    plt.annotate(word, (embeddings_2d[i, 0], embeddings_2d[i, 1]),
fontsize=10)
plt.title("Word Embeddings Visualized with t-SNE")
plt.xlabel("t-SNE Dimension 1")
plt.ylabel("t-SNE Dimension 2")
plt.grid(alpha=0.2)
plt.tight_layout()
plt.show()
```

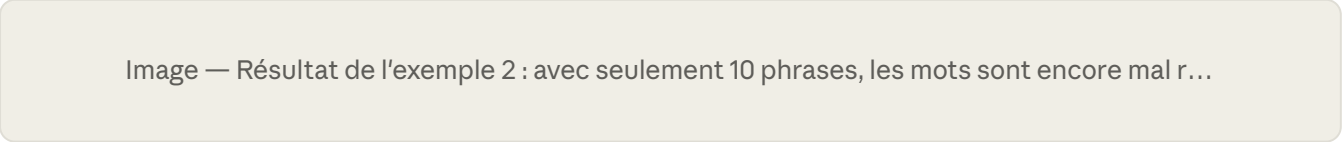


Image — Résultat de l'exemple 2 : avec seulement 10 phrases, les mots sont encore mal r…

Observation : avec 10 phrases, le modèle n'a pas assez d'exemples et les groupes (animaux, véhicules, fruits) restent peu visibles. C'est la motivation de l'exemple 4, avec des vecteurs préentraînés sur des milliards de mots.

14.3 GloVe

GloVe part de la **matrice de cooccurrences** X : X_ij compte combien de fois le mot j apparaît près du mot i. On cherche des vecteurs dont le produit scalaire approche le logarithme de ce compte :

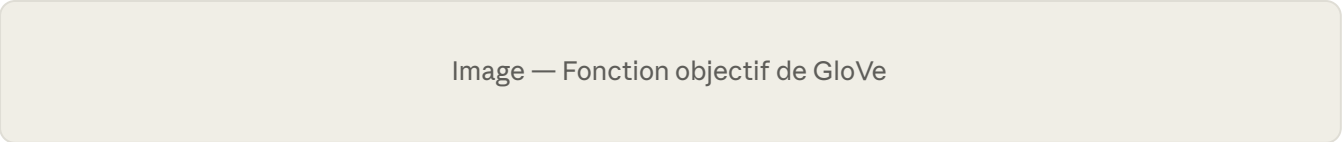


Image — Fonction objectif de GloVe

La fonction f limite le poids des paires très fréquentes, et f(0) = 0 évite le calcul de log 0. J est minimisée par descente de gradient.

Pourquoi des rapports de probabilités ? L'article de GloVe (Pennington, Socher et Manning, 2014) part de cet exemple, mesuré sur un corpus de 6 milliards de mots. P(k | ice) est la probabilité de voir le mot k près de « ice » (glace).

	k = solid	k = gas	k = water	k = fashion
P(k | ice)	$1{,}9 \times 10^{-4}$	$6{,}6 \times 10^{-5}$	$3{,}0 \times 10^{-3}$	$1{,}7 \times 10^{-5}$
P(k | steam)	$2{,}2 \times 10^{-5}$	$7{,}8 \times 10^{-4}$	$2{,}2 \times 10^{-3}$	$1{,}8 \times 10^{-5}$
Rapport ice / steam	8,9	0,085	1,36	0,96

Le rapport est grand pour « solid » (lié à la glace, pas à la vapeur), petit pour « gas » (lié à la vapeur), et proche de 1 pour « water » (lié aux deux) et « fashion » (lié à aucun). Les rapports séparent donc bien le sens des deux mots : GloVe apprend des vecteurs dont les différences reproduisent ces rapports.

Ce que GloVe capture. Une même relation entre mots correspond à un même déplacement dans l'espace des vecteurs. La figure ci-dessous a été calculée avec de vrais vecteurs GloVe, sur le modèle des figures du projet GloVe de Stanford.

Image — Vecteurs GloVe réels : masculin vers féminin, pays vers capitale, adjectif vers…

L'axe vertical suit la direction moyenne de chaque relation : les flèches montent toutes dans le même sens. C'est ce qui rend possibles les analogies. Avec ces vecteurs, on obtient :

Calcul	Mot le plus proche	Similarité
king − man + woman	queen	0,77
paris − france + italy	rome	0,82
dakar − senegal + ghana	accra	0,72

Image — Les plus proches voisins de « frog » et de « student » dans GloVe

Les voisins de « frog » (grenouille) sont d'autres animaux, dont « litoria », un genre de grenouilles. Ceux de « student » relèvent tous du monde scolaire. On obtient ces résultats avec `model.most_similar("frog", topn=8)`, comme dans l'exemple 3.

Exemple 3 — Utiliser des vecteurs GloVe préentraînés et most_similar.

```
import gensim.downloader as api

model = api.load("glove-wiki-gigaword-100")   # 400 000 mots, vecteurs de 100
nombres

print(model.most_similar("computer", topn=5))
print(model.similarity("cat", "dog"))         # proche de 1
print(model.similarity("cat", "car"))         # plus faible

# Analogie : roi - homme + femme = ?
print(model.most_similar(positive=["king", "woman"], negative=["man"],
topn=1))
# [('queen', ...)]

print(model.doesnt_match(["apple", "banana", "orange", "car"]))   # 'car'
```

Exemple 4 — Visualiser des embeddings GloVe préentraînés avec t-SNE.

```
# À installer d'abord :
# pip install gensim scikit-learn matplotlib

import gensim.downloader as api
import matplotlib.pyplot as plt
from sklearn.manifold import TSNE

# 1. Charger les embeddings préentraînés
model = api.load("glove-wiki-gigaword-100")

# 2. Choisir des mots par familles
words = [
    "king", "queen", "prince", "princess",
    "man", "woman", "boy", "girl",
    "father", "mother", "brother", "sister",
    "car", "truck", "bus", "train",
    "dog", "cat", "wolf", "tiger",
    "apple", "orange", "banana", "fruit"
]
words = [w for w in words if w in model]    # garder les mots connus du
modèle

# 3. Récupérer les vecteurs
X = model[words]

# 4. Réduire à 2 dimensions avec t-SNE
tsne = TSNE(n_components=2, perplexity=5, init="pca",
            learning_rate="auto", random_state=42)
```

```
X_2d = tsne.fit_transform(X)

# 5. Afficher
plt.figure(figsize=(12, 9))
plt.scatter(X_2d[:, 0], X_2d[:, 1], s=50)
for i, word in enumerate(words):
    plt.annotate(word, (X_2d[i, 0], X_2d[i, 1]), xytext=(4, 4),
                 textcoords="offset points", fontsize=10)
plt.title("Pretrained GloVe Word Embeddings with t-SNE")
plt.xlabel("t-SNE Dimension 1")
plt.ylabel("t-SNE Dimension 2")
plt.grid(alpha=0.2)
plt.tight_layout()
plt.show()
```

Résultat attendu : quatre groupes nets apparaissent (famille et royauté, véhicules, animaux, fruits). Le premier chargement télécharge environ 130 Mo.

14.4 Comment un embedding apprend ses valeurs (PyTorch)

Une couche `nn.Embedding` est une table de vecteurs dont les valeurs sont ajustées par descente de gradient. On lui apprend une association et on regarde les vecteurs bouger.

Exemple 5 — Rapprocher « chat » et « poisson ».

```
import torch
import torch.nn as nn

vocab = {"chat": 0, "chien": 1, "poisson": 2}
embedding = nn.Embedding(3, 2)            # 3 mots, vecteurs de 2 nombres
optimizer = torch.optim.SGD(embedding.parameters(), lr=0.1)

chat = torch.tensor([vocab["chat"]])
poisson = torch.tensor([vocab["poisson"]])

for epoch in range(100):
    optimizer.zero_grad()
    v_chat = embedding(chat)
    v_poisson = embedding(poisson)
    loss = ((v_chat - v_poisson) ** 2).mean()   # on veut des vecteurs
proches
    loss.backward()
    optimizer.step()

print("Embedding de chat :", embedding.weight[0].detach())
print("Embedding de poisson :", embedding.weight[2].detach())
```

Le problème : l'effondrement des représentations. Si l'on ne fait que rapprocher, « chat » et « poisson » deviennent presque identiques. Si l'on rapproche tous les mots de la même manière, tous les embeddings finissent au même point : c'est l'**effondrement des représentations** (*representation collapse*). La perte vaut zéro, mais les vecteurs ne permettent plus de distinguer les mots.

La solution : ajouter des **exemples négatifs**. On rapproche les mots liés et on éloigne les mots sans rapport.

Image — Avant l'entraînement, après effondrement, et avec paires positive et négative

Exemple 6 — Paire positive et paire négative.

```
import torch
import torch.nn as nn
import torch.nn.functional as F

torch.manual_seed(42)
vocab = {"chat": 0, "poisson": 1, "voiture": 2}
embedding = nn.Embedding(3, 2)
optimizer = torch.optim.SGD(embedding.parameters(), lr=0.1)

chat = torch.tensor([vocab["chat"]])
poisson = torch.tensor([vocab["poisson"]])
voiture = torch.tensor([vocab["voiture"]])

for epoch in range(200):
    optimizer.zero_grad()
    v_chat = embedding(chat)
    v_poisson = embedding(poisson)
    v_voiture = embedding(voiture)

    # Rapprocher la paire positive (chat, poisson)
    positive_loss = (1 - F.cosine_similarity(v_chat, v_poisson)).mean()

    # Éloigner la paire négative (chat, voiture)
    negative_loss = F.relu(F.cosine_similarity(v_chat, v_voiture) -
0.2).mean()

    loss = positive_loss + negative_loss
    loss.backward()
    optimizer.step()
```

```
print("chat :", embedding.weight[0].detach())
print("poisson :", embedding.weight[1].detach())
print("voiture :", embedding.weight[2].detach())
```

C'est le même principe que l'**échantillonnage négatif** de Word2Vec : le modèle apprend autant de ce qui est différent que de ce qui se ressemble.

14.5 Embeddings statiques et embeddings contextuels

Word2Vec et GloVe sont **statiques** : chaque mot possède un seul vecteur, quel que soit son contexte. Or beaucoup de mots ont plusieurs sens. Dans « Il a posé un avocat sur la table », s'agit-il du fruit ou du juriste ? Word2Vec donne le même vecteur aux deux sens : une sorte de moyenne, coincée entre les deux.

Les modèles **contextuels** (BERT, DistilBERT, CamemBERT pour le français) calculent le vecteur d'un mot à partir de toute la phrase, grâce à l'attention (chapitre 17). Le même mot reçoit donc un vecteur différent dans chaque phrase.

Image — À gauche, Word2Vec place « avocat » entre les fruits et la justice ; à droite, …

	Statique (Word2Vec, GloVe)	Contextuel (BERT, DistilBERT)
Vecteurs par mot	Un seul	Un par phrase où le mot apparaît
« avocat » fruit et juriste	Même vecteur	Deux vecteurs différents
Calcul	Lecture dans une table	Passage dans un réseau Transformer
Coût	Très faible	Plus élevé (DistilBERT : 66 millions de paramètres)
Usage typique	Similarité de mots, petits projets	Classification, questions-réponses, recherche sémantique

DistilBERT est une version réduite de BERT obtenue par *distillation* : un petit modèle (l'élève) apprend à imiter un grand (le maître). Il est 40 % plus petit et 60 % plus rapide que BERT, pour environ 97 % de ses performances.

Exemple 7 — Les vecteurs contextuels de « avocat ». Le code calcule le vecteur du mot dans trois phrases, puis compare ces vecteurs avec la similarité cosinus.

```
# pip install transformers torch
import torch
from transformers import AutoTokenizer, AutoModel

nom = "distilbert-base-multilingual-cased"
tokenizer = AutoTokenizer.from_pretrained(nom)
modele = AutoModel.from_pretrained(nom)

def vecteur_du_mot(phrase, mot):
    debut = phrase.index(mot)
    fin = debut + len(mot)
    entrees = tokenizer(phrase, return_tensors="pt",
return_offsets_mapping=True)
    positions_mot = entrees.pop("offset_mapping")[0].tolist()
    with torch.no_grad():
        sorties = modele(**entrees).last_hidden_state[0]        # un vecteur
par token
    positions = [i for i, (d, f) in enumerate(positions_mot) if d >= debut
and f <= fin and f > d]
    return sorties[positions].mean(dim=0)                      # moyenne des
morceaux du mot

fruit1 = vecteur_du_mot("Il a mangé un avocat bien mûr.", "avocat")
fruit2 = vecteur_du_mot("J'achète un avocat et des mangues au marché.",
"avocat")
juriste = vecteur_du_mot("Le juge écoute un avocat au tribunal.", "avocat")

cos = torch.nn.functional.cosine_similarity
print("fruit / fruit   :", round(cos(fruit1, fruit2, dim=0).item(), 2))
print("fruit / juriste :", round(cos(fruit1, juriste, dim=0).item(), 2))
```

Résultat attendu : la similarité fruit / fruit est plus élevée que fruit / juriste. Les valeurs exactes dépendent du modèle ; avec Word2Vec, les deux similarités vaudraient exactement 1, puisque le vecteur est le même. Le premier lancement télécharge le modèle (environ 500 Mo).

Exemple 8 — Des embeddings de phrases. La bibliothèque Sentence Transformers produit directement un vecteur pour une phrase entière. Deux phrases de même sens ont des vecteurs proches, même sans mot commun.

```
# pip install sentence-transformers
from sentence_transformers import SentenceTransformer, util

modele = SentenceTransformer("paraphrase-multilingual-MiniLM-L12-v2")
phrases = ["Comment obtenir une bourse ?",
           "Je cherche une aide financière pour mes études.",
           "Les examens commencent en janvier."]
vecteurs = modele.encode(phrases)
print(util.cos_sim(vecteurs, vecteurs))
```

Résultat attendu : une matrice 3 × 3 où la case (phrase 1, phrase 2) est nettement plus grande que les cases avec la phrase 3. « bourse » et « aide financière » n'ont aucun mot commun : TF-IDF les jugerait sans rapport. Cet exemple sert de base au mini-projet de moteur de recherche (chapitre 18).

Exercices

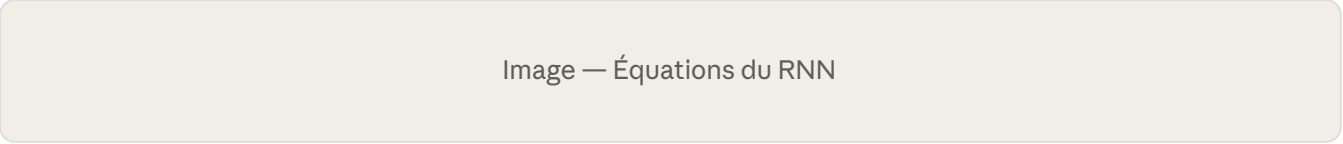
1. Avec les vecteurs GloVe, compléter les analogies : Paris − France + Italy = ? ; walking − walk + swim = ?
2. Dans l'exemple 5, ajouter les pertes chat–voiture et poisson–voiture. Que deviennent les trois vecteurs ?
3. Dans l'exemple 6, remplacer 0.2 par −0.5. L'angle entre « chat » et « voiture » augmente-t-il ?

Chapitre 15 — Réseaux récurrents : RNN, LSTM et GRU

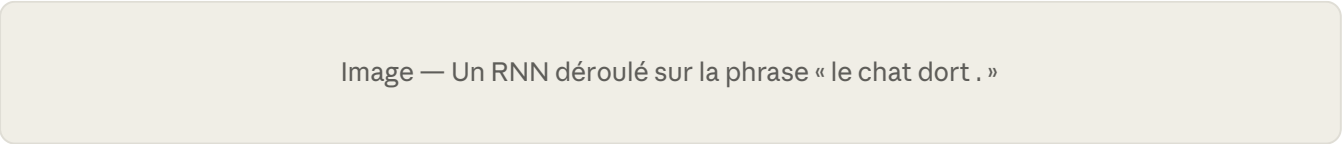
Un texte est une séquence : le sens d'un mot dépend des mots précédents. Un réseau récurrent lit les mots un par un et garde une mémoire de ce qu'il a déjà lu.

15.1 Le RNN simple

À chaque instant t, le réseau combine le mot courant x_t et sa mémoire précédente h_(t−1) pour produire une nouvelle mémoire h_t.



Les mêmes poids W_x, W_h et W_y sont utilisés à chaque instant.



15.2 Rétropropagation dans le temps et disparition du gradient

Pour entraîner un RNN, on le « déroule » sur toute la phrase puis on rétropropage l'erreur de la fin vers le début (*Backpropagation Through Time*). À chaque pas en arrière, le gradient est multiplié par un facteur lié à W_h :

- facteur inférieur à 1 → le gradient **disparaît** (*vanishing gradient*) : le réseau oublie le début des phrases longues ;
- facteur supérieur à 1 → le gradient **explose**.

Image — Évolution de la taille du gradient selon le facteur multiplié à chaque pas

Exemple : avec un facteur 0,5, après 10 pas il ne reste que $0{,}5^{10} \approx 0{,}001$ du gradient.

15.3 LSTM

Le LSTM (*Long Short-Term Memory*) ajoute une **mémoire à long terme** c_t et trois **portes**, des valeurs entre 0 et 1 qui décident de ce qui passe.

Porte	Rôle
Oubli f_t	Que faut-il effacer de la mémoire ?
Entrée i_t	Que faut-il ajouter à la mémoire ?
Sortie o_t	Quelle partie de la mémoire afficher ?

Image — La cellule LSTM : ligne de mémoire c en haut, porte d'oubli, porte d'entrée, ca…

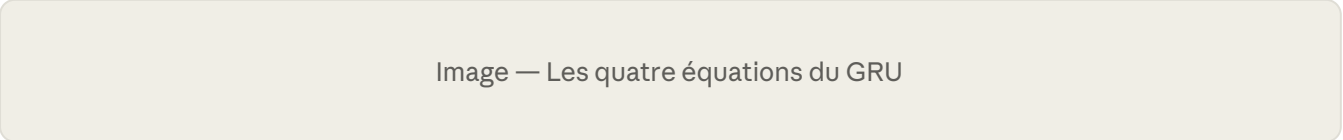
On lit le schéma de gauche à droite. La ligne bleue est la mémoire à long terme c : la porte d'oubli f(t) efface une partie de c(t−1) (×), puis la porte d'entrée i(t) ajoute une partie du candidat c̃(t) (+). La porte de sortie o(t) choisit enfin quelle partie de la mémoire devient h(t), la sortie de la cellule. Les équations ci-dessous décrivent exactement ces opérations.

Image — Les six équations du LSTM

σ est la fonction sigmoïde, ⊙ le produit case par case. La mémoire c_t est mise à jour par une **addition** : le gradient la traverse sans être multiplié à chaque pas, ce qui limite sa disparition.

15.4 GRU

Le GRU (*Gated Recurrent Unit*) simplifie le LSTM : deux portes et une seule mémoire h_t.



z_t (mise à jour) dose l'ancien et le nouveau ; r_t (réinitialisation) décide quelle part du passé utiliser.

	RNN	LSTM	GRU
Portes	0	3	2
Mémoire	h	h et c	h
Phrases longues	Mal	Bien	Bien
Paramètres	Peu	Beaucoup	Moyen

15.5 Exemple en PyTorch

```
import torch
import torch.nn as nn

# Une phrase de 4 mots, chaque mot = vecteur de 10 nombres
phrase = torch.randn(1, 4, 10)        # (lots, longueur, taille d'un mot)

rnn = nn.RNN(input_size=10, hidden_size=20, batch_first=True)
lstm = nn.LSTM(input_size=10, hidden_size=20, batch_first=True)
gru = nn.GRU(input_size=10, hidden_size=20, batch_first=True)

sorties, h = rnn(phrase)
print(sorties.shape)    # (1, 4, 20) : une sortie par mot
print(h.shape)          # (1, 1, 20) : la mémoire finale

sorties, (h, c) = lstm(phrase)    # le LSTM renvoie aussi c
sorties, h = gru(phrase)
```

Exemple : classer le sentiment d'une phrase.

```
class Sentiment(nn.Module):
    def __init__(self, taille_vocab):
        super().__init__()
        self.embedding = nn.Embedding(taille_vocab, 32)
        self.lstm = nn.LSTM(32, 64, batch_first=True)
        self.sortie = nn.Linear(64, 2)      # positif ou négatif

    def forward(self, mots):
        x = self.embedding(mots)
        _, (h, c) = self.lstm(x)
        return self.sortie(h[-1])          # on décide à partir de la mémoire
finale

modele = Sentiment(taille_vocab=1000)
phrase = torch.tensor([[12, 45, 7, 300]])   # indices des mots
print(modele(phrase))
```

15.6 Les limites des RNN et ce que le LSTM corrige

Un RNN simple a deux limites principales.

1. Les dépendances à longue distance. Dans « Les **étudiants** qui ont suivi le cours de vision par ordinateur pendant tout le semestre **sont** très motivés », accorder « sont » au pluriel demande de se souvenir de « étudiants », treize mots plus tôt. Dans un RNN simple, cette information traverse treize mises à jour successives de la mémoire. Pendant l'entraînement, le gradient disparaît sur ce trajet : le réseau n'apprend pas le lien.

2. La nature séquentielle. Pour calculer h_t, il faut d'abord h_(t−1). Les mots sont donc traités l'un après l'autre, sans calcul en parallèle. Le GPU reste sous-utilisé et l'entraînement sur un grand corpus est lent.

Problème	RNN simple	LSTM
Information lointaine	Effacée après quelques mots	Gardée dans la cellule c_t tant que la porte d'oubli reste proche de 1
Gradient	Multiplié à chaque pas, il disparaît	Mise à jour par addition, il circule mieux
Lecture mot par mot	Oui	Toujours oui : non corrigé
Sens d'un mot ambigu	Dépend de l'embedding d'entrée	Idem

Le LSTM corrige donc surtout la mémoire, pas la lenteur. Deux problèmes restent :

- la lecture reste séquentielle ;
- les embeddings donnés en entrée (Word2Vec, GloVe) sont **non contextualisés** : « avocat » arrive avec le même vecteur dans toutes les phrases, et le réseau doit reconstruire seul le contexte.

Les Transformers (chapitre 17) répondent aux deux : l'attention relie deux mots quelconques en une seule étape, tous les mots sont traités en parallèle, et chaque mot reçoit une représentation qui dépend de toute la phrase.

Expérience (TP3, chapitre 18) : entraîner un RNN simple et un LSTM sur la même tâche, puis comparer leur précision sur des phrases courtes (moins de 10 mots) et longues (plus de 30 mots). L'écart entre les deux modèles doit augmenter avec la longueur.

Exercices

1. Calculer à la main h_1 pour un RNN à une dimension avec $W_x = 1$, $W_h = 0{,}5$, $b = 0$, $h_0 = 0$, $x_1 = 1$. Réponse : $\tanh(1) \approx 0{,}76$.
2. Calculer ensuite h_2 avec $x_2 = 0$. Réponse : $\tanh(0{,}38) \approx 0{,}36$.
3. Expliquer pourquoi une porte d'oubli f_t proche de 1 permet de garder une information longtemps.

Chapitre 16 — Traduction automatique neuronale

Un modèle Seq2Seq traduit en deux temps : un **encodeur** lit la phrase source, un **décodeur** écrit la traduction mot par mot.

16.1 Qu'est-ce que la traduction automatique ?

La **traduction automatique** consiste à faire produire par un programme, à partir d'une phrase dans une langue source, une phrase de même sens dans une langue cible.

```
Français : Je suis étudiant en informatique.
Anglais  : I am a computer science student.
```

Ce n'est pas une traduction mot à mot. Un dictionnaire appliqué mot par mot donne « I am student in informatics. ». L'anglais exige l'article « a », place le complément avant le nom (« computer science student ») et emploie un autre terme pour « informatique ». Traduire, c'est saisir le sens de la phrase entière, puis le reformuler selon les règles de la langue cible.

Approche	Résultat
Mot à mot avec un dictionnaire	I am student in informatics.

Approche	Résultat
Traduction correcte	I am a computer science student.

Les ambiguïtés. « Il a posé un avocat sur la table. » : fruit (*avocado*) ou juriste (*lawyer*) ? La phrase seule ne permet pas de trancher. Si la suite est « … puis il l'a coupé en deux », c'est le fruit. Un bon système doit donc tenir compte du contexte, parfois au-delà de la phrase.

Les expressions idiomatiques. « Il pleut des cordes » se traduit par *It's raining cats and dogs*, et non par *It is raining ropes*. « Coûter les yeux de la tête » devient *to cost an arm and a leg*. Le sens d'une expression n'est pas la somme du sens de ses mots.

Les capacités mobilisées. Traduire fait appel à presque tout ce que l'on a vu dans cette partie.

Capacité	Rôle dans la traduction	Chapitre
Tokenisation	Découper la phrase source en unités	12 et 17
Analyse syntaxique	Savoir qui fait quoi, pour réordonner les mots	12
Représentation sémantique	Représenter le sens par des vecteurs	14
Prise en compte du contexte	Lever les ambiguïtés comme « avocat »	14, 16 et 17
Génération	Écrire une phrase correcte dans la langue cible, mot par mot	16

Trois générations de modèles neuronaux. Avant 2014, on utilisait des règles écrites par des linguistes, puis des modèles statistiques fondés sur des comptages. Les modèles neuronaux se sont ensuite succédé :

1. l'encodeur-décodeur RNN ou LSTM (2014) : il lit la phrase, la résume dans un vecteur, puis écrit la traduction
2. l'ajout de l'attention (2015) : à chaque mot, le décodeur regarde les parties utiles de la phrase source
3. le Transformer (2017) : plus de récurrence, uniquement de l'attention ; c'est la base des traducteurs actuels

16.2 L'architecture Seq2Seq

```
ENCODEUR (lit le français)                DÉCODEUR (écrit l'anglais)

 le → chat → noir → dort → [ vecteur c ] → the → black → cat → sleeps → <fin>
                              (contexte)
```

1. L'encodeur (un GRU ou un LSTM) lit les mots source un par un.
2. Sa mémoire finale, le **vecteur de contexte** c, résume toute la phrase.
3. Le décodeur part de c et produit un mot à la fois ; chaque mot produit devient l'entrée de l'étape suivante.

16.3 Le goulot d'étranglement

Toute la phrase source doit tenir dans un seul vecteur c de taille fixe. Pour une phrase de 5 mots cela marche ; pour une phrase de 50 mots, le début est oublié et la traduction se dégrade. C'est le **goulot d'étranglement**.

Image — En haut, le Seq2Seq sans attention fait passer toute la phrase par un seul vect…

En haut, toute l'information passe par le cercle rouge c : c'est le goulot. En bas, chaque étape du décodeur va chercher l'information directement dans les états de l'encodeur (flèches vertes) : c'est le mécanisme d'attention de la section 16.4. Par exemple, pour écrire « student », le décodeur regarde surtout « étudiant ».

16.4 Le mécanisme d'attention

Avec l'attention, le décodeur n'utilise plus un seul vecteur c. À chaque mot qu'il écrit, il regarde **tous** les états de l'encodeur et choisit ceux qui sont utiles.

Pour écrire le mot n° t :

1. Calculer un score entre l'état du décodeur s_t et chaque état de l'encodeur h_i : $score_i = s_t \cdot h_i$.
2. Transformer les scores en poids α qui somment à 1 (softmax).
3. Faire la moyenne pondérée des états de l'encodeur pour obtenir le contexte c_t :

Image — Formules de l'attention : poids α puis vecteur de contexte c

Image — Matrice d'attention : pour écrire « black », le modèle regarde surtout « noir »

La matrice d'attention montre que le modèle a appris à réordonner les mots : en anglais, l'adjectif « black » vient avant le nom « cat ».

16.5 Calcul d'attention à la main

États de l'encodeur (en 2 dimensions) : h_1 = (1, 0), h_2 = (0, 1), h_3 = (1, 1). État du décodeur : s = (2, 0).

Mot source	Score s · h_i	exp(score)	Poids α
h_1	2	7,39	7,39 / 15,78 = 0,47
h_2	0	1,00	1,00 / 15,78 = 0,06
h_3	2	7,39	7,39 / 15,78 = 0,47

Contexte : c = 0,47 × (1, 0) + 0,06 × (0, 1) + 0,47 × (1, 1) = (0,94 ; 0,53).

```
import torch

h = torch.tensor([[1., 0.], [0., 1.], [1., 1.]])   # états de l'encodeur
s = torch.tensor([2., 0.])                           # état du décodeur

scores = h @ s                       # produits scalaires : [2, 0, 2]
poids = torch.softmax(scores, dim=0) # [0.47, 0.06, 0.47]
contexte = poids @ h                 # [0.94, 0.53]
print(poids, contexte)
```

16.6 Un Seq2Seq minimal en PyTorch

```
import torch
import torch.nn as nn

class Encodeur(nn.Module):
    def __init__(self, vocab_source, taille):
        super().__init__()
        self.embedding = nn.Embedding(vocab_source, taille)
        self.gru = nn.GRU(taille, taille, batch_first=True)

    def forward(self, phrase):
        sorties, h = self.gru(self.embedding(phrase))
        return h                        # le vecteur de contexte

class Decodeur(nn.Module):
    def __init__(self, vocab_cible, taille):
        super().__init__()
```

```
        self.embedding = nn.Embedding(vocab_cible, taille)
        self.gru = nn.GRU(taille, taille, batch_first=True)
        self.sortie = nn.Linear(taille, vocab_cible)

    def forward(self, mot, h):
        sortie, h = self.gru(self.embedding(mot), h)
        return self.sortie(sortie), h  # scores pour chaque mot du
vocabulaire

# Traduire (après entraînement) : on génère mot par mot
encodeur, decodeur = Encodeur(100, 32), Decodeur(100, 32)
phrase = torch.tensor([[5, 12, 8]])         # indices des mots source
h = encodeur(phrase)
mot = torch.tensor([[1]])                    # 1 = jeton <debut>

for _ in range(5):
    scores, h = decodeur(mot, h)
    mot = scores.argmax(dim=-1)              # le mot le plus probable
    print(mot.item())
```

Pour l'entraînement, on compare à chaque pas le mot prédit au vrai mot avec `nn.CrossEntropyLoss`. Le TP consiste à entraîner ce modèle sur quelques milliers de paires français–anglais.

16.7 Organiser l'entraînement d'un traducteur

Entraîner un petit traducteur français → anglais est à la portée d'un ordinateur portable ou de Google Colab. Voici l'organisation retenue pour le TP.

Élément	Choix pour le TP	Raison
Corpus	Corpus parallèle FR–EN de quelques milliers à quelques dizaines de milliers de paires de phrases courtes (par exemple le fichier fra-eng du projet Tatoeba)	Assez pour apprendre des structures simples, assez petit pour un entraînement rapide
Prétraitement	Minuscules, tokenisation, un vocabulaire par langue avec `<pad>`, `<unk>`, `<debut>`, `<fin>` (section 12.6)	Transformer chaque phrase en suite d'indices
Bibliothèque	PyTorch	Le code reste proche des équations
Embeddings	Dimension 128	Suffisant pour un petit vocabulaire
Encodeur	LSTM, 1 ou 2 couches	Lit la phrase française

Élément	Choix pour le TP	Raison
Décodeur	LSTM, 1 ou 2 couches, puis une couche linéaire vers le vocabulaire anglais	Écrit la phrase anglaise mot par mot
Entraînement	*Teacher forcing*, entropie croisée, optimiseur Adam	Apprentissage rapide et stable
Évaluation	Phrases de test jamais vues, BLEU et chrF (section 16.8), lecture des erreurs	Mesurer la généralisation, pas la mémoire

Le *teacher forcing*. Pendant l'entraînement, à chaque étape, le décodeur reçoit en entrée le **vrai** mot précédent de la traduction de référence, et non le mot qu'il vient de prédire. Sans cela, une erreur au début fausserait toute la suite de la phrase et l'apprentissage serait très lent.

Étape	Entrée du décodeur	Mot attendu en sortie
1	`<debut>`	i
2	i	am
3	am	a
4	a	student
5	student	`<fin>`

L'entrée est donc la phrase cible décalée d'un cran : `<debut>` suivi de la cible sans son dernier mot. Au moment de traduire, il n'y a plus de référence : le décodeur utilise ses propres prédictions, comme dans la fonction `traduire` ci-dessous.

Exemple complet sur six paires de phrases. Le principe est identique avec des milliers de paires ; seule la boucle sur les données change.

```
import torch
import torch.nn as nn
torch.manual_seed(0)

paires = [("je suis étudiant", "i am a student"),
          ("je suis professeur", "i am a teacher"),
          ("tu es étudiant", "you are a student"),
          ("il est professeur", "he is a teacher"),
          ("elle est étudiante", "she is a student"),
          ("nous sommes étudiants", "we are students")]
```

```
def construire_vocab(phrases):
    vocab = {"<pad>": 0, "<debut>": 1, "<fin>": 2}
    for p in phrases:
        for mot in p.split():
            vocab.setdefault(mot, len(vocab))
    return vocab

vocab_fr = construire_vocab(fr for fr, en in paires)
vocab_en = construire_vocab(en for fr, en in paires)
inverse_en = {i: m for m, i in vocab_en.items()}

def vers_indices(phrase, vocab):
    return [vocab[m] for m in phrase.split()] + [vocab["<fin>"]]

class Seq2Seq(nn.Module):
    def __init__(self, n_fr, n_en, dim=128):
        super().__init__()
        self.emb_fr = nn.Embedding(n_fr, dim)
        self.emb_en = nn.Embedding(n_en, dim)
        self.encodeur = nn.LSTM(dim, dim, batch_first=True)
        self.decodeur = nn.LSTM(dim, dim, batch_first=True)
        self.sortie = nn.Linear(dim, n_en)

    def forward(self, source, cible_entree):
        _, etat = self.encodeur(self.emb_fr(source))            # résumé de
la phrase source
        sorties, _ = self.decodeur(self.emb_en(cible_entree), etat)
        return self.sortie(sorties)                              # un score
par mot du vocabulaire

modele = Seq2Seq(len(vocab_fr), len(vocab_en))
optimiseur = torch.optim.Adam(modele.parameters(), lr=0.01)
perte_fn = nn.CrossEntropyLoss()

for epoque in range(100):
    total = 0
    for fr, en in paires:
        source = torch.tensor([vers_indices(fr, vocab_fr)])
        cible = vers_indices(en, vocab_en)                       # i am a
student <fin>
        entree = torch.tensor([[vocab_en["<debut>"]] + cible[:-1]])   #
<debut> i am a student
        attendu = torch.tensor([cible])
        scores = modele(source, entree)                          # teacher
forcing
```

```
        perte = perte_fn(scores.view(-1, len(vocab_en)), attendu.view(-1))
        optimiseur.zero_grad()
        perte.backward()
        optimiseur.step()
        total += perte.item()
    if epoque % 25 == 0:
        print("époque", epoque, "perte", round(total / len(paires), 3))

def traduire(phrase, longueur_max=10):
    with torch.no_grad():
        source = torch.tensor([vers_indices(phrase, vocab_fr)])
        _, etat = modele.encodeur(modele.emb_fr(source))
        mot = torch.tensor([[vocab_en["<debut>"]]])
        resultat = []
        for _ in range(longueur_max):
            sortie, etat = modele.decodeur(modele.emb_en(mot), etat)
            mot = modele.sortie(sortie).argmax(dim=-1)            # le mot le
plus probable
            if mot.item() == vocab_en["<fin>"]:
                break
            resultat.append(inverse_en[mot.item()])
        return " ".join(resultat)

print(traduire("je suis étudiant"), "|", traduire("il est professeur"), "|",
traduire("tu es professeur"))
```

Résultat obtenu :

```
époque 0 perte 2.569
époque 25 perte 0.002
époque 50 perte 0.001
époque 75 perte 0.0
i am a student | he is a teacher | you are a student
```

Interprétation. Les deux premières phrases font partie des données d'entraînement : elles sont parfaitement traduites. La troisième, « tu es professeur », n'a jamais été vue : le modèle répond « you are a student ». Il a appris que « tu es » donne « you are », mais il recopie la seule phrase d'entraînement qui commençait ainsi. Une perte nulle ne prouve donc rien : avec six paires, le modèle **mémorise** au lieu de **généraliser**. C'est pourquoi le TP utilise des milliers de paires et évalue toujours sur des phrases absentes de l'entraînement.

Dans le TP, les phrases sont regroupées par lots ; les phrases courtes sont complétées avec `<pad>`, que la perte doit ignorer : `nn.CrossEntropyLoss(ignore_index=0)`.

16.8 Évaluer une traduction : BLEU, chrF, METEOR, COMET

Le score BLEU compare la traduction produite à une traduction de référence en comptant les n-grammes communs. Il va de 0 à 1 (souvent exprimé sur 100) ; plus il est élevé, plus la traduction ressemble à la référence.

```
from nltk.translate.bleu_score import sentence_bleu

reference = [["the", "black", "cat", "sleeps"]]
traduction = ["the", "black", "cat", "is", "sleeping"]

print(sentence_bleu(reference, traduction, weights=(0.5, 0.5)))   #
unigrammes et bigrammes
```

Limite : BLEU ne comprend pas les synonymes. « the cat sleeps » et « the feline sleeps » ont un score faible alors que le sens est le même.

D'autres métriques complètent BLEU. On en rapporte toujours plusieurs.

Métrique	Principe	Avantage	Limite
BLEU	Compte les n-grammes de mots communs avec la référence (0 à 100)	Simple, la plus utilisée, permet de comparer avec les articles	Ignore les synonymes, pénalise les reformulations correctes
chrF	Compte les n-grammes de caractères et calcule un F-score (0 à 100)	Tolère les variations de forme (*student* / *students*), adaptée aux langues à morphologie riche	Ne juge pas le sens
METEOR	Aligne les mots en acceptant les racines et les synonymes, favorise le rappel (0 à 1)	Reconnaît *sleeps* / *sleeping*	Repose sur des ressources linguistiques, surtout pour l'anglais
COMET	Un réseau de neurones, entraîné sur des notes données par des humains, lit la source, la traduction et la référence	La plus proche du jugement humain	Modèle lourd, score moins facile à interpréter

BLEU et chrF avec sacrebleu, la bibliothèque de référence pour comparer des résultats :

```
import sacrebleu    # pip install sacrebleu

references = [["I am a computer science student.", "The black cat sleeps on
the bed."]]
traductions = ["I am a student in computer science.", "The black cat is
sleeping on the bed."]

bleu = sacrebleu.corpus_bleu(traductions, references)
chrf = sacrebleu.corpus_chrf(traductions, references)
print("BLEU :", round(bleu.score, 1))
print("chrF :", round(chrf.score, 1))
for t, r in zip(traductions, references[0]):
    print(round(sacrebleu.sentence_bleu(t, [r]).score, 1),
round(sacrebleu.sentence_chrf(t, [r]).score, 1))
```

Résultat obtenu :

```
BLEU : 33.3
chrF : 73.1
28.1 74.6
43.2 71.5
```

METEOR avec NLTK :

```
import nltk
from nltk.translate.meteor_score import meteor_score
nltk.download("wordnet")

reference = "I am a computer science student .".split()
traduction = "I am a student in computer science .".split()
print(round(meteor_score([reference], traduction), 3))
```

Résultat obtenu : 0.894 (et 0.98 pour la phrase du chat).

Interprétation. Les deux traductions sont correctes, pourtant BLEU ne donne que 33,3 sur 100 : la reformulation « a student in computer science » casse de nombreux n-grammes. chrF (73,1) et METEOR (0,89) sont plus indulgents, car ils voient les caractères et les racines communs. Une seule métrique ne suffit donc pas : on compare plusieurs scores et on **lit** un échantillon de traductions (analyse qualitative).

COMET s'utilise ainsi (code non exécuté ici : il télécharge un modèle d'environ 2 Go) :

```
# pip install unbabel-comet
from comet import download_model, load_from_checkpoint

modele = load_from_checkpoint(download_model("Unbabel/wmt22-comet-da"))
donnees = [{"src": "Je suis étudiant en informatique.",
            "mt": "I am a student in computer science.",
            "ref": "I am a computer science student."}]
print(modele.predict(donnees, batch_size=8, gpus=0).scores)
```

Le score COMET est proche de 1 pour une bonne traduction. Comme il juge le sens, cette reformulation correcte doit obtenir un score élevé, contrairement à BLEU.

Exercices

1. Refaire le calcul de 16.5 avec s = (0, 2). Quel état de l'encodeur reçoit le plus d'attention ?
2. Expliquer pourquoi l'attention résout le problème du goulot d'étranglement.

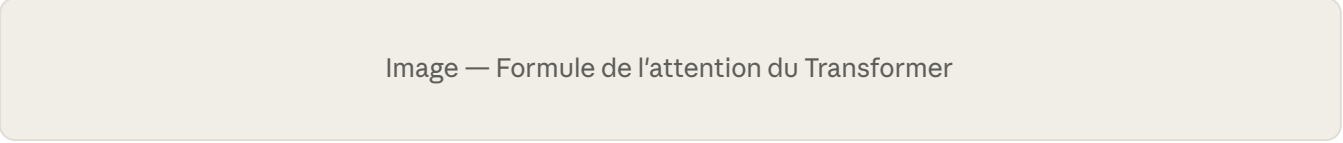
Chapitre 17 — Transformers et modèles préentraînés

Le Transformer supprime la récurrence : il traite tous les mots en même temps et les relie entre eux uniquement par l'attention. C'est l'architecture de BERT, GPT et des traducteurs modernes.

17.1 L'auto-attention

Chaque mot regarde tous les autres mots de la même phrase. Dans « Le chat ne boit pas le lait car **il** est trop chaud », l'auto-attention permet de relier « il » à « lait ».

Chaque mot produit trois vecteurs : une **requête** Q (ce que je cherche), une **clé** K (ce que je contiens) et une **valeur** V (ce que je transmets).



La division par √d_k (d_k = taille des vecteurs) évite des scores trop grands.

```
import torch

# 3 mots, vecteurs de taille 4
Q = torch.randn(3, 4)
K = torch.randn(3, 4)
V = torch.randn(3, 4)

scores = Q @ K.T / 4 ** 0.5          # (3, 3) : chaque mot face à chaque mot
poids = torch.softmax(scores, dim=1)  # chaque ligne somme à 1
sortie = poids @ V
print(poids)
```

17.2 RNN et Transformer

	RNN / LSTM	Transformer
Lecture	Mot par mot	Tous les mots en même temps
Lien entre mots éloignés	Difficile	Direct, par l'attention
Entraînement sur GPU	Lent (séquentiel)	Rapide (parallèle)
Ordre des mots	Naturel	Ajouté par un encodage de position

17.3 L'architecture complète du Transformer

Le Transformer de l'article « Attention Is All You Need » (Vaswani et al., 2017) a été conçu pour la traduction. Il garde l'idée encodeur-décodeur du chapitre 16, mais remplace la récurrence par l'attention.

Image — Architecture d'un Transformer encodeur-décodeur : embeddings et position, couch…

On lit le schéma de bas en haut :

1. **Embeddings et encodage de position.** Chaque mot devient un vecteur (chapitre 14). L'attention ne connaît pas l'ordre des mots : on ajoute donc à chaque vecteur un vecteur qui indique sa position dans la phrase.
2. **Encodeur** : N couches identiques (6 dans l'article original). Chaque couche contient une auto-attention multi-têtes, où chaque mot regarde tous les mots, puis un petit réseau dense (*feed-forward*) appliqué à chaque mot. Après chacun des deux, on ajoute l'entrée à la sortie (connexion résiduelle) puis on normalise.

3. **Décodeur** : N couches avec trois blocs. L'auto-attention masquée regarde les mots déjà écrits. L'attention croisée regarde la sortie de l'encodeur : c'est l'attention du chapitre 16. Le réseau dense vient en dernier.
4. **Sortie** : une couche linéaire puis un softmax donnent la probabilité de chaque mot du vocabulaire. On choisit le mot suivant, puis on recommence.

Image — L'entrée de BERT : embedding du token, plus embedding du segment, plus embeddin…

Le premier bloc du schéma, « embeddings + encodage de position », se voit bien dans BERT. Chaque token reçoit trois vecteurs qui sont additionnés : celui du token, celui de sa phrase (A ou B) et celui de sa position. Le jeton `[CLS]` sert à résumer toute l'entrée pour la classification, et `[SEP]` sépare les phrases.

Multi-têtes signifie que l'attention est calculée plusieurs fois en parallèle (8 têtes dans l'article original), avec des poids différents. Chaque tête peut se spécialiser : l'une suit l'accord sujet-verbe, une autre relie les pronoms à leur nom.

	Seq2Seq LSTM (chapitre 16)	Transformer
Lecture de la source	Mot par mot	Tous les mots en même temps
Lien entre la source et la cible	Vecteur c, puis attention	Attention croisée à chaque couche
Ordre des mots	Donné par la récurrence	Donné par l'encodage de position
Profondeur	1 ou 2 couches	6 couches ou plus de chaque côté

17.4 Attention globale, locale et causale

Tous les Transformers n'autorisent pas chaque mot à regarder tous les autres. On choisit, avec un **masque**, quels mots chaque mot a le droit de regarder.

Image — Trois masques d'attention sur la phrase « Les étudiants qui ont suivi le cours …

Attention	Chaque mot regarde	Coût pour n mots	Utilisation
Globale	Tous les mots	n × n scores	Encodeurs comme BERT
Locale (fenêtre glissante)	Ses voisins à ± w positions	n × (2w + 1) scores	Textes très longs
Causale (masquée)	Lui-même et les mots précédents	Environ n × n / 2 scores	Décodeurs qui génèrent du texte

- **Globale.** C'est la plus riche, mais son coût croît comme n^2 : 1 000 tokens demandent 1 million de scores par tête et par couche, 10 000 tokens en demandent 100 millions.
- **Locale.** Le coût ne croît plus qu'avec n. En contrepartie, un mot ne voit pas directement les mots lointains : dans la figure, « sont » ne voit pas « étudiants ». L'information passe quand même d'une couche à l'autre, de voisin en voisin.
- **Causale.** Pendant l'entraînement, un modèle qui apprend à prédire le mot suivant ne doit pas pouvoir le lire : sinon il tricherait. Le masque cache donc les mots futurs.

En pratique, on remplace les scores interdits par $-\infty$ avant le softmax ; leur poids devient alors exactement 0.

```
import torch
torch.manual_seed(0)
n = 6
positions = torch.arange(n)
globale = torch.ones(n, n)
locale = ((positions[:, None] - positions[None, :]).abs() <= 2).float()
causale = torch.tril(torch.ones(n, n))

scores = torch.randn(n, n)                                    # scores
d'attention bruts
for nom, masque in [("globale", globale), ("locale", locale), ("causale",
causale)]:
    scores_masques = scores.masked_fill(masque == 0, float("-inf"))
    poids = torch.softmax(scores_masques, dim=-1)               # -inf
devient un poids nul
    print(nom, [round(p, 2) for p in poids[2].tolist()])      # ce que
regarde le mot n° 3
```

Résultat obtenu :

```
globale [0.13, 0.39, 0.35, 0.09, 0.03, 0.02]
locale [0.13, 0.4, 0.35, 0.09, 0.03, 0.0]
causale [0.15, 0.45, 0.4, 0.0, 0.0, 0.0]
```

Le mot n° 3 donne un poids nul au mot n° 6 avec la fenêtre locale (trop loin), et aux mots 4, 5 et 6 avec le masque causal (dans le futur). Les poids restants sont recalculés pour que leur somme reste égale à 1.

17.5 Trois familles : Encoder-only, Decoder-only, Encoder-Decoder

Les modèles modernes gardent soit l'encodeur, soit le décodeur, soit les deux. Ce choix décide de ce que le modèle sait faire.

Image — Les trois familles de Transformers : encodeur seul, décodeur seul, encodeur et …

Famille	Attention	Entraînement typique	Exemples	Usages
Encoder-only	Globale : chaque mot voit toute la phrase	Deviner des mots cachés dans une phrase	BERT, DistilBERT, CamemBERT	Classification, entités nommées, questions-réponses extractives, embeddings
Decoder-only	Causale : chaque mot voit les mots précédents	Prédire le mot suivant	GPT, Llama, Mistral	Génération de texte, assistants conversationnels
Encoder-Decoder	Globale dans l'encodeur ; causale et croisée dans le décodeur	Transformer une phrase en une autre	Transformer original, T5, BART, MarianMT	Traduction, résumé

Les modèles d'embeddings sont le plus souvent encoder-only. Pour résumer le sens d'une phrase dans un vecteur, chaque mot doit voir toute la phrase, avant et après lui : c'est l'attention globale. Le modèle n'a pas besoin de générer de texte, donc pas de décodeur. Les modèles Sentence Transformers de l'exemple 8 (section 14.5) sont construits ainsi, et DistilBERT, utilisé en TP pour la classification, est lui aussi un encodeur.

17.6 La tokenisation : un problème « à un million de dollars »

Le tokenizer décide de ce que le modèle voit. Il est fixé avant l'entraînement et ne peut plus changer sans tout réentraîner. Un mauvais découpage coûte donc à chaque phrase lue, à chaque entraînement et à chaque utilisation : d'où l'expression de problème « à un million de dollars ».

Trois façons de découper le mot « inscriptions ».

Découpage	Résultat	Avantage	Problème
En mots	[inscriptions]	Chaque token a un sens	Vocabulaire énorme, mots inconnus (section 12.6)
En caractères	[i, n, s, c, r, i, p, t, i, o, n, s] : 12 tokens	Aucun mot inconnu, une centaine de symboles	Séquences très longues, un caractère a peu de sens
En sous-mots	[in, scription, s]	Les mots fréquents restent entiers, les rares sont découpés	Le découpage n'est pas toujours linguistique

Les modèles actuels utilisent des sous-mots, avec trois algorithmes principaux : **BPE** (GPT, Llama), **WordPiece** (BERT, DistilBERT, qui marque les morceaux suivants par `##`) et **Unigram** via SentencePiece (T5, CamemBERT).

Le principe de BPE (*Byte Pair Encoding*) : partir des caractères, puis fusionner plusieurs fois la paire de symboles voisins la plus fréquente du corpus.

```
from collections import Counter

corpus = ["inscription", "inscriptions", "inscrire", "description",
"prescription"]
mots = [list(m) + ["</w>"] for m in corpus]          # au départ : des
caractères

def paire_la_plus_frequente(mots):
    paires = Counter()
    for m in mots:
        for a, b in zip(m, m[1:]):
            paires[(a, b)] += 1
    return paires.most_common(1)[0][0]

def fusionner(mot, paire):
    resultat, i = [], 0
```

```
    while i < len(mot):
        if i < len(mot) - 1 and (mot[i], mot[i + 1]) == paire:
            resultat.append(mot[i] + mot[i + 1])
            i += 2
        else:
            resultat.append(mot[i])
            i += 1
    return resultat

for etape in range(10):
    paire = paire_la_plus_frequente(mots)
    mots = [fusionner(m, paire) for m in mots]
    print(etape + 1, paire)
for m in mots:
    print(m)
```

Résultat obtenu (fin de l'affichage) :

```
9 ('i', 'n')
10 ('scription', '</w>')
['in', 'scription</w>']
['in', 'scription', 's', '</w>']
['in', 'scri', 'r', 'e', '</w>']
['d', 'e', 'scription</w>']
['p', 'r', 'e', 'scription</w>']
```

Après dix fusions, le sous-mot « scription » est apparu. Le modèle peut relier « inscription », « description » et « prescription », et un mot jamais vu comme « souscription » sera découpé en morceaux connus au lieu de devenir `<unk>`.

Les enjeux.

Enjeu	Le problème	Conséquence
Linguistique	Les tokenizers sont surtout entraînés sur de l'anglais. Les mots des langues peu dotées (fon, yoruba, wolof…) et ceux qui portent des tons ou des accents sont découpés en nombreux petits morceaux sans sens	Le modèle comprend moins bien ces langues
Technique	Plus de tokens par mot donne des séquences plus longues	Moins de texte tient dans la fenêtre de contexte ; le coût de l'attention croît comme n^2 (section 17.4)

Enjeu	Le problème	Conséquence
Économique	Les API sont facturées au token, et l'entraînement coûte selon le nombre de tokens	Le même texte coûte plus cher dans certaines langues : on parle de « taxe des tokens »

On mesure le problème avec la **fertilité** : le nombre moyen de tokens par mot. Une étude de 2025 sur 10 grands modèles et 16 langues africaines (Lundin et al., *The Token Tax*) montre qu'une fertilité plus élevée prédit toujours une précision plus faible. Les auteurs rappellent aussi que doubler le nombre de tokens multiplie par quatre le coût et le temps d'entraînement.

Calculer la fertilité d'un tokenizer (code non exécuté ici, il télécharge le tokenizer) :

```
from transformers import AutoTokenizer

tokenizer = AutoTokenizer.from_pretrained("bert-base-multilingual-cased")

def fertilite(phrase):
    return len(tokenizer.tokenize(phrase)) / len(phrase.split())

print(tokenizer.tokenize("Les inscriptions commencent lundi."))
print(round(fertilite("Les inscriptions commencent lundi."), 2))
```

À vous : calculer la fertilité d'une même phrase en français, en anglais et dans une langue locale que vous connaissez. Quelle langue « paie » le plus de tokens ?

17.7 Utiliser un modèle préentraîné avec Hugging Face

La bibliothèque `transformers` donne accès à des milliers de modèles déjà entraînés.

Traduction :

```
from transformers import pipeline

traducteur = pipeline("translation", model="Helsinki-NLP/opus-mt-fr-en")
print(traducteur("Le chat noir dort sur le lit."))
# [{'translation_text': 'The black cat sleeps on the bed.'}]
```

Analyse de sentiments :

```
analyse = pipeline("sentiment-analysis",
                   model="nlptown/bert-base-multilingual-uncased-sentiment")
print(analyse("Ce cours est vraiment passionnant !"))
```

17.8 Le fine-tuning

Affiner (*fine-tuning*) un modèle, c'est continuer son entraînement sur nos propres données. Le modèle connaît déjà la langue ; il apprend seulement notre tâche.

```
from transformers import AutoTokenizer, AutoModelForSequenceClassification
from transformers import Trainer, TrainingArguments
from datasets import Dataset

# 1. Petites données d'exemple : 1 = positif, 0 = négatif
donnees = Dataset.from_dict({
    "text": ["J'adore ce film", "Quel ennui", "Très bon livre", "Je déteste
attendre"],
    "label": [1, 0, 1, 0],
})

# 2. Modèle et tokenizer préentraînés
nom = "distilbert-base-multilingual-cased"
tokenizer = AutoTokenizer.from_pretrained(nom)
modele = AutoModelForSequenceClassification.from_pretrained(nom,
num_labels=2)

# 3. Transformer les textes en nombres
donnees = donnees.map(lambda x: tokenizer(x["text"], truncation=True,
                                          padding="max_length",
max_length=32))

# 4. Entraîner
args = TrainingArguments(output_dir="resultats", num_train_epochs=3)
trainer = Trainer(model=modele, args=args, train_dataset=donnees)
trainer.train()

modele.save_pretrained("mon_modele")
tokenizer.save_pretrained("mon_modele")
```

En TP, on utilise un vrai jeu de données de quelques milliers d'avis, et on garde une partie des données pour l'évaluation.

17.9 Intégrer le modèle dans une application

On place le modèle dans une fonction, puis on l'utilise dans un programme.

```
# traduction.py
from transformers import pipeline

traducteur = pipeline("translation", model="Helsinki-NLP/opus-mt-fr-en")

def traduire(phrase):
    if phrase.strip() == "":
        return ""
    return traducteur(phrase)[0]["translation_text"]
```

```
# application.py
from traduction import traduire

while True:
    phrase = input("Phrase en français (vide pour quitter) : ")
    if phrase == "":
        break
    print("Traduction :", traduire(phrase))
```

Projet

Par binôme : construire une petite application (traducteur, détecteur de sentiments ou extracteur d'entités) à partir d'un modèle préentraîné ou affiné. Évaluer le modèle.

Chapitre 18 — Évaluer et pratiquer

Objectifs : choisir et calculer les métriques adaptées à une tâche de compréhension, lire une matrice de confusion, mener les travaux pratiques de la partie II.

Prérequis : chapitres 12 à 17.

18.1 Les métriques de compréhension

Un modèle n'est utile que si l'on sait mesurer sa qualité sur des données qu'il n'a jamais vues. Pour une classe donnée, on compte les vrais positifs (VP : bien classés dans la classe), les faux positifs (FP : mis dans la classe à tort) et les faux négatifs (FN : oubliés).

Métrique	Ce qu'elle mesure	Calcul	Quand l'utiliser
Accuracy (exactitude)	La part de bonnes réponses	bonnes réponses / total	Classes de tailles proches
Précision	Parmi les textes mis dans la classe, la part correcte	VP / (VP + FP)	Quand une fausse alerte coûte cher

Métrique	Ce qu'elle mesure	Calcul	Quand l'utiliser
Rappel	Parmi les textes vraiment dans la classe, la part retrouvée	VP / (VP + FN)	Quand un oubli coûte cher
F1	Le compromis entre précision P et rappel R	2 × P × R / (P + R)	Classes déséquilibrées
Exact Match	La réponse est-elle identique à la référence ?	1 si identique, 0 sinon, puis moyenne	Questions-réponses extractives
Similarité cosinus	La proximité de deux vecteurs de sens	cos(u, v), entre −1 et 1	Recherche sémantique

18.2 Exemple : classer les questions des étudiants

Un service de scolarité veut ranger automatiquement les questions reçues dans quatre catégories : bourse, inscription, examen, paiement. On teste le modèle sur 20 questions dont on connaît la vraie catégorie.

```
from sklearn.metrics import accuracy_score, classification_report,
confusion_matrix, f1_score

vraies = ["bourse", "bourse", "bourse", "inscription", "inscription",
"inscription", "inscription",
          "inscription", "inscription", "examen", "examen", "examen",
"examen", "examen",
          "paiement", "paiement", "paiement", "paiement", "paiement",
"paiement"]
predites = ["bourse", "paiement", "bourse", "inscription", "inscription",
"inscription", "examen",
            "inscription", "inscription", "examen", "examen", "inscription",
"examen", "examen",
            "paiement", "paiement", "bourse", "paiement", "paiement",
"paiement"]

print("Accuracy :", accuracy_score(vraies, predites))
print("F1 macro :", round(f1_score(vraies, predites, average="macro"), 3))
print(classification_report(vraies, predites, digits=2))
categories = ["bourse", "inscription", "examen", "paiement"]
print(confusion_matrix(vraies, predites, labels=categories))
```

Résultat obtenu :

```
Accuracy : 0.8
F1 macro : 0.783
              precision    recall  f1-score   support

      bourse       0.67      0.67      0.67         3
      examen       0.80      0.80      0.80         5
 inscription       0.83      0.83      0.83         6
    paiement       0.83      0.83      0.83         6

    accuracy                           0.80        20
   macro avg       0.78      0.78      0.78        20
weighted avg       0.80      0.80      0.80        20

[[2 0 0 1]
 [0 5 1 0]
 [0 1 4 0]
 [1 0 0 5]]
```

Image — Matrice de confusion des 20 questions : la diagonale compte les bonnes réponses

Lire la matrice. Chaque ligne est une vraie catégorie, chaque colonne une catégorie prédite. La diagonale compte les bonnes réponses (2 + 5 + 4 + 5 = 16, soit une accuracy de 16 / 20 = 0,80). Hors de la diagonale, on voit les confusions : bourse et paiement se mélangent dans les deux sens, comme inscription et examen. C'est logique : une question comme « Quand ma bourse sera-t-elle versée ? » parle aussi d'argent.

Calcul à la main pour la classe bourse. VP = 2 (case en haut à gauche). FP = 1 (une question de paiement classée bourse, colonne bourse). FN = 1 (une question de bourse classée paiement, ligne bourse). Précision = 2 / 3 = 0,67, rappel = 2 / 3 = 0,67, F1 = 0,67. C'est la classe la moins bien traitée, et aussi celle qui a le moins d'exemples.

Pourquoi le F1 macro ? Le F1 macro est la moyenne simple des F1 des quatre classes : (0,67 + 0,80 + 0,83 + 0,83) / 4 = 0,78. Chaque classe pèse autant, qu'elle soit grande ou petite. Imaginons 100 questions dont 90 portent sur le paiement, et un modèle paresseux qui répond toujours « paiement ». Son accuracy vaut 0,90, ce qui semble excellent. Mais son F1 vaut 0,95 pour paiement et 0 pour les trois autres classes : le F1 macro tombe à 0,24 et révèle le problème.

Recommandation. Pour une tâche de classification, rapporter ensemble l'accuracy, la précision, le rappel et le F1. Si les classes sont déséquilibrées, utiliser le F1 macro comme métrique principale. Toujours regarder la matrice de confusion pour savoir quelles classes sont confondues.

18.3 Exact Match et similarité cosinus

Exact Match (EM) sert aux questions-réponses extractives, où la réponse est un morceau du texte. On normalise d'abord les deux réponses (minuscules, espaces, point final), puis on les compare.

```
def exact_match(prediction, reference):
    normaliser = lambda t: " ".join(t.lower().strip(" .").split())
    return int(normaliser(prediction) == normaliser(reference))

print(exact_match("Le 15 novembre.", "le 15 novembre"), exact_match("15
novembre", "le 15 novembre"))
```

Résultat obtenu : `1 0`. La deuxième réponse est juste mais obtient 0 à cause du seul mot « le » : EM est très sévère. On l'accompagne souvent d'un F1 calculé sur les mots communs.

La similarité cosinus mesure l'angle entre deux vecteurs : 1 pour la même direction, 0 pour des vecteurs sans rapport. C'est la mesure de base de la recherche sémantique.

```
import numpy as np

def cosinus(u, v):
    return np.dot(u, v) / (np.linalg.norm(u) * np.linalg.norm(v))

chat = np.array([0.9, 0.8, 0.1])
chien = np.array([0.8, 0.9, 0.2])
voiture = np.array([0.1, 0.2, 0.9])
print(round(cosinus(chat, chien), 3), round(cosinus(chat, voiture), 3))
```

Résultat obtenu : `0.99 0.303`. « chat » est bien plus proche de « chien » que de « voiture ».

18.4 Travaux pratiques

Les TP suivent l'ordre des chapitres. Chacun se termine par une évaluation chiffrée et une courte analyse écrite des erreurs.

TP	Thème	Outils	Chapitres	Métriques
TP1	Segmentation et compréhension linguistique	spaCy	12	Comptage d'erreurs
TP2	Embeddings et similarité sémantique	Gensim, Sentence Transformers	14	Similarité cosinus
TP3	RNN et LSTM	Keras ou PyTorch	15	Accuracy, F1

TP	Thème	Outils	Chapitres	Métriques
TP4	Classification avec DistilBERT	Hugging Face transformers	17, 18	Accuracy, précision, rappel, F1, matrice de confusion
TP5	Traduction Seq2Seq	PyTorch, sacrebleu	16	BLEU, chrF, analyse qualitative
Mini-projet	Moteur de recherche sémantique	scikit-learn, Sentence Transformers	13, 14, 18	Bonne réponse en première position

TP1 — Segmentation et compréhension linguistique (spaCy). Objectif : parcourir les quatre étapes du TALN sur un vrai texte.

1. Prendre un texte administratif de l'université (un avis d'inscription, une page de règlement).
2. Le segmenter en phrases et en tokens ; afficher pour chaque token le lemme, la catégorie et la dépendance (sections 12.3 et 12.4).
3. Extraire les entités nommées (dates, lieux, organisations) et les sujets des verbes.
4. À rendre : un tableau des erreurs du modèle `fr_core_news_sm` (entités ratées ou mal classées) et une explication de leur cause.

TP2 — Embeddings et similarité sémantique (Gensim, Sentence Transformers). Objectif : comparer embeddings statiques et contextuels.

1. Entraîner Word2Vec sur un petit corpus et utiliser `most_similar` ; refaire l'exercice avec les vecteurs GloVe préentraînés (chapitre 14).
2. Visualiser les vecteurs avec t-SNE et commenter les groupes obtenus.
3. Avec Sentence Transformers, calculer la similarité cosinus de 10 paires de questions d'étudiants, dont certaines de même sens sans mot commun (exemple 8).
4. À rendre : un tableau des similarités et un paragraphe sur le cas de « avocat » (section 14.5).

TP3 — RNN et LSTM (Keras ou PyTorch). Objectif : mesurer l'effet de la longueur des phrases.

1. Classer le sentiment de phrases courtes et longues (par exemple des avis de films).
2. Entraîner un RNN simple, puis un LSTM, avec les mêmes embeddings et le même nombre d'époques.
3. Calculer l'accuracy et le F1 séparément pour les phrases de moins de 10 mots et de plus de 30 mots.

4. À rendre : les courbes de perte et un tableau RNN / LSTM × courtes / longues, commenté à l'aide de la section 15.6.

TP4 — Classification avec DistilBERT. Objectif : affiner un modèle préentraîné et l'évaluer correctement.

1. Constituer un jeu de questions d'étudiants réparties en quatre catégories (bourse, inscription, examen, paiement), ou reprendre un jeu d'avis.
2. Séparer entraînement (80 %) et test (20 %).
3. Créer une base de comparaison simple : TF-IDF suivi d'une régression logistique.
4. Affiner `distilbert-base-multilingual-cased` comme à la section 17.8.
5. À rendre : accuracy, précision, rappel, F1 (macro si les classes sont déséquilibrées) et matrice de confusion pour les deux modèles, avec une analyse des confusions (section 18.2).

TP5 — Traduction Seq2Seq. Objectif : entraîner et évaluer un petit traducteur.

1. Suivre l'organisation de la section 16.7 : corpus parallèle FR–EN, embeddings de taille 128, encodeur et décodeur LSTM, *teacher forcing*.
2. Comparer une version à une couche et une version à deux couches.
3. Calculer BLEU et chrF sur des phrases de test jamais vues (section 16.8).
4. Analyse qualitative : lire 20 traductions et classer les erreurs (mot inconnu, ordre des mots, ambiguïté, phrase longue).
5. À rendre : scores, exemples réussis et ratés, et une comparaison avec un modèle préentraîné (section 17.7).

Mini-projet — Moteur de recherche sémantique. Objectif : répondre aux questions des étudiants en retrouvant le bon document d'une foire aux questions.

Étape 1 : une version TF-IDF, rapide à écrire (chapitre 13).

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

documents = ["Les demandes de bourse se font en ligne avant le 30
septembre.",
             "Le paiement des frais d'inscription se fait par mobile money.",
             "Les examens du premier semestre commencent en janvier.",
             "Pour s'inscrire, il faut le relevé de notes du baccalauréat."]

question = "Quand faut-il demander une bourse ?"

tfidf = TfidfVectorizer()
X = tfidf.fit_transform(documents + [question])
scores = cosine_similarity(X[-1], X[:-1]).ravel()
for i in scores.argsort()[::-1]:
    print(round(scores[i], 2), documents[i])
```

Résultat obtenu :

```
0.21 Pour s'inscrire, il faut le relevé de notes du baccalauréat.
0.09 Les demandes de bourse se font en ligne avant le 30 septembre.
0.0 Les examens du premier semestre commencent en janvier.
0.0 Le paiement des frais d'inscription se fait par mobile money.
```

Le moteur se trompe : il classe en tête le document sur l'inscription, qui partage les mots « il » et « faut » avec la question. Pour TF-IDF, « demander » et « demandes » sont deux mots différents : seul « bourse » relie la question au bon document.

Étape 2 : une version sémantique avec Sentence Transformers (chapitre 14, exemple 8).

```
from sentence_transformers import SentenceTransformer, util

modele = SentenceTransformer("paraphrase-multilingual-MiniLM-L12-v2")
vecteurs_docs = modele.encode(documents)

def rechercher(question):
    vecteur_question = modele.encode(question)
    scores = util.cos_sim(vecteur_question, vecteurs_docs)[0]
    for i in scores.argsort(descending=True):
        print(round(scores[i].item(), 2), documents[i])

rechercher("Quand faut-il demander une bourse ?")
```

Résultat attendu : le document sur la bourse arrive en première position, car la question et le document ont le même sens.

Étape 3 : évaluer. Écrire 20 questions de test, chacune avec le numéro du bon document, puis calculer la part des questions dont le bon document est en première position, pour TF-IDF et pour Sentence Transformers. Ajouter au moins cinq questions sans mot commun avec leur document (par exemple « aide financière » pour « bourse »).

À rendre : le code, le tableau des deux scores et trois exemples où les deux moteurs ne donnent pas la même réponse, commentés.

Compétences acquises : choisir une métrique adaptée à la tâche, calculer précision, rappel et F1 à la main et en Python, lire une matrice de confusion, évaluer un traducteur avec plusieurs métriques et analyser ses erreurs.